DRAFT

From Primitive Provenance to Mathematical Structure

Supplementary Material: Nodewise Derivations, Proof Obligations, and Code Traceability

Editor: Jan Seeck

Current draft: 6 August 2026

Cryptographic binding to the current main paper

This supplementary document applies to the exact PDF file `derivation/paper/main/paper.pdf`. Its SHA-256 digest is

`dd219a5f21266ec6509fde1271ac425fcbfe1c5c5e5e540a89bb1e4d5955e939`.

Any change to the main-paper PDF requires regeneration of this supplementary PDF and this digest.

Conceptual scope: generalized open provenance systems

The general research object is an **open provenance system**: an observed present is treated as a reduced view of a real, historically grown provenance state. The current CNNA DAG is one finite deterministic specialization, not a proof that every empirical open system has this form.

A generic state is organized schematically as

$$\mathfrak{P}_n = (\Gamma_n, \mathsf{Rec}_n, \mathsf{Live}_n, \mathcal{K}_n, \mathcal{C}_n, \mathcal{O}_n).$$

The intended roles are: event provenance, immutable birth record, mutable live response, response kernel, cut-relative complement, and observation/coarse-graining channel. The weak hypothesis allows intrinsic stochasticity after provenance completion. The strong CNNA hypothesis seeks deterministic completion. **The strong universal claim is not currently proved.**

Specialization map

Layer	DAG locus	Current status
Event-provenance carrier and order	C003, C018, C005, C004	finite CNNA definitions/results
Cut-relative complement and exact response	M001, C006, P001, C007, C020–C025	finite directed Schur/DtN specialization; nodewise status applies
Response-coupled event generation	M003, M004, C008	M003/M004 verified; state mutation remains C008
Record/live and backreaction	C008, C016, C017, C024, C043	downstream construction

Layer	DAG locus	Current status
Nested cuts, refinement and cocycle	C044, C045, C047, C046, C043	planned/partially migrated according to node status
Finite provenance-derived POVM	C025, M030, C032, C033, C053, C044, C047	Legacy migration target; not yet a current DAG theorem
Open quantum process/instrument specialization	C043, C053, C057, C060 and later dedicated nodes	conceptual target; no universal OQS theorem claimed
Weyl/GNS/Araki–Woods and modular/AQFT	Sections 5–8	late mathematical specialization; nodewise proof gates remain authoritative

Non-identifications

- Schur/DtN elimination is an effective response reduction, not a partial trace.
- A POVM supplies outcome probabilities; an instrument additionally supplies outcome-conditioned state updates.
- A process tensor or quantum comb is a quantum multi-time specialization, not the definition of provenance itself.
- Event provenance may remain acyclic even when the live state has recurrent or cyclic dynamics.

The reduced-dynamics ambiguity under initial correlations motivates a provenance-dependent reconstruction map. **EXT-REF-OPS-PECHUKAS-001 — specialization context.** Philip Pechukas, *Reduced Dynamics Need Not Be Completely Positive*, Physical Review Letters 73(8) (1994), 1060–1062. DOI: 10.1103/PhysRevLett.73.1060. Exact location: principal result. Context: Initial correlations and assignment-map qualification of reduced dynamics. Formal status: INTERPRETIVE_SPECIALIZATION_CONTEXT_NOT_IMPORTED_AS_AXIOM

The process-tensor framework is the primary quantum reference for operational multi-time memory. **EXT-REF-OPS-POLLOCK-001 — specialization context.** Felix A. Pollock, César Rodríguez-Rosario, Thomas Frauenheim, Mauro Paternostro, and Kavan Modi, *Non-Markovian quantum processes: Complete framework and efficient characterization*, Physical Review A 97(1) (2018), 012127. DOI: 10.1103/PhysRevA.97.012127. Exact location: Abstract and process-tensor construction. Context: Operational multi-time memory and process statistics. Formal status: INTERPRETIVE_SPECIALIZATION_CONTEXT_NOT_IMPORTED_AS_AXIOM

The Davies–Lewis instrument framework is the target distinction between probabilities and post-measurement states. **EXT-REF-OPS-DAVIESLEWIS-001 — specialization context.** E. B. Davies and J. T. Lewis, *An operational approach to quantum probability*, Communications in Mathematical Physics 17(3) (1970), 239–260. DOI: 10.1007/BF01647093. Exact location: operational instrument framework. Context: Outcome probabilities and outcome-conditioned state updates. Formal status: INTERPRETIVE_SPECIALIZATION_CONTEXT_NOT_IMPORTED_AS_AXIOM

The quantum-network/comb formalism is retained as context for later compositional specialization. **EXT-REF-OPS-CHIRIBELLA-001 — specialization context.** Giulio Chiribella, Giacomo Mauro D'Ariano, and Paolo Perinotti, *Theoretical framework for quantum networks*, Physical Review A 80(2) (2009), 022339. DOI: 10.1103/PhysRevA.80.022339. Exact location: quantum-network and comb framework. Context: Compositional multi-step quantum specialization. Formal status: INTERPRETIVE_SPECIALIZATION_CONTEXT_NOT_IMPORTED_AS_AXIOM

Legacy POVM migration guard

The retained Legacy calculation constructed a finite Parseval-frame POVM from a compressed response state,

$$E_v = z_v z_v^{\mathsf{T}}, \qquad \sum_v E_v = I, \qquad p_v = \mathrm{tr}(D_N E_v).$$

It is evidence and a migration specification. It did **not** establish exact projective compatibility of independently reconstructed depths, an infinite state limit, a lift to the raw Weyl net, or a full outcome-conditioned instrument. Any new DAG nodes for this chain must be derived from current predecessors rather than importing Legacy coefficients or numerical fallback rules.

General completion hypothesis

For an empirically accessible open system S, the research program asks whether there exist a provenance-complete extension $\widehat{S}$, a retained region B, a complement-elimination operation Eff_{I_B}, and an observation channel Q_B such that the complete family of observable process statistics satisfies

$$\mathbb{P}_S = \mathbb{P}_{Q_B \circ \mathrm{Eff}_{I_B}(\widehat{S})}.$$

The nontrivial content is not merely that the system has a past. It is that the dynamically sufficient part of that past remains a real component of the present state and can alter future responses even when two reduced present descriptions coincide.

Let X_n denote a reduced present and Π_n its dynamically relevant provenance. The target sufficient state is

$$Z_n = (X_n, \Pi_n), \qquad \mathbb{P}(X_{n+1} \mid X_{\le n}, a_{\le n}) = \mathbb{P}(X_{n+1} \mid Z_n, a_n).$$

This equation defines a target property. The present DAG establishes it only for those finite deterministic update components that are explicitly formalized; it does not establish the quantifier over all empirical systems.

Weak and strong forms

The **weak completion hypothesis** allows an intrinsically stochastic update kernel

$$Z_{n+1} \sim \mathcal{U}_n(\,\cdot \mid Z_n, a_n).$$

It claims only that omitted history is no longer an additional source of non-Markovianity once the sufficient provenance state is included.

The **strong deterministic CNNA hypothesis** instead seeks

$$Z_{n+1} = \mathcal{U}_n(Z_n, a_n),$$

or $Z_{n+1} = \mathcal{U}_n(Z_n)$ in the absence of an external intervention. This strong form is an ontological hypothesis over and above the current finite proofs.

Cut-relative environment and coherent refinement

For one complete finite carrier V_n and retained region $B_n \subseteq V_n$, define

$$I_{B_n} = V_n \setminus B_n.$$

The environment is therefore a role relative to a cut. If $B_1 \subseteq B_2$, then $I_{B_2} \subseteq I_{B_1}$: a coordinate treated as environment at one resolution may be retained at a finer resolution.

A generalized open-provenance theory must consequently prove more than the existence of an effective response at each isolated cut. For nested retained regions $B_1 \subseteq B_2 \subseteq B_3$, the

corresponding reductions should satisfy a compositional law whenever all domains are admissible,

$$\mathrm{Eff}_{B_3\setminus B_1} = \mathrm{Eff}_{B_2\setminus B_1} \circ \mathrm{Eff}_{B_3\setminus B_2},$$

or else expose the controlled defect as a refinement cocycle. C044, C045, C047, C046, and C043 are the designated DAG locus for this requirement.

Record/live divergence and backreaction

The immutable record and mutable live state have different mathematical roles:

$$\mathsf{Rec}_{n+1} = \mathsf{Rec}_n \cup \{r_{n+1}\}, \qquad \mathsf{Live}_{n+1} = \mathcal{U}_n(\mathsf{Live}_n, r_{n+1}, \Lambda_n).$$

A record stores the conditions under which a relation was born. The live channel stores how that relation currently acts after later growth. Their difference is not treated as bookkeeping noise; it is the prospective carrier of backreaction and effective memory. C008 owns the update, C016 the immutable record, C017 the live channel, C024 the backreaction stream, and C043 its multiscale reconstruction.

Non-Markovianity and identifiability

In this framework Markovianity is relative to the chosen state description. A reduced state can be non-Markovian because two provenance states $\Pi_n^{(1)} \neq \Pi_n^{(2)}$ yield the same X_n but different future response laws. This motivates, but does not prove, the interpretation

$$\text{effective memory} = \text{dynamically visible unresolved provenance}.$$

The corresponding inverse problem is identifiability. Provenances Π_1 and Π_2 are observationally equivalent at a chosen intervention class if they induce the same complete family of process statistics. The scientifically meaningful reconstruction target may therefore be a provenance-equivalence class rather than a unique microscopic history.

Specialization ladder

The intended order is one-way and derived-only:

$$\begin{aligned}&\text{event provenance and response-coupled growth}\\ &\longrightarrow \text{cut-relative effective response}\\ &\longrightarrow \text{record/live and nested-channel coherence}\\ &\longrightarrow \text{response algebra and positive state structures}\\ &\longrightarrow \text{finite POVM and quantum-process specializations}\\ &\longrightarrow \text{Weyl/GNS/Araki–Woods and modular/AQFT structures}.\end{aligned}$$

The reverse interpretation is not licensed: provenance is not defined retrospectively from a quantum state, and a quantum reduction is not assumed in order to derive the earlier response structure.

Falsification and obstruction gates

The generalized program must be weakened or rejected for a proposed system class if any of the following persists after all admissible extensions are considered:

1. no finite or controlled provenance-sufficient state reproduces the observable multi-time statistics;
2. effective reductions for nested cuts cannot be made coherent and no controlled cocycle accounts for their defect;

3. record/live separation fails to define invariant birth data and a well-defined current state;
4. two supposedly equivalent provenance representatives yield different physical outputs after the registered quotient;
5. the finite POVM migration cannot derive positivity and completeness from current predecessors without importing Legacy fallback choices;
6. the OQS/AQFT specialization requires independent physical assumptions that are falsely presented as consequences of primitive provenance.

These are scientific obstruction criteria, not merely software tests.

Traceability and reading convention

Three coordinates, one scientific object

Every scientific node is identified by three distinct coordinates, which must not be conflated:

1. **Canonical node number** `NNN`, for example `001`. This is the stable global presentation label and follows the canonical derivation order.
2. **Semantic node ID**, for example `I001` or `C001`. This identifies the scientific owner of a definition, construction, theorem, control, or obstruction.
3. **Current section path**, for example `1.1.1`. This records only the node's present location in the document hierarchy. It may change when a section is inserted or reorganized.

The visible node label is therefore `NNN · ID`, for example `001 · I001`. The section path is deliberately excluded from that label. Consequently, inserting a new section can change `1.1.1` without changing either `001` or `I001`.

One DAG and its document projections

`derivation/registry/dag/NODES.tsv` and `derivation/registry/dag/EDGES.tsv` are the canonical semantic graph. The yEd Live GraphML is the canonical visual layout of the same graph. Main-paper sections, supplementary sections, directory indices, and code-anchor tables are generated projections; none of them constitutes a second DAG.

For each node, `derivation/registry/nodes/<ID>.json` records its canonical number, semantic ID, current section path, documentation tier, incoming and outgoing ownership, document artifacts, and code anchors.

Directory correspondence

The final directory component for a node has the stable form

```
NNN_ID__descriptive_slug
```

For example:

```
001_I001__branching_parameter_b
```

The parent directories encode the current section hierarchy. They may change if the paper is reorganized; the final node-directory component remains tied to `NNN · ID`.

The main-paper and supplementary paths are mirrored:

```
derivation/paper/main/sections/<section parents>/NNN_ID__slug/SECTION.tex
derivation/supplement/sections/<section parents>/NNN_ID__slug/DOCUMENTATION.md
```

Relation to Python and Lean

The semantic ID is the authoritative join key between the DAG, prose, Python, and Lean.

Python modules use a local module-order prefix and the lower-case semantic ID:

```
sKK_id__descriptive_slug.py
```

Lean modules use the corresponding local module-order prefix and the upper-case semantic ID:

```
SKK_ID_DescriptiveName.lean
```

Here `KK` is the order inside the current code section. It is not the canonical node number `NNN`. A reader must therefore not infer the scientific identity from `S01`, `S02`, and so forth alone.

The exact cross-layer lookup is provided by `derivation/registry/documentation/CODE_ANCHORS.tsv`. Each code anchor contains:

- node number and semantic ID;
- source layer and role;
- exact source path;
- declaration or function symbol;
- declaration kind;
- start and end lines;
- exact source SHA-256.

The symbol plus source hash is the stable code identity. Line numbers are a direct reading aid and must be regenerated after any source edit.

Documentation tiers and completion state

The per-node tier `D0`, `D1`, or `D2` specifies the required depth: atomic definition, structural construction, or full proof dossier.

A node is marked `COMPLETE_V2` only when its main-paper text, supplementary dossier, countercheck, code anchors, artifact hashes, and validation gates agree. Nodes are read in canonical node order.

Reproducible lookup procedure

From a DAG node to the scientific text and code:

1. Read `NNN · ID` from the node.
2. Locate `ID` in `NODES.tsv` or `NODE_TRACEABILITY_INDEX.tsv`.
3. Open the registered main-paper or supplementary artifact.
4. Filter `CODE_ANCHORS.tsv` by `ID` to obtain exact Python and Lean symbols, line intervals, and source hashes.
5. Verify the theorem or implementation status in `derivation/registry/nodes/<ID>.json` and the relevant build or axiom audit.

From a code declaration back to the DAG:

1. Use the semantic ID in the module name when present.
2. Confirm the source path and symbol in `CODE_ANCHORS.tsv`.
3. Follow the matching row to `NNN · ID`, the current section path, and the node record.

This correspondence separates scientific identity from mutable editorial placement and from local implementation ordering. It is the basis for all D0, D1, and D2 documentation.

---

001 · I001 — Branching parameter b

Canonical node label: `001 · I001`
Semantic ID: `I001`
Current section path: `1.1.1`
Documentation tier: `D0`
Node role: free structural input
Verification status: Python tests reproduced; Lean source kernel-built in the current prefix-free 26-core-job package.

Position in the derivation

`I001` is the first node in the canonical derivation order and one of the three visible origin nodes. It has no incoming edge. Its local responsibility is limited to the branching multiplicity that will later parameterize the address grammar. It neither depends on the empty carrier `C001` nor creates any carrier state.

This separation is intentional: a free parameter and an initial object may coexist at the origin without one being derived from the other. The single DAG records both as inputs to later constructions.

Definition or statement

The scientific contract is the subtype

$$\mathcal{B} = \{b \in \mathbb{N} \mid 2 \leq b\}.$$

A CNNA finite-provenance instance supplies an explicit element `b` of this set. The node asserts no default value and no probability distribution over admissible values.

The lower bound is part of the declared model family. In particular, the documentation does **not** infer that `b = 1` is inconsistent in mathematics; it states only that the unary case lies outside the admissible domain of `I001` and may be considered separately as a comparison or control.

Introduction reason

The child-slot alphabet used by `C003` cannot be defined before its cardinality is known. `I001` is therefore introduced at the origin and handed to `C003` through edge `E001` (`parameterizes_branching`). Keeping the lower-bound proof at this boundary prevents downstream modules from silently assuming `b ≥ 2` as an undocumented side condition.

Construction or encoding

Mathematical representation

The abstract value is a natural number paired with evidence of `2 ≤ b`.

Python representation

`BranchingParameter` is a frozen, slotted dataclass with one stored field, `value`. Construction checks:

1. `type(value) is int`;
2. `value >= 2`.

The use of exact built-in type equality is deliberate. Python implements `bool` as a subclass of `int`; accepting `isinstance(True, int)` would allow a logical flag to enter as the numerical branching value `1`.

Lean representation

Lean uses

```
structure BranchingParameter where
  value : Nat
  ge_two : 2 ≤ value
```

The proposition is stored in the value itself. `lowerBound` is an explicit projection theorem and `mk_value` exposes the constructor equation at the module boundary. There is no default inhabitant and no local axiom declaration.

Cross-layer correspondence

Python and Lean implement the same admissible numerical values but enforce validity differently:

- Python permits attempted invalid construction and rejects it at runtime;
- Lean requires a proof argument before the invalid value can be constructed.

This is semantic equivalence of the accepted domain, not syntactic identity of the implementations.

Boundary case or countercheck

The node-local counterchecks are:

- `b = 2` is admitted as the lower boundary;
- larger integers are admitted without a fixed upper bound;
- `b ∈ {-3,-1,0,1}` is rejected;
- `True`, `False`, `2.0`, `"2"`, and `None` are rejected by the Python type boundary;
- in Lean, any claimed inhabitant automatically carries `2 ≤ value`, so a value below two cannot be produced without inconsistency in the ambient logic.

These tests establish the input boundary only. They do not prove properties of the later `b`-ary carrier, schedule, or growth law.

Result

`I001` closes the following narrow statement:

> One explicit branching multiplicity `b` is available, and every accepted representation satisfies exactly `b ∈ ℕ` and `b ≥ 2`.

No geometry, node, relation, event ordering, conductance, response, or dynamics is introduced. The node is frozen as a verified input contract; its documentation is now complete under schema v2.

Downstream handoff

- `E001: I001 → C003` supplies the branching multiplicity for the finite slot alphabet and address grammar.
- `E147: I001 → P004` supplies the same bound to the later proof that the bounded canonical schedule enumerates the finite carrier.

`E147` is proof support, not an additional definition of `b`.

Code anchors

All line intervals refer to the current source hashes. Symbol plus source SHA-256 is the stable identity; line numbers are reading aids.

Python source

- Path: `derivation/code/python/src/cnna/derivation/s01_primitive_response_coupled_finite_provenance/s01_primitive_inputs_grammar_and_event_order/s01_i001__branching_parameter_b.py`
- Symbol: `BranchingParameter`
- Kind: class
- Lines: 20–46
- SHA-256: `504e678397db5f6f969a5c69a6da9a2a36f34308201b0629b558331b453c1f69`

Python counterchecks

- Path: `derivation/code/python/tests/derivation/s01_primitive_response_coupled_finite_provenance/s01_primitive_inputs_grammar_and_event_order/test_s01_i001__branching_parameter_b.py`
- Symbol: `TestBranchingParameter`
- Kind: test class
- Lines: 12–33
- SHA-256: `189ec3b317faa474ece1d2105eaeb347fee053f210fe0bd24c5e8d10106c7d9c`
- Covered cases: lower boundary, larger values, values below two, non-integer and Boolean rejection.

Lean source

Path: `derivation/code/lean/core/src/CNNA/Derivation/S01_PrimitiveResponseCoupledFiniteProvenance/S01_PrimitiveInputsGrammarAndEventOrder/S01_I001_BranchingParameterB.lean`
SHA-256: `b826f2eca8aa535447d1c5c0090989d838ea30fac381e4c3c925b98c2bd96c96`

- `BranchingParameter`, structure, lines 17–23: value and lower-bound proof.
- `lowerBound`, theorem, lines 24–27: explicit projection `2 ≤ b.value`.
- `mk_value`, theorem, lines 28–34: definitional constructor equation.

---

002 · I002 — Finite approximant depth L

Canonical node label: `002 · I002`
Semantic ID: `I002`
Current section path: `1.1.2`
Documentation tier: `D0`
Node role: free finite-cutoff input
Verification status: Python tests reproduced; Lean source kernel-built in the current prefix-free 26-core-job package.

Position in the derivation

`I002` is the second canonical node and a visible origin input. It has no incoming edge. It supplies a finite terminal depth to later constructions but does not itself construct an address, carrier, root, or schedule.

Its independence from `I001` is exact: branching multiplicity and finite depth are separate coordinates of the finite approximant. Neither determines the other.

Definition or statement

The contract is

$$L \in \mathbb{N}_0.$$

Equivalently, `L` is a nonnegative integer. The value is supplied explicitly, `L = 0` is admissible, and no infinity, `None`, negative value, or special unbounded sentinel belongs to the domain.

This node does not identify `L` with physical time, graph distance, or a continuum coordinate. It is a finite cutoff on provenance-address depth. The interpretation of `L = 0` as a root-only approximant requires the subsequent root and grammar nodes; at `I002` alone it is only the lower endpoint of the input domain.

Introduction reason

The grammar `C003` must know which address depths are admitted, and the later completion node `C019` must know where finite construction terminates. Therefore `I002` is fixed at the origin and transmitted through `E002` and `E081`. Stating finiteness here prevents an implicit infinity convention from entering downstream code.

Construction or encoding

Mathematical representation

The abstract domain is $\mathbb{N}_0$; no additional predicate is required.

Python representation

`FiniteApproximantDepth` is a frozen, slotted dataclass with one `value` field. Its constructor checks exact built-in-integer type and the lower bound `value >= 0`. The value is returned without implicit coercion by `to_int`.

Lean representation

Lean uses a one-field structure:

```
structure FiniteApproximantDepth where
  value : Nat
```

No proof field is necessary because `Nat` already enforces nonnegativity. `mk_value` records the constructor equation, and `zeroAdmissible` proves the existence of an ordinary inhabitant with value zero.

Cross-layer correspondence

The accepted mathematical values agree exactly:

- Python built-in integers at least zero;
- Lean natural numbers.

Python performs a runtime rejection of invalid external values. Lean excludes them by type. The Python rejection of Boolean values is an implementation-boundary refinement needed because Python's class hierarchy differs from Lean's type separation.

Boundary case or countercheck

The node-local checks establish:

- `L = 0` is admitted and round-trips exactly;
- arbitrary positive integers are admitted;
- negative integers are rejected;
- floating-point zero, strings, `None`, and Boolean values are rejected.

The countercheck distinguishes an ordinary zero-depth value from an infinity or missing-value sentinel. No theorem in this node proves convergence as $L \to \infty$; finite-to-infinite completion remains a separate downstream obligation.

Result

`I002` closes the following statement:

> One explicit finite approximant depth is available, with accepted values exactly $\mathbb{N}_0$, including zero and excluding every non-finite sentinel.

The node introduces no temporal, geometric, or dynamical interpretation. Its documentation is complete under schema v2.

Downstream handoff

- `E002: I002 → C003` bounds the admitted provenance addresses.
- `E081: I002 → C019` sets the finite completion depth.
- `E148: I002 → P004` provides the finite cutoff to the later enumeration and termination proof.

These edges reuse the same input; they do not define different meanings of `L`.

Code anchors

Python source

- Path: `derivation/code/python/src/cnna/derivation/s01_primitive_response_coupled_finite_provenance/s01_primitive_inputs_grammar_and_event_order/s02_i002__finite_approximant_depth_l.py`
- Symbol: `FiniteApproximantDepth`
- Kind: class
- Lines: 20–46
- SHA-256: `f0f5fefe55d90835e8deead733579848d547b6b6df7e40b6491a1f578b64b326`

Python counterchecks

- Path: `derivation/code/python/tests/derivation/s01_primitive_response_coupled_finite_provenance/s01_primitive_inputs_grammar_and_event_order/test_s02_i002__finite_approximant_depth_l.py`
- Symbol: `TestFiniteApproximantDepth`
- Kind: test class
- Lines: 12–33
- SHA-256: `8836bb6a9740dc29d66c65ea121d3e3dc2445764b6f28f61e8928b5b6f70772b`
- Covered cases: zero, positive values, negative rejection, non-integer and Boolean rejection.

Lean source

Path: `derivation/code/lean/core/src/CNNA/Derivation/S01_PrimitiveResponseCoupledFiniteProvenance/S01_PrimitiveInputsGrammarAndEventOrder/S02_I002_FiniteApproximantDepthL.lean`
SHA-256: `ae23109e12622fc6494aad18bc056bf59206d18199288c101520d9c58f5b99e8`

- `FiniteApproximantDepth`, structure, lines 18–23: natural-valued finite cutoff.
- `mk_value`, theorem, lines 24–28: definitional constructor equation.
- `zeroAdmissible`, theorem, lines 29–34: zero is a genuine admissible value.

---

003 · C001 — Empty carrier ∅

Canonical node label: `003 · C001`
Semantic ID: `C001`
Current section path: `1.1.3`
Documentation tier: `D0`
Node role: primitive initial carrier
Verification status: Python tests reproduced; Lean source kernel-built in the current prefix-free 26-core-job package.

Position in the derivation

`C001` is the third canonical node and the nonparametric initial object of the constructive chain. It appears on the same visible origin rank as `I001` and `I002`, but its role is different: it is a unique state rather than a freely selected numerical input.

There is no incoming DAG edge. The lack of an incoming edge records that the empty carrier is not generated from an earlier carrier. Its only hard outgoing edge is the root-genesis handoff to `C002`.

Definition or statement

Let `V` be the set of present provenance nodes and `R ⊆ V × V` the present directed relations. The C001 state is

$$X_\varnothing = (V_\varnothing, R_\varnothing), \qquad V_\varnothing = \varnothing, \qquad R_\varnothing = \varnothing.$$

The complete local contract has three parts:

1. a pre-root carrier value exists;
2. it contains no node;
3. it contains no relation.

The node does **not** construct the root, an address, event order, geometry, conductance, response, or hidden initialization payload.

Introduction reason

A derivation beginning with root genesis must distinguish the state before genesis from the rooted state after genesis. Treating the former as `None`, an absent variable, or a constructorless type would erase the source object of the transition `C001 → C002`. C001 therefore supplies one explicit, payload-free state on which the next construction acts.

Construction or encoding

Mathematical representation

The node and relation components are both empty. The representation is unique up to equality because no local degree of freedom is present.

In the category of sets, the empty set is an initial object: for every set `A`, exactly one map `∅ → A` exists. This is an exact contextual comparison for the empty component, not a claim that the full CNNA derivation has already been formulated categorically.

Python representation

`EmptyCarrier` is a frozen, slotted dataclass with no stored fields. It exposes:

- `nodes = ()`;
- `relations = ()`;
- `contains_node(_) = False`;
- `contains_relation(_,_) = False`.

`EMPTY_CARRIER` is the canonical executable value. Constructing another zero-field instance yields an equal value, while attempting to pass a payload raises `TypeError`.

Lean representation

Lean uses an inhabited singleton:

```
inductive EmptyCarrier : Type where
  | empty
```

`canonical` names the unique constructor value. `ContainsNode` and `ContainsRelation` are defined as `False`; `noNode` and `noRelation` expose the consequences. `eqCanonical` proves that every inhabitant is the canonical state.

A constructorless inductive type would have no inhabitant and would therefore encode “no pre-root state exists,” which is not the contract. The one constructor represents uniqueness of the carrier value, not one provenance node.

Cross-layer correspondence

Python and Lean agree on the observable state:

- one canonical zero-payload carrier value exists;
- every node-membership query is false;
- every relation-membership query is false;
- no hidden payload distinguishes alternative carriers.

The runtime and proof representations are structurally different but semantically equivalent at the contract boundary.

Boundary case or countercheck

The counterchecks exclude four common conflations:

1. **Empty carrier versus missing object:** `EMPTY_CARRIER` is an actual value, not `None`.
2. **Empty carrier versus rooted state:** no root is present; root creation belongs to `C002`.
3. **Empty mathematical collections versus empty datatype:** Lean’s carrier type is inhabited even though its node and relation predicates are empty.
4. **Unique representation versus hidden parameter:** Python has no dataclass fields or instance dictionary, and a payload argument is rejected.

The Lean theorem `eqCanonical` is the exact uniqueness check. The Python equality and field-reflection tests are executable finite checks of the same boundary.

Result

C001 closes the statement:

> There exists exactly one payload-free pre-root carrier representation, and it contains neither provenance nodes nor directed relations.

This result is structural only. It does not prove or assume any property of the root that will be introduced by C002. The documentation is complete under schema v2.

Downstream handoff

E003: C001 → C002 has relation root_genesis. The handoff transfers the canonical empty carrier to the root-genesis construction. The edge note records that this is the only node birth before any relation exists.

No other outgoing edge leaves C001; all later structures must pass through the rooted state or other registered descendants.

Code anchors

Python source

- Path: derivation/code/python/src/cnna/derivation/s01_primitive_response_coupled_finite_provenance/s01_primitive_inputs_grammar_and_event_order/s03_c001__empty_carrier_empty.py
- Symbol: EmptyCarrier
- Kind: class
- Lines: 23–52
- SHA-256: ff6f7d36a4d2290b4007d27c384d2756bb8b6e61ed5618428f5faa30383d5ccc
- Canonical value: EMPTY_CARRIER, line 55.

Python counterchecks

- Path: derivation/code/python/tests/derivation/s01_primitive_response_coupled_finite_provenance/s01_primitive_inputs_grammar_and_event_order/test_s03_c001__empty_carrier_empty.py
- Symbol: TestEmptyCarrier
- Kind: test class
- Lines: 14–38
- SHA-256: 873b6eef6f8400448e914409f38c16ec657f021ade4b5b70761d2c7a9349aa93
- Covered cases: empty collections, vacuous queries, zero stored fields, semantic uniqueness, hidden-payload rejection.

Lean source

Path: derivation/code/lean/core/src/CNNA/Derivation/S01_PrimitiveResponseCoupledFiniteProvenance/S01_PrimitiveInputsGrammarAndEventOrder/S03_C001_EmptyCarrierEmpty.lean
SHA-256: d5e5daf9a0b04bb596666443a8c873a7a554bd35cf7f33229aab86e930595253

- EmptyCarrier, inductive type, lines 21–26: inhabited singleton representation.
- canonical, definition, lines 27–30: canonical pre-root value.
- ContainsNode, definition, lines 31–34: node membership is false.
- ContainsRelation, definition, lines 35–39: relation membership is false.
- noNode, theorem, lines 40–44: no node can belong.
- noRelation, theorem, lines 45–49: no ordered pair can belong.
- eqCanonical, theorem, lines 50–56: every inhabitant is the canonical state.

004 · C002 — Root genesis r

Canonical node label: `004 · C002`
Semantic ID: `C002`
Current section path: `1.1.4`
Documentation tier: `D1`
Documentation state: `COMPLETE_V2`

Position In Derivation

`C002` is canonical node 004. Its only hard predecessor is `003 · C001`, the unique empty carrier. The node is the first construction that changes the cardinality of the provenance carrier: before it, no provenance node exists; after it, exactly the root exists.

The order is essential. Neither `I001` nor `I002` is needed to create the root, and no address grammar is available yet. Hence root genesis is independent of branching multiplicity and finite cutoff. The hard incoming edge is `E003: C001 -> C002` with relation `root_genesis`; the outgoing edge is `E004: C002 -> C003` with relation `anchors_provenance_grammar`.

Mathematical Contract

Let

$$X_\varnothing = (V_\varnothing, R_\varnothing), \qquad V_\varnothing = \varnothing, \quad R_\varnothing = \varnothing$$

be the C001 state. `C002` defines one transition

$$\Gamma_r(X_\varnothing) = X_r,$$

such that

$$V(X_r) = \{r\}, \qquad R(X_r) = \varnothing.$$

The complete local contract is:

1. the root token `r` exists;
2. the post-genesis carrier contains exactly `r`;
3. no ordered pair is a relation;
4. `r` has no provenance parent;
5. no geometric position is assigned;
6. no address, node ID, rank, event index, conductance, load or response is introduced.

The contract is about the **immediate post-genesis state**. It does not claim that the root remains relation-free after later births.

Introduction Reason

The empty carrier alone does not supply a node on which later provenance addresses or relations can be based. `C002` isolates the irreducible birth of the root from every downstream structure. This separation prevents the root address, first edge or first conductance from being mistaken for primitive content of genesis.

Explicit Construction

Mathematical construction

Introduce a single zero-payload symbol `r`. Define the carrier `X_r` by the singleton node set and empty relation set above. Define parenthood and geometric-position predicates for `r` to be false at this stage.

Python construction

Python uses two frozen, slotted, zero-field dataclasses:

- `Root` is the root token type;
- `RootedCarrier` is the post-genesis carrier type.

`ROOT` and `ROOTED_CARRIER` are their canonical values. `RootedCarrier.nodes` returns `(ROOT,)`, `relations` returns `()`, `contains_node` recognizes only `ROOT`, and `contains_relation` is constantly false. `root_genesis` accepts exactly an `EmptyCarrier` value and returns `ROOTED_CARRIER`.

Lean construction

Lean uses singleton inductive types `Root` and `RootedCarrier`. Membership is represented by

```
node = Root.canonical
```

and relation, parent and position predicates are definitionally `False`. The transition

```
def rootGenesis (_carrier : EmptyCarrier) : RootedCarrier :=
  RootedCarrier.canonical
```

is total on the C001 type and contains no branch or selected witness.

Invariants

The construction establishes the following invariants:

Invariant	Formal content	Evidence
Singleton node content	every admitted node equals `Root.canonical`	Lean `rootPresent`, `nodeUnique`; Python `nodes == (ROOT,)`
Empty relation content	no source-target pair belongs	Lean `ContainsRelation := False`, `noRelation`; Python `relations == ()`
Parentlessness	no parent value can witness parenthood of the root	Lean `HasParent := False`, `rootHasNoParent`; Python `parent_of(ROOT) is None`
No geometry	no position value can witness a root position	Lean `HasGeometricPosition := False`, `rootHasNoGeometricPosition`; Python metadata-reflection test
Zero local payload	root and carrier have no data fields	Python dataclass reflection; Lean one-constructor inductives

The distinction between “one carrier value” and “one provenance node” is explicit: `RootedCarrier` is the state representation; `Root.canonical` is the sole provenance node inside it.

Canonicity Or Uniqueness

Lean proves two separate uniqueness statements:

$$\forall r' : \mathrm{Root}, \quad r' = r,$$

and

$$\forall X' : \text{RootedCarrier}, \quad X' = X_r.$$

These are `Root.eqCanonical` and `RootedCarrier.eqCanonical`. Since `EmptyCarrier` is itself unique, `rootGenesis_eqCanonical` shows that every admissible source representation produces the same rooted carrier. Thus no hidden root choice, carrier choice or genesis seed occurs at C002.

Python provides the executable analogue: zero-field dataclass instances compare equal, canonical constants are returned, and there is no instance dictionary in which hidden state could be stored.

Boundary Cases

- **No self-edge:** `(r,r)` is not a relation. Root birth is not first-relation birth.
- **No parent sentinel as data:** Python returns `None` from `parent_of(ROOT)`, but Lean states the stronger proposition that no parent witness exists. `None` is an API representation, not a mathematical parent value.
- **Exact source domain:** Python rejects `None`, tuples, arbitrary objects, the root token and the already rooted carrier as genesis inputs. Only C001's carrier type is accepted.
- **No downstream metadata:** addresses, depths, ranks, event indices, conductances, responses and coordinates are absent.
- **No dependence on `b` or `L`:** neither parameter occurs in the constructor or theorem signatures.

Python Lean Cross Layer

Contract component	Python	Lean	Agreement
Root token	zero-field `Root` dataclass	singleton `Root` inductive	unique zero-payload value
Rooted state	zero-field `RootedCarrier` dataclass	singleton `RootedCarrier` inductive	unique carrier representation
Node membership	equality with `ROOT`	`node = Root.canonical`	exact singleton membership
Relation membership	always `False`	proposition `False`	exact empty relation set
Genesis map	runtime type check, canonical return	total function on `EmptyCarrier`	same mathematical domain and value
Uniqueness evidence	equality/reflection tests	kernel theorems	executable check plus formal proof

There is no known cross-layer mismatch. Python performs explicit dynamic rejection because its type annotations are not runtime proofs; Lean's function domain already excludes invalid source types.

Countercheck

The node-local Python suite contains four targeted tests:

Test	Lines	Failure excluded
`test_genesis_creates_exactly_one_root_and_no_relation`	22-28	extra node or relation, including a self-edge

Test	Lines	Failure excluded
`test_root_is_unique_zero_payload_and_has_no_parent`	30-38	hidden dataclass payload or parent
`test_c002_does_not_smuggle_downstream_metadata`	40-60	address, geometry, event, conductance or response fields
`test_genesis_domain_is_exactly_empty_carrier`	62-67	coercion from unrelated source objects

Lean separately proves the positive singleton statement and the three negative predicates. The combination is stronger than either an executable example or a type-level singleton alone.

Result

`C002` closes the structural statement:

> From the unique empty carrier there is a canonical root-genesis transition to a unique carrier containing exactly one zero-payload root and no relation; the root has no parent or geometry at this derivation stage.

No later structure is included in this result. The node remains frozen under the existing core gate; the documentation update does not alter its Python or Lean source.

Downstream Handoff

- `E003` is the completed incoming genesis edge from `C001`.
- `E004` passes the rooted carrier to `C003`.

At `C003`, the root token is anchored to the empty word. That anchoring is a new construction and must not be read backward into C002.

Code Anchors

Python source

Path: `derivation/code/python/src/cnna/derivation/s01_primitive_response_coupled_finite_provenance/s01_primitive_inputs_grammar_and_event_order/s04_c002__root_genesis_r.py`
Source SHA-256: `889514d52db406e4a2ea975ecf9439e19fe52000431fce888ee98ced0b3482ef`

Symbol	Kind	Lines	Role
`Root`	`CLASS`	22-23	`SOURCE`
`RootedCarrier`	`CLASS`	30-50	`SOURCE`
`root_genesis`	`FUNCTION`	56-65	`SOURCE`

Python tests

Path: `derivation/code/python/tests/derivation/s01_primitive_response_coupled_finite_provenance/s01_primitive_inputs_grammar_and_event_order/test_s04_c002__root_genesis_r.py`
Source SHA-256: `cbc246c9536122c79f031c7e79ea3f94bd39360c0566f1b58a3adf99ca5cf488`

Symbol	Kind	Lines	Role
TestRootGenesis	CLASS	21-67	TEST

Lean core

Path: derivation/code/lean/core/src/CNNA/Derivation/S01_PrimitiveResponseCoupledFiniteProvenance/S01_PrimitiveInputsGrammarAndEventOrder/S04_C002_RootGenesisR.lean
Source SHA-256: f8e3f0a55eea3b91203686cfeb1950abddd39f5d66fb7c8e0b218a26863146a5

Symbol	Kind	Lines	Role
Root	INDUCTIVE	22-27	SOURCE
canonical	DEF	28-31	SOURCE
eqCanonical	THEOREM	32-38	SOURCE
RootedCarrier	INDUCTIVE	39-44	SOURCE
canonical	DEF	45-48	SOURCE
ContainsNode	DEF	49-52	SOURCE
ContainsRelation	DEF	53-57	SOURCE
HasParent	DEF	58-62	SOURCE
HasGeometricPosition	DEF	63-67	SOURCE
rootPresent	THEOREM	68-72	SOURCE
nodeUnique	THEOREM	73-77	SOURCE
noRelation	THEOREM	78-82	SOURCE
rootHasNoParent	THEOREM	83-88	SOURCE
rootHasNoGeometricPosition	THEOREM	89-94	SOURCE
eqCanonical	THEOREM	95-101	SOURCE
rootGenesis	DEF	102-105	SOURCE
rootGenesis_canonical	THEOREM	106-110	SOURCE
rootGenesis_eqCanonical	THEOREM	111-115	SOURCE

---

005 · C003 — Finite b-ary provenance grammar

Canonical node label: 005 · C003
Semantic ID: C003
Current section path: 1.1.5
Documentation tier: D1
Documentation state: COMPLETE_V2

Position In Derivation

C003 is canonical node 005 and the first three-input join in the chain. Its hard predecessors are:

- 001 · I001 through E001, supplying $b \geq 2$;
- 002 · I002 through E002, supplying $L \geq 0$;
- 004 · C002 through E004, supplying the already-born canonical root.

The grammar must follow root genesis because it anchors an existing root token. It must follow both inputs because the slot alphabet depends on b and finite admission depends on L.

Mathematical Contract

Let

$$S_b = \{0, 1, \ldots, b-1\}, \qquad b \geq 2.$$

An intrinsic provenance address is a finite word

$$a = (a_1, \ldots, a_d) \in S_b^*,$$

with depth $|a| = d$. The canonical root address is the empty word ε. For $u \in S_b^*$ and $q \in S_b$, define right extension

$$\mathrm{snoc}(u, q) = u \| (q).$$

The finite approximant at cutoff `L` admits

$$\mathcal{A}_{b,L} = \{a \in S_b^* : |a| \leq L\}.$$

Every nonempty address has a unique decomposition

$$a = u \| (q),$$

which defines its immediate parent `u` and final slot/rank `q`. The contract owns exactly:

1. the local slot alphabet;
2. intrinsic finite words;
3. root anchoring to ε;
4. child extension;
5. depth, parent and final-rank recovery;
6. finite admission by $|a| \leq L$.

It owns no event order, node ID, geometry, metric, conductance, response or dynamics.

Introduction Reason

The rooted carrier from C002 contains a root token but no language for descendants. C003 introduces the minimal grammar needed to refer canonically to potential descendants without importing a temporal order or physical state. Separating the intrinsic word constructor from the finite cutoff is essential: `L` restricts the current approximant but must not change the local `b`-ary branching rule.

Explicit Construction

Intrinsic grammar

The alphabet is `S_b`. The root word is empty. `snoc` appends one slot at the right. The inverse operation `unsnoc?` returns `none` on the root and otherwise returns the prefix word and final slot.

The immediate parent relation is defined by

```
Parent(u,a) iff parent?(a) = some(u).
```

Depth is word length. Therefore the grammar is rooted and ordered: the final symbol records the local child slot.

Finite admission

A bounded address is a pair consisting of an intrinsic address and a proof that its depth is at most L. The bounded child constructor additionally requires a proof that the successor depth remains within the cutoff.

This gives two distinct layers:

$$S_b^* \quad \text{(intrinsic grammar)}$$

and

$$\mathcal{A}_{b,L} \subseteq S_b^* \quad \text{(finite admitted carrier)}.$$

Root anchor and predecessor join

`FiniteBAryProvenanceGrammar` stores exactly the three predecessor objects: the C002 rooted carrier, I001 branching parameter and I002 cutoff. `rootAddress` maps the unique root token to the bounded empty word. The `build_*` equations expose that the canonical constructor preserves each predecessor without transformation.

Invariants

Invariant	Mathematical statement	Formal evidence
Root depth	$\|\varepsilon\| = 0$	`depth_root`
Successor depth	`|snoc(u,q)| = |u| + 1`	`depth_snoc`, bounded `child_depth`
Exact decomposition	`unsnoc?(snoc(u,q)) = (u, q)`	`unsnoc?_snoc`
Root parentlessness	$\text{parent?}(\varepsilon) = \text{none}$	`parent?_root`
Root has no final rank	$\text{finalSlot?}(\varepsilon) = \text{none}$	`finalSlot?_root`
Child parent equation	`parent?(snoc(u,q)) = some(u)`	`parent?_snoc`, `parent_snoc`
Child rank equation	`finalSlot?(snoc(u,q)) = some(q)`	`finalSlot?_snoc`
Parent uniqueness	equal child words imply equal prefixes	`snoc_parent_unique`
Slot uniqueness	equal child words imply equal final slots	`snoc_slot_unique`
Cutoff preservation	bounded child depth remains $\leq L$	`BoundedProvenanceAddress.child`
Root admission	empty word admitted for every L, including zero	`BoundedProvenanceAddress.root`

These invariants show that parenthood and sibling rank are derivable from syntax. They are not independent node labels that could disagree with the address.

Canonicity Or Uniqueness

Three distinct canonicity claims are established:

1. **Root anchor:** the unique C002 root maps to the empty word.
2. **Non-root decomposition:** a child word determines its parent and final slot uniquely.
3. **Predecessor join:** the grammar constructor contains exactly the supplied rooted carrier, branching parameter and cutoff.

No enumeration order is claimed. Multiple words at the same depth are incomparable until C018 supplies its BFS/lexicographic schedule. Hence C003 proves canonical syntax, not canonical time.

Boundary Cases

Zero cutoff

For $L = 0$, the admitted carrier contains only ε. The intrinsic alphabet and `child_address` remain defined, but no non-root child is admitted into the finite approximant.

Root operations

The root has no parent and no final rank. Python raises `ValueError`; Lean returns `none`. These are equivalent partial-operation encodings.

Invalid slots and addresses

Ranks below zero or at least `b` are rejected. Python additionally rejects Boolean, floating-point, string and `None` ranks, and rejects list-based addresses because the executable canonical representation is a tuple. Lean prevents out-of-range slots by the type `Fin b.value`.

Cutoff independence

The intrinsic constructor can produce a word deeper than `L`; the bounded grammar then rejects it. This countercheck prevents the common but incorrect interpretation that the terminal level changes the slot alphabet or makes child extension undefined in the unbounded grammar.

Python Lean Cross Layer

Concept	Python	Lean	Semantic relation
Slot	built-in `int` checked against $0 \le q < b$	`Fin b.value`	same finite alphabet; Lean internalizes the bound
Address	`tuple[int, ...]`	`List (ProvenanceSlot b)`	same finite-word semantics
Root	empty tuple `()`	empty list `[]`	exact empty word
Child	tuple concatenation	recursive `snoc`	right extension
Parent/rank	slicing and final element	`unsnoc?`, projections	same unique decomposition
Cutoff	runtime length check	proof field `depth_le_cutoff`	same admitted words
Grammar join	frozen dataclass	structure	same three predecessors

Python's `Address` type alias is not by itself a runtime proof; `validate_unbounded_address` supplies the checks. Lean's dependent slot type prevents malformed slot values from entering an address in the first place. Conversely, Python explicitly tests foreign runtime types that cannot inhabit the corresponding Lean types.

Countercheck

The node-local Python suite contains eight targeted groups:

Test	Lines	Property or failure excluded
`test_three_predecessors_join_and_root_is_anchored_to_empty_word`	31-38	missing predecessor or nonempty root address
`test_slot_alphabet_is_exactly_zero_through_b_minus_one`	40-46	off-by-one slot alphabet
`test_child_parent_rank_and_depth_are_word_derived`	48-61	independent or inconsistent parent/rank/depth fields
`test_local_b_ary_word_constructor_is_independent_of_cutoff`	63-70	conflation of intrinsic grammar with finite admission
`test_zero_cutoff_is_root_only_for_admitted_words`	72-83	accidental non-root admission at `L=0`
`test_root_has_no_parent_or_final_rank`	85-89	fictitious root predecessor or rank
`test_invalid_ranks_addresses_and_predecessor_types_are_rejected`	91-109	malformed slots, addresses or predecessor types
`test_c003_adds_no_event_geometry_response_or_node_id_fields`	111-122	hidden schedule, geometry or physical state

The nontrivial Lean proof chain is:

```
unsnoc?_snoc
  -> parent?_snoc and finalSlot?_snoc
  -> parent_snoc
  -> snoc_parent_unique and snoc_slot_unique
```

with `depth_snoc` and bounded `child_depth` providing the independent depth/cutoff chain. These theorems establish the general identities that the Python cases instantiate.

Result

`C003` closes the statement:

> Given the canonical root, branching parameter $b \geq 2$, and finite cutoff $L \geq 0$, there is a canonical finite b-ary provenance grammar whose nodes are bounded words, whose root is the empty word, and whose non-root addresses determine unique parents and final slots.

The result is grammatical and finite. It does not define a birth order, metric, conductance or response.

Downstream Handoff

- `E005: C003 -> C004A` supplies the grammar for the first provenance slot.

- `E063: C003 -> C018` supplies the grammar to the separate canonical schedule construction.
- `E140` and `E149` expose future proof obligations for order and finite enumeration.
- `E402: C003 -> M050` remains blocked because provenance-derived metric selection is not established by the grammar alone.
- `E260` supports the later node-local metric-family proof gate.

The outgoing multiplicity reflects reuse of one grammar, not parallel versions of the DAG.

Code Anchors

Python source

Path: `derivation/code/python/src/cnna/derivation/s01_primitive_response_coupled_finite_provenance/s01_primitive_inputs_grammar_and_event_order/s05_c003__finite_b_ary_provenance_grammar.py`
Source SHA-256: `fa2ad5221cd2131508b98d54673a4cb8324e673d50b339a8aab857b69fe435cb`

Symbol	Kind	Lines	Role
`slot_alphabet`	`FUNCTION`	35-39	`SOURCE`
`validate_unbounded_address`	`FUNCTION`	42-53	`SOURCE`
`root_address`	`FUNCTION`	56-58	`SOURCE`
`child_address`	`FUNCTION`	61-68	`SOURCE`
`address_parent`	`FUNCTION`	71-76	`SOURCE`
`final_slot`	`FUNCTION`	79-84	`SOURCE`
`address_depth`	`FUNCTION`	87-89	`SOURCE`
`is_parent_of`	`FUNCTION`	92-96	`SOURCE`
`FiniteBAryProvenanceGrammar`	`CLASS`	100-164	`SOURCE`
`build_finite_b_ary_provenance_grammar`	`FUNCTION`	167-173	`SOURCE`

Python tests

Path: `derivation/code/python/tests/derivation/s01_primitive_response_coupled_finite_provenance/s01_primitive_inputs_grammar_and_event_order/test_s05_c003__finite_b_ary_provenance_grammar.py`
Source SHA-256: `1d6630ef73e34981eacecbb02cd8100d8ae12f6aa5f28905ede76084fd02b581`

Symbol	Kind	Lines	Role
`TestFiniteBAryProvenanceGrammar`	`CLASS`	25-122	`TEST`

Lean core

Path: `derivation/code/lean/core/src/CNNA/Derivation/S01_PrimitiveResponseCoupledFiniteProvenance/S01_PrimitiveInputsGrammarAndEventOrder/S05_C003_FiniteBAryProvenanceGrammar.lean`
Source SHA-256: `81563345b44177915f32a03d6c42df7adfa1ca4176ab27d6569584105ad25866`

Symbol	Kind	Lines	Role
ProvenanceSlot	ABBREV	26-28	SOURCE
ProvenanceAddress	ABBREV	29-33	SOURCE
root	DEF	34-37	SOURCE
snoc	DEF	38-42	SOURCE
depth	DEF	43-46	SOURCE
depth_root	THEOREM	47-50	SOURCE
depth_snoc	THEOREM	51-60	SOURCE
unsnoc	DEF	61-69	SOURCE
unsnoc	THEOREM	70-82	SOURCE
parent	DEF	83-88	SOURCE
finalSlot	DEF	89-94	SOURCE
Parent	DEF	95-98	SOURCE
parent	THEOREM	99-102	SOURCE
finalSlot	THEOREM	103-106	SOURCE
parent	THEOREM	107-112	SOURCE
finalSlot	THEOREM	113-118	SOURCE
parent_snoc	THEOREM	119-123	SOURCE
snoc_parent_unique	THEOREM	124-132	SOURCE
snoc_slot_unique	THEOREM	133-143	SOURCE
BoundedProvenanceAddress	STRUCTURE	144-151	SOURCE
root	DEF	152-157	SOURCE
child	DEF	158-168	SOURCE
root_address	THEOREM	169-173	SOURCE
child_address	THEOREM	174-181	SOURCE
child_depth	THEOREM	182-192	SOURCE
FiniteBAryProvenanceGrammar	STRUCTURE	193-200	SOURCE
build	DEF	201-207	SOURCE
rootPresent	THEOREM	208-212	SOURCE
rootAddress	DEF	213-217	SOURCE
rootAddress_canonical	THEOREM	218-222	SOURCE
rootAddress_unique	THEOREM	223-227	SOURCE
build_rootedCarrier	THEOREM	228-232	SOURCE
build_branching	THEOREM	233-237	SOURCE
build_cutoff	THEOREM	238-244	SOURCE

Open-provenance role: Provenance-complete carrier

C003 supplies the finite event-address grammar of the current CNNA specialization. In the generalized open-provenance interpretation it is the carrier on which immutable event provenance can be recorded; it does not by itself assert that every empirical system has such a carrier.

---

006 · C018 — Canonical BFS/lexicographic provenance birth schedule

Canonical node label: `006 · C018`
Semantic ID: `C018`
Current section path: `1.1.6`
Documentation tier: `D2`
Documentation state: `COMPLETE_V2`

Position In Derivation

C018 is canonical node 006. Its sole hard scientific predecessor is `005 · C003`, which supplies the finite branching grammar, the root address, the local slot alphabet and the depth cutoff. C018 is introduced only after those objects exist because an order cannot be defined before its carrier and local rank type are available.

The root itself is not scheduled: it was created by C002 and anchored to the empty word by C003. C018 orders non-root provenance births. The incoming hard edge is `E063: C003 -> C018` with relation `defines_canonical_birth_schedule`. The proof-certification edge `E14 1: P002 -> C018` is non-hard and records the dedicated static-order closure source. Its local kernel build and axiom audit remain the current gate; least-open state selection is owned by C004.

Formal Statement

Let `b` be the C001 branching parameter and write

$$S_b = \{0, \dots, b-1\}, \qquad S_b^* = \bigcup_{d \geq 0} S_b^d.$$

For local ranks `r,s in S_b`, define

$$r <_S s \iff r < s.$$

For provenance words $a, c \in S_b^*$, let $a <_{\mathrm{lex}} c$ be the lexicographic lift of $<_S$. C018 defines

$$a \prec c \iff |a| < |c| \ \vee \ \big(|a| = |c| \wedge a <_{\mathrm{lex}} c\big).$$

The current Lean source proves:

1. `prec` is irreflexive;
2. `prec` is transitive;
3. `prec` is asymmetric;
4. for all addresses `a,c`, exactly one of the mutually exclusive cases `a prec c`, `a=c`, or `c prec a` holds;
5. every parent precedes each child;
6. for one parent, smaller sibling rank implies earlier child;
7. the relation induced on bounded slots through their child addresses is irreflexive, transitive and asymmetric;
8. two slots selecting distinct child addresses are comparable.

The phrase **strict total order** in the C018 contract refers to this address order and, conditionally on distinct selected children, to the induced slot order. It does not by itself assert that a dynamically evolving set of currently open slots has been constructed or that its least element is unique.

Hypotheses

The address-order theorems require only the C003 address type generated from `b`. The lower bound `b >= 2` is carried by `BranchingParameter`, although the order-theoretic proof itself uses only the finiteness and decidable natural values of the local slot type.

The bounded-slot layer additionally requires:

- a `CanonicalBirthSchedule` carrying a C003 grammar;
- a bounded parent address;
- a local rank;
- a proof that the child depth satisfies `|parent|+1 <= L`.

The theorem `openSlotBefore_total_of_distinct_children` explicitly assumes that the two selected child addresses are unequal. No theorem in C018 assumes a born-set, an event history, a response history, or a physical state.

Introduction Reason

C003 constructs a rooted ordered word grammar but deliberately does not linearize its nodes. Recurrent growth requires one deterministic choice of which admissible slot is considered next. C018 therefore fixes a schedule convention before C004A selects the first non-root slot and before C004 defines the recurrent next-open-slot interface.

The convention is canonical only **relative to the locked CNNA rule**. Other strict total orders on the same finite grammar exist. The current node proves that the chosen rule is mathematically coherent and unambiguous after selection; it does not derive shortlex order uniquely from the grammar alone.

Proof Strategy

The formal proof is layered so that each mathematical responsibility is isolated.

1. **Local rank order.** `SlotBefore r s` is defined as strict natural-number comparison of the finite slot values.
2. **Word order.** `AddressLexBefore` uses `List.Lex SlotBefore`.
3. **Schedule order.** `BirthBefore` is the disjoint depth/lexicographic definition above.
4. **Same-parent compatibility.** Induction on the common parent word proves that appending ranks `r<s` preserves lexicographic order.
5. **Word-order laws.** Structural recursion on lists proves lexicographic irreflexivity, transitivity and constructive trichotomy.
6. **Depth/lex lift.** Case analysis over the two defining branches and `Nat.lt_trichotomy` yields address-order transitivity and trichotomy.
7. **Asymmetry and totality.** Asymmetry follows by composing opposite directions and contradicting irreflexivity; totality for distinct addresses follows by eliminating the equality branch of trichotomy.
8. **Bounded slots.** A slot is mapped to its bounded C003 child address, and all order laws are inherited from `BirthBefore`.

This architecture avoids treating the Python loop as the proof. The executable enumeration is evidence for the same specification, while the Lean order laws are proved directly from the typed address grammar.

Lemma Chain

The load-bearing chain is

```
SlotBefore
  -> AddressLexBefore
  -> BirthBefore

lex_snoc_same_parent
  -> sameParentIncreasingRank

slotBefore_irrefl
  -> addressLexBefore_irrefl
  -> birthBefore_irrefl

addressLexBefore_trans
  -> birthBefore_trans
  -> birthBefore_asymm

addressLexBefore_trichotomy
  -> birthBefore_trichotomy
  -> birthBefore_total_of_ne
```

```
OpenBirthSlot.childAddress
  -> OpenSlotBefore
  -> sameParentOpenSlotIncreasingRank
  -> openSlotBefore_irrefl
  -> openSlotBefore_trans
  -> openSlotBefore_asymm
  -> openSlotBefore_total_of_distinct_children
```

The proof of `addressLexBefore_trans` exhausts the constructors of the two lexicographic witnesses. The proof of `addressLexBefore_trichotomy` compares the heads by `Nat.lt_trichotomy`; equal heads reduce recursively to the tails. `birthBefore_trans` then handles the four combinations depth/depth, depth/lex, lex/depth and lex/lex.

Formal Realization

Python

Source: `derivation/code/python/src/cnna/derivation/s01_primitive_response_coupled_finite_provenance/s01_primitive_inputs_grammar_and_event_order/s06_c018__canonical_bfs_lexicographic_provenance_birth_schedule.py`
SHA-256: `c99afaef142d6f3d4d74b26de21db6c1450ff1af1098f449746da5e5f6c4dae7`

`OpenBirthSlot` stores parent, rank and child explicitly. `open_slot_key` returns `(len(parent), parent, rank)`. `canonical_birth_slots` maintains a FIFO parent list and cursor. Whenever one parent is processed, ranks are emitted in increasing order and each child is appended to the end of the parent queue. Therefore all parents at depth `d` are processed before any parent at depth `d+1`, and parents at one depth occur in lexicographic order.

The key and the enumerator are extensionally consistent: comparing `(len(u),u,r)` and `(len(v),v,s)` is equivalent to comparing child words `u||(r)` and `v||(s)` by depth and lexicographic order. This statement is supported by the definitions and exhaustive finite tests in the current package; it is not yet exported as a cross-language theorem.

Lean

Source: `derivation/code/lean/core/src/CNNA/Derivation/S01_PrimitiveResponseCoupledFiniteProvenance/S01_PrimitiveInputsGrammarAndEventOrder/S06_C018_CanonicalBfsLexicographicProvenanceBirthSchedule.lean`
SHA-256: `d332704d80a9aaa1eca72b7961d868aabceeead7320bbbd5766b1d2ba2bb53a1`

Lean represents ranks by `Fin b.value` and words by lists of those ranks. Thus malformed local ranks cannot inhabit the slot type. The address relation is independent of `L`; the cutoff enters only when `OpenBirthSlot` stores a bounded parent and a proof that the selected child remains within the approximant. The derived slot order compares the selected child addresses rather than comparing an unrelated slot identifier.

Executable tests

Source: `derivation/code/python/tests/derivation/s01_primitive_response_coupled_finite_provenance/s01_primitive_inputs_grammar_and_event_order/test_s06_c018__canonical_bfs_lexicographic_provenance_birth_schedule.py`
SHA-256: `12fcb32879313f7cd3e4bd9b5b004437767bd1876920aef7bf365fa5ae154653`

The nine node-local tests verify:

- the exact binary depth-three address order;
- the ternary depth-two parent/rank order;
- child extension and strict adjacency direction;
- irreflexivity, pairwise total direction and transitivity on the finite sample;

- the empty non-root schedule at `L=0`;
- determinism and absence of hidden schedule inputs;
- absence of event, time, geometry, conductance, response and batch state;
- rejection of invalid predecessors and operands;
- cardinality $\sum_{d=1}^{L} b^d$ and uniqueness of emitted child addresses for the tested ranges.

Counterexamples Or Necessity Checks

1. **Depth comparison is necessary.** Pure lexicographic order may place a deeper word before a shallower word. For example, `(0,0)` is lexicographically before `(1)` under ordinary list lexicography, whereas the active BFS convention requires `(1)` before every depth-two word.
2. **Sibling rank is necessary.** Omitting the final rank from the executable key makes all siblings of one parent indistinguishable and fails to determine a strict order.
3. **Root exclusion is necessary.** Including the empty word as a scheduled birth would duplicate the already completed C002 genesis and shift every downstream event label.
4. **The cutoff proof is necessary for bounded slots.** Without `|parent|+1 <= L`, the slot structure could select a child outside the finite approximant even though its parent is admitted.
5. **Distinct-child hypothesis is necessary for slot totality.** Two record values that select the same child address are not ordered in either direction because the underlying address relation is irreflexive. C018 therefore proves total comparability only after excluding this equality case.
6. **Order does not imply least-open uniqueness.** A strict total order on the ambient address set does not define which slots are currently open. A born-set predicate and proof that the relevant open set is nonempty are required before a least element can be selected. This state-dependent selection problem is deliberately outside P002 and is owned by C004.
7. **Finite enumeration is not yet a Lean theorem.** Python's loop terminates and the tests match the carrier cardinality, but C018's Lean module defines the order and bounded slot objects, not an executable finite enumeration theorem. Exhaustivity and termination remain P004.

Axiom Profile

C018 is part of the mathlib-free `cnna_core` package. The current source contains no `axiom`, `sorry`, `admit`, `Classical`, `noncomputable`, `unsafe`, `partial`, `simp`, or `simpa` use. The prefix-free source was included in the successful 26-job core build reported for Lean 4.31.0. The node-local registry classifies the proof audit as PASS.

The source-local theorem chain is constructive. No transitive Mathlib choice profile is involved because this module does not import Mathlib. The separate long-term project goal of eliminating transitive axioms in Mathlib-dependent proof packages is therefore not a limitation of C018.

Result

C018 establishes a coherent deterministic shortlex schedule convention:

- all shallower addresses are earlier;
- addresses at one depth are lexicographically ordered;
- parents precede children;
- siblings follow increasing local rank;
- the address order is strict and total;
- bounded admissible slots inherit the order through their selected children.

The Python implementation enumerates the bounded non-root addresses in that order for the tested finite grammars, and its state surface contains no downstream physical fields.

Remaining Limits

C018 does **not** close the following statements:

- construction of the state-dependent set of currently open slots;
- existence and uniqueness of its least element;
- a kernel theorem equating the Python selector with the Lean relation;
- finite-schedule exhaustivity and termination for every `b,L`;
- event-index or birth-time assignment;
- label-equivariance and absence of hard-coded rank bias;
- response, conductance, geometry or dynamics.

These are downstream responsibilities represented by P002, P004, C010, T003 and later nodes. Accordingly C018 retains the status `IMPLEMENTED_VERIFIED_EXTENSION_REQUIRED`, not an unrestricted closure claim.

Downstream Handoff

- `E064 C018 -> C004A`: selects the first provenance slot.
- `E065 C018 -> C004`: orders the recurrent next-open-slot interface.
- `E066 C018 -> C010`: supplies the order from which event indices will later be assigned; C018 itself assigns none.
- `E073 C018 -> T003`: supplies the label-order contract for the future equivariance theorem.
- `E080 C018 -> C019`: orders the future finite iteration.
- `E089 C018 -> C042`: fixes the active schedule scope for ancestor-return rank closure.
- `E106 C018 -> CTRL005`: defines the active reference against which the layer-batch control is compared.
- `E144 C018 -> P003`: supports birth-cut canonicity.
- `E150 C018 -> P004`: supports finite exhaustivity and termination.
- `E141 P002 -> C018`: certification of the dedicated static-order closure after its local kernel build.

Code Line Register

Python source

Symbol	Kind	Lines	Scientific role
`OpenBirthSlot`	`CLASS`	32-37	slot record or bounded admissible slot structure
`_require_grammar`	`FUNCTION`	40-43	exact C003 predecessor type gate
`open_slot_key`	`FUNCTION`	46-53	executable BFS/lex ordering key
`slot_precedes`	`FUNCTION`	56-58	strict comparison induced by the key
`canonical_birth_slots`	`FUNCTION`	61-88	breadth-first finite slot enumerator
`canonical_birth_addresses`	`FUNCTION`	91-93	projection to selected child addresses
`CanonicalBirthSchedule`	`CLASS`	97-114	rule object carrying only C003

Symbol	Kind	Lines	Scientific role
`build_canonical_birth_schedule`	`FUNCTION`	117-121	canonical Python constructor

Python tests

Symbol	Kind	Lines	Scientific role
`TestCanonicalBfsLexicographicSchedule`	`CLASS`	22-123	node-local executable contract and negative controls

Lean core

Symbol	Kind	Lines	Scientific role
`SlotBefore`	`DEF`	24-27	strict local rank relation
`AddressLexBefore`	`DEF`	28-32	lexicographic word relation
`BirthBefore`	`DEF`	33-39	depth-then-lex address relation
`shallower_before`	`THEOREM`	40-46	depth branch introduction
`sameDepthLex_before`	`THEOREM`	47-53	equal-depth lex branch introduction
`lex_snoc_same_parent`	`THEOREM`	54-64	lexicographic preservation under common prefix
`sameParentIncreasingRank`	`THEOREM`	65-73	sibling-order theorem
`parent_before_child`	`THEOREM`	74-81	breadth-first parent/child theorem
`slotBefore_irrefl`	`THEOREM`	82-86	rank-order irreflexivity
`addressLexBefore_irrefl`	`THEOREM`	87-100	word-order irreflexivity
`addressLexBefore_trans`	`THEOREM`	101-141	word-order transitivity
`addressLexBefore_trichotomy`	`THEOREM`	142-172	constructive word trichotomy
`birthBefore_irrefl`	`THEOREM`	173-182	address-order irreflexivity
`birthBefore_trans`	`THEOREM`	183-199	address-order transitivity
`birthBefore_asymm`	`THEOREM`	200-204	address-order asymmetry
`birthBefore_trichotomy`	`THEOREM`	205-225	constructive address trichotomy

Symbol	Kind	Lines	Scientific role
birthBefore_total_of_ne	THEOREM	226-240	total comparability for distinct addresses
CanonicalBirthSchedule	STRUCTURE	241-246	rule object carrying only C003
build	DEF	247-250	Lean constructor carrying only C003
build_grammar	THEOREM	251-256	constructor equation
OpenBirthSlot	STRUCTURE	257-265	slot record or bounded admissible slot structure
childAddress	DEF	266-271	bounded C003 child constructor
childAddress_address	THEOREM	272-279	child word equation
OpenSlotBefore	DEF	280-286	slot order induced by child addresses
sameParentOpenSlotIncreasingRank	THEOREM	287-301	bounded sibling-order theorem
openSlotBefore_irrefl	THEOREM	302-306	slot-order irreflexivity
openSlotBefore_trans	THEOREM	307-313	slot-order transitivity
openSlotBefore_asymm	THEOREM	314-319	slot-order asymmetry
openSlotBefore_total_of_distinct_children	THEOREM	320-329	slot comparability under distinct-child hypothesis

The canonical machine-readable register is `derivation/registry/documentation/CODE_ANCHORS.tsv`. The stable anchor identity is source path plus symbol plus source SHA-256; line numbers are a human navigation aid and are regenerated after source edits.

Open-provenance role: Event provenance versus recurrent live dynamics

C018 orders birth events in an acyclic provenance history. Later live-state feedback may be recurrent or cyclic without turning the event-provenance relation itself into a cycle.

---

007 · P002 — Canonical schedule strict-total-order closure

Canonical node label: `007 · P002`
Semantic ID: `P002`
Current section path: `1.1.6.1`
Documentation tier: `D2`
Documentation state: `COMPLETE_V2`
Proof state: `SOURCE_COMPLETE_KERNEL_BUILD_REQUIRED`

Position In Derivation

P002 is the proof-certification child of `006 · C018`. Its hard mathematical input is the C003 provenance-address grammar, while its Lean proof module imports the C018 owner module in the permitted proof-to-core direction. The DAG certification edge points from P002 to C018; the Lean import points from `CNNAProofsP002` to `CNNA`.

P002 has one static responsibility:

> Prove that the C018 breadth-first/lexicographic relation is a strict total order on provenance addresses and induces a strict total order on open-slot selected children modulo extensional equality of the selected child address.

The node does not own state-dependent least-open selection. C004 owns least-open existence and uniqueness after C005 introduces the born prefix and unsaturation predicate.

Formal Statement

Let `BirthBefore` be the C018 order on provenance addresses. The public closure packages:

1. `¬ BirthBefore a a`;
2. `BirthBefore a b -> BirthBefore b c -> BirthBefore a c`;
3. asymmetry;
4. trichotomy `a < b ∨ a = b ∨ b < a`;
5. comparison of distinct addresses.

For C018 `OpenBirthSlot` records it packages irreflexivity, transitivity, and asymmetry of `OpenSlotBefore`, together with the extensional trichotomy

$$s <_{slot} t \ \vee \ \mathrm{child}(s) = \mathrm{child}(t) \ \vee \ t <_{slot} s.$$

For a predicate `Q` on slot records,

$$\mathrm{IsMinimalSelectedChild}(Q, s) \iff Q(s) \wedge \forall t\, [Q(t) \Rightarrow \neg(t <_{slot} s)].$$

The uniqueness theorem concludes equality of the selected child addresses of any two minimal witnesses.

Hypotheses

The order closure quantifies only over:

- a C003 branching parameter and provenance addresses;
- a C018 canonical schedule;
- C018 open-slot records;
- an arbitrary predicate on those records for the minimality theorem.

No `ResponseCapableState`, born prefix, unsaturation proof, numerical response, or Python execution is a P002 hypothesis.

Introduction Reason

C018 defines and proves the order primitives. P002 exposes their reusable proof contract without enlarging C018's core API and without importing later state layers. This gives downstream termination proofs one named certification boundary while preserving the package direction `CNNAProofsP002 -> CNNA`.

Proof Strategy

1. Construct the address fields directly from C018 `birthBefore_*` theorems.
2. Construct the slot fields directly from C018 `openSlotBefore_*` theorems.
3. Derive extensional slot trichotomy by applying address trichotomy to the two selected child addresses.
4. For minimal-witness uniqueness, split on equality of selected children.
5. Under inequality, use C018 total comparison of distinct selected children.
6. Each comparison direction contradicts one of the two minimality hypotheses.
7. Export the closure through a stable proposition-valued public contract.

Lemma Chain

```
C018.BirthBefore
  -> birthBefore_irrefl
  -> birthBefore_trans
  -> birthBefore_asymm
  -> birthBefore_trichotomy
  -> birthBefore_total_of_ne

C018.OpenSlotBefore
  -> openSlotBefore_irrefl
  -> openSlotBefore_trans
  -> openSlotBefore_asymm
  -> openSlotBefore_total_of_distinct_children

P002
  -> CanonicalScheduleStrictTotalOrderClosure
  -> canonicalScheduleStrictTotalOrderClosure
  -> IsMinimalSelectedChild
  -> minimalSelectedChild_unique
  -> CanonicalScheduleStrictTotalOrderContract
  -> canonicalScheduleStrictTotalOrderContract
```

Formal Realization

The proof source is:
`derivation/code/lean/proofs/src/CNNAProofs/Derivation/S01_PrimitiveResponseCoupledFiniteProvenance/S01_PrimitiveInputsGrammarAndEventOrder/S06_C018_CanonicalBfsLexicographicProvenanceBirthSchedule/Proofs/S01_P002_CanonicalScheduleStrictTotalOrderClosure.lean`

The independent root `proofs/src/CNNAProofsP002.lean` exports exactly this module. The separate library target prevents changes to the exact source sets already bound to the P001 and M003/M004 kernel evidence.

There is no P002 Python module. Static order closure is a theorem packaging task, not a second implementation of the C018 schedule.

Counterexamples Or Necessity Checks

1. **Remove transitivity:** two locally comparable steps no longer justify an earlier-than conclusion across a chain.
2. **Remove total comparison:** two distinct minimal selected children may remain incomparable, so uniqueness does not follow.
3. **Demand record equality:** proof-bearing slot records may encode the same selected child without being definitionally identical; the correct result is extensional child equality.

4. **Add least-open state selection:** the statement would require C005/C004 and create a backward dependency at node 007.
5. **Add Python agreement:** this would conflate a static theorem facade with the later executable selector owned by C004.

Axiom Profile

The P002 axiom audit enumerates all six public declarations with `#print axioms`. The source policy excludes project-local axioms and automation shortcuts. The exact transitive profile is not asserted until the local Lean 4.31.0 build runs; only `propext`, `Quot.sound`, and `Classical.choice` are permitted by the audit infrastructure.

Result

The contract and all six public declarations are implemented. Static package-boundary checks can verify source presence, imports, declaration coverage, forbidden-token absence, and hash-scope separation. Kernel verification remains pending the local build.

Remaining Limits

P002 does not prove:

- existence of an un-born address in an unsaturated C005 state;
- uniqueness of the C004 next-open address;
- equality with the Python positional selector;
- termination of the full finite birth process.

The first three are C004 responsibilities. Full finite schedule exhaustivity and termination belong to P004.

Downstream Handoff

`E141: P002 -> C018` certifies the owner closure after the local build. `E151: P002 -> P004` supplies the static order interface used by finite schedule exhaustivity. The obsolete `P002 -> P003` edge is absent because P003 consumes C018/C004 directly.

Code Line Register

Path: `derivation/code/lean/proofs/src/CNNAProofs/Derivation/S01_PrimitiveResponseCoupledFiniteProvenance/S01_PrimitiveInputsGrammarAndEventOrder/S06_C018_CanonicalBfsLexicographicProvenanceBirthSchedule/Proofs/S01_P002_CanonicalScheduleStrictTotalOrderClosure.lean`

- `CanonicalScheduleStrictTotalOrderClosure`, structure, lines 19–65.
- `canonicalScheduleStrictTotalOrderClosure`, theorem, lines 69–99.
- `IsMinimalSelectedChild`, definition, lines 104–109.
- `minimalSelectedChild_unique`, theorem, lines 115–131.
- `CanonicalScheduleStrictTotalOrderContract`, definition, lines 134–135.
- `canonicalScheduleStrictTotalOrderContract`, theorem, lines 138–140.

---

008 · C004A — First provenance slot s_1

Canonical node label: `008 · C004A`
Semantic ID: `C004A`
Current section path: `1.2.1`

Documentation tier: D1
Documentation state: COMPLETE_V2

Position In Derivation

C004A is canonical node 008 and the first node of the bootstrap group. Its verified hard predecessors are:

- E005: C003 -> C004A, supplying the finite b-ary provenance grammar;
- E064: C018 -> C004A, supplying the canonical breadth-first/lexicographic order.

The node must follow both predecessors. C003 alone gives the root, the slot alphabet and address extension, but does not select a first non-root address. C018 alone gives an order parameterized by a grammar, but cannot identify a slot without the same C003 instance. C004A joins these inputs and selects the first structural slot. The downstream edge E007: C004A -> C013 is ACTIVE_VERIFIED and transfers the slot to the actual first-birth construction.

Mathematical Contract

Let G be a C003 grammar with branching parameter $b \geq 2$, root address ε, local slot alphabet

$$S_b = \{0, 1, \ldots, b-1\},$$

and finite cutoff $L \geq 0$. Let Σ be the C018 schedule associated with the same grammar. The first provenance slot is the triple

$$s_1 = \big(\varepsilon, 0, \varepsilon\|(0)\big) = \big(\varepsilon, 0, (0)\big).$$

Its contract is:

1. the parent is the C003 root ε;
2. the stored local sibling rank is zero;
3. the intrinsic child address is (0);
4. (0) is the minimum non-root address under the C018 order;
5. the address is admitted into the finite approximant exactly when its depth one satisfies $1 \leq L$;
6. the node performs no birth and carries no physical payload.

The notation s_1 is one-based ordinal prose. The internal C003/C018 rank is zero-based. Therefore s_1 means first slot'', notslot with rank one''.

Introduction Reason

The derivation requires an exceptional bootstrap before the recurrent state exists. The root is already present, but there is not yet a born-prefix state from which a recurrent least-open slot could be selected. C004A therefore isolates the one structural slot that can be selected without any prior non-root birth: the rank-zero child of the root under the already-fixed C018 convention.

Keeping this selection separate from C013 is scientifically necessary. The address (0) is a provenance possibility. A birth additionally requires finite-cutoff admission and later construction of a new carrier state, directed relation and conductance data. Conflating those operations would hide the exceptional bootstrap assumption inside an address definition.

Explicit Construction

Predecessor object

The Python and Lean objects carry the same direct predecessor data:

```
grammar  : C003 FiniteBAryProvenanceGrammar
schedule : C018 CanonicalBirthSchedule
coherence: schedule.grammar = grammar
```

Python checks the coherence dynamically in `__post_init__`. Lean stores it as the proof field `schedule_grammar`.

Derived rank

The active domain $b \geq 2$ guarantees that zero is a valid local slot. Lean internalizes this by constructing

```
firstRank S : Fin S.grammar.branching.value
```

with value zero and a proof that $0 < b$. Python obtains the same value from the first element of the exact tuple `grammar.slots = (0,...,b-1)`.

Derived parent and address

The parent is not stored independently:

$$\text{parentAddress}(s_1) = \varepsilon.$$

The address is C003 right extension:

$$\text{address}(s_1) = \text{snoc}(\varepsilon, 0) = (0).$$

This construction is intrinsic and does not consult `L`.

Cutoff gate

Finite admission is the proposition

$$\text{WithinCutoff}(s_1) \iff |\text{address}(s_1)| \leq L.$$

Because the depth is exactly one, positive cutoffs admit the slot and zero cutoff rejects it. Python exposes the Boolean `admitted_by_cutoff` and the partial operation `require_admitted_address`; Lean exposes the proposition `WithinCutoff` and separate proofs for the positive and zero cases.

Invariants

Invariant	Mathematical statement	Formal evidence
Predecessor coherence	schedule and slot use one C003 grammar	Python `__post_init__`; Lean `schedule_grammar`
Least local rank	$\text{rank}(s_1) = 0$	Python `rank`; Lean `firstRank_val`
Root parent	$\text{parent}(s_1) = \varepsilon$	Python `parent`; Lean `parentAddress_root`
Intrinsic address	$\text{addr}(s_1) = \varepsilon \| 0 = (0)$	Python `address`; Lean `address_eq_snoc`
Depth	$\|\text{addr}(s_1)\| = 1$	Lean `address_depth`; Python tuple length

Invariant	Mathematical statement	Formal evidence
Parent recovery	`parent?((0)) = some(ε)`	Lean `address_parent`; Python C003 parent operation
Rank recovery	`finalSlot?((0)) = some(0)`	Lean `address_finalSlot`; Python C003 final-rank operation
Root-sibling minimality	(0) precedes every root child of nonzero rank	`firstRank_eq_or_before`, `address_before_distinct_root_sibling`
Global non-root minimality	every non-root address equals (0) or follows it	`address_eq_or_before_nonroot`
Strict global minimum	every distinct non-root address follows (0)	`address_before_distinct_nonroot`
Positive-cutoff admission	$1 \le L \Rightarrow (0) \in \mathcal{A}_{b,L}$	`withinCutoff_of_one_le`
Zero-cutoff exclusion	$L = 0 \Rightarrow (0) \notin \mathcal{A}_{b,0}$	`notWithinCutoff_at_zero`

Parent, rank, address and admission are derived facts. They cannot be supplied as mutually inconsistent constructor fields.

Canonicity Or Uniqueness

The sources establish canonicity at three levels.

Local-rank canonicity

For every local slot $r \in S_b$,

$$r = 0 \quad \vee \quad 0 < r.$$

Lean theorem `firstRank_eq_or_before` expresses exactly this dichotomy as equality with `firstRank` or strict C018 slot precedence.

Address canonicity

For every non-root address c,

$$c = (0) \quad \vee \quad (0) \prec_{\mathrm{BFSlex}} c.$$

The proof splits the address syntax:
- the empty word contradicts non-rootness;
- a depth-one word is either rank zero or a later sibling;
- a word of depth at least two follows (0) by breadth-first depth priority.

Together with C018 irreflexivity, this makes (0) the unique minimum non-root address.

Constructor canonicity boundary

`build_canonical_first_provenance_slot` and Lean `build` use the active C018 constructor. The formal source does not separately state a theorem that all inhabitants of the `FirstProvenanceSlot` structure are definitionally equal. The proved scientific claim is the canonicity of the extracted parent/rank/address data under coherent C003/C018 predecessors, not an overstrong global structure-equality theorem.

Boundary Cases

Cutoff L = 0

The C003 intrinsic word (0) exists, and C004A still computes the same structural slot. However:

$$\mathcal{A}_{b,0} = \{\varepsilon\}, \qquad (0) \notin \mathcal{A}_{b,0}.$$

Python therefore has `schedule.slots == ()`, `admitted_by_cutoff == False`, and `require_admitted_address()` raises `ValueError`. Lean theorem `notWithinCutoff_at_zero` proves the same exclusion proposition. No first non-root birth is possible at `L=0`.

Positive cutoff

For every tested and proved $L \geq 1$, (0) is admitted. This gate only returns or proves an admissible address; it does not perform C013.

One-based name versus zero-based rank

Replacing rank zero by rank one would select the second child and violate both C018 minimality and the source tests. The notation and stored rank must therefore remain distinct.

Incoherent predecessors

A grammar paired with a schedule based on another grammar is rejected. Without this condition, the rank/address projection could be interpreted in a different branching or cutoff context from the schedule order.

Active branching domain

The proof uses the project domain $b \geq 2$. The existence of rank zero would require only a nonempty alphabet, but C004A does not enlarge or alter the active CNNA input domain fixed by I001.

Python Lean Cross Layer

Concept	Python	Lean	Semantic relation
C004A carrier	frozen dataclass `FirstProvenanceSlot`	structure `FirstProvenanceSlot`	same two direct predecessors
predecessor coherence	runtime equality check	proof field `schedule_grammar`	same grammar identity condition
first rank	`grammar.slots[0]`	`Fin` value `0` with bound proof	same zero-based local slot
parent	`grammar.root`	`ProvenanceAddress.root`	same empty word
address	`child_address(..., (), 0)`	`ProvenanceAddress.snoc root firstRank`	same word (0)
cutoff admission	Boolean and rejecting accessor	proposition and theorems	same depth-one inequality

Concept	Python	Lean	Semantic relation
C018 minimality	first schedule element and `slot_precedes` tests	general theorems over all non-root addresses	Lean proves the universal statement exercised finitely by Python
payload boundary	dataclass field audit	structure fields only	no birth/physical payload in either layer

Python's `schedule.slots[0]` is available only for positive cutoff, whereas C004A's structural `address` is available independently of cutoff. Lean encodes the structural side directly and treats finite admission as a proposition. This is a representation difference, not a semantic mismatch.

Countercheck

The node-local Python suite contains eight targeted tests.

Test	Lines	Property or failure excluded
`test_s1_is_root_rank_zero_and_address_zero`	25-32	wrong parent, off-by-one rank or wrong child word across b, L samples
`test_one_based_name_does_not_shift_zero_based_rank`	34-37	confusion of ordinal s_1 with rank one
`test_c018_selects_s1_as_first_admitted_slot_when_L_positive`	39-52	disagreement with C018 enumeration or reverse precedence
`test_L_zero_retains_structural_slot_but_admits_no_birth_slot`	54-60	conflation of structural word with finite admission
`test_positive_cutoff_admits_s1_without_performing_birth`	62-67	failure of the depth-one cutoff gate or hidden birth side effect
`test_two_direct_predecessors_must_share_same_grammar`	69-74	incoherent C003/C018 predecessor join
`test_invalid_predecessor_types_are_rejected`	76-84	acceptance of foreign runtime values
`test_payload_contains_only_direct_predecessors`	86-100	hidden geometry, IDs, event time, conductance or response

The nontrivial Lean chain is:

```
firstRank_eq_or_before
  -> address_before_distinct_root_sibling

address_depth
  + firstRank_eq_or_before
  + C018.shallower_before
  -> address_eq_or_before_nonroot
```

```
  -> address_before_distinct_nonroot

address_depth
  -> withinCutoff_of_one_le
  -> notWithinCutoff_at_zero
```

The first chain proves local minimality, the second upgrades it to all non-root addresses, and the third isolates finite-cutoff admissibility. These are logically distinct obligations.

Result

`C004A` closes the statement:

> Given a C003 grammar and its coherent C018 schedule, the canonical first structural provenance slot has root parent, zero-based rank zero and address (0). This address is the unique minimum non-root address under the C018 order and belongs to the finite approximant exactly when $L \geq 1$.

The result is structural. It does not create the first non-root carrier element or any directed relation.

Downstream Handoff

- `E005: C003 -> C004A` supplies the address grammar and root.
- `E064: C018 -> C004A` supplies the strict schedule order.
- `E007: C004A -> C013` is `ACTIVE_VERIFIED` and supplies the first slot to the first non-root birth construction.

C013 additionally consumes A001 and N001 and requires cutoff admission. It owns the newborn vertex, the first directed relation and the initial conductances. C004A owns none of those objects.

Code Anchors

Python source

Path: `derivation/code/python/src/cnna/derivation/s01_primitive_response_coupled_finite_provenance/s02_bootstrap_of_the_first_relation/s01_c004a__first_provenance_slot_s1.py`
Source SHA-256: `b25b3e8a8c9f22c2611efe064c002af3690874c16a1a8ab51e68f6238dc70de8`

Symbol	Kind	Lines	Scientific role
`FirstProvenanceSlot`	`CLASS`	36-82	coherent predecessor carrier, derived slot data and cutoff gate
`build_first_provenance_slot`	`FUNCTION`	85-90	explicit two-predecessor constructor
`build_canonical_first_provenance_slot`	`FUNCTION`	93-99	constructor using the active C018 schedule

Python tests

Path: derivation/code/python/tests/derivation/s01_primitive_response_coupled_finite_provenance/s02_bootstrap_of_the_first_relation/test_s01_c004a__first_provenance_slot_s1.py
Source SHA-256: 2958ae8e902dea945ed5032fff08806f43005c64b04f642e341c44a37db2c11

Symbol	Kind	Lines	Scientific role
TestFirstProvenanceSlot	CLASS	19-100	eight executable contract and negative-control tests

Lean core

Path: derivation/code/lean/core/src/CNNA/Derivation/S01_PrimitiveResponseCoupledFiniteProvenance/S02_BootstrapOfTheFirstRelation/S01_C004A_FirstProvenanceSlotS1.lean
Source SHA-256: 5e3fffc01b8d380d477130872955ba9348ae8c0af813e18d07801438ca8b9947

Symbol	Kind	Lines	Scientific role
FirstProvenanceSlot	STRUCTURE	23-30	C003/C018 predecessor carrier with coherence proof
fromPredecessors	DEF	31-39	explicit coherent predecessor constructor
build	DEF	40-45	canonical constructor using C018 build
firstRank	DEF	46-49	bounded zero-based first slot
parentAddress	DEF	50-53	root parent definition
address	DEF	54-57	intrinsic child address (0)
firstRank_val	THEOREM	58-61	rank value equation
parentAddress_root	THEOREM	62-66	parent equals root
address_eq_snoc	THEOREM	67-71	child-word construction equation
address_depth	THEOREM	72-76	exact depth one
address_parent	THEOREM	77-88	C003 parent recovery
address_finalSlot	THEOREM	89-100	C003 final-rank recovery
firstRank_eq_or_before	THEOREM	101-112	least local-rank dichotomy

Symbol	Kind	Lines	Scientific role
address_before_distinct_root_sibling	THEOREM	113-127	strict precedence over other root children
address_eq_or_before_nonroot	THEOREM	128-156	global non-root minimum alternative
address_before_distinct_nonroot	THEOREM	157-168	strict global minimum for distinct addresses
WithinCutoff	DEF	169-172	finite-admission proposition
withinCutoff_of_one_le	THEOREM	173-179	positive-cutoff admission
notWithinCutoff_at_zero	THEOREM	180-189	zero-cutoff exclusion

The canonical machine-readable register is `derivation/registry/documentation/CODE_ANCHORS.tsv`. The stable anchor identity is source path plus symbol plus source SHA-256; line numbers are a direct reading aid and are regenerated after source edits.

---

009 · A001 — Genesis seed ⋆

Canonical node label: `009 · A001`
Semantic ID: `A001`
Current section path: `1.2.2`
Documentation tier: `D0`
Documentation state: `COMPLETE_V2`

Position In Derivation

A001 is an auxiliary graph root with no hard predecessor. It is placed after C004A in the reading order because C013 is the first construction that needs an explicit bootstrap argument. This placement does not mean that A001 is derived from C004A. Verified edge `E006: A001 -> C013` is its only direct hard handoff in the bootstrap block.

Definition Or Statement

The seed carrier is the singleton set

$$\mathcal{S}_\star = \{\star\}.$$

The element $\star$ has no numerical value and no address, parent, child, relation, conductance, response, birth time, event index, geometry or dynamical field. A001 is an explicit constructor token, not a free CNNA model input and not a physical initial state.

Introduction Reason

The first non-root birth is exceptional: before the first relation exists there is no nontrivial response network from which a recurrent birth law can be evaluated. An explicit bootstrap argument makes that exceptional constructor visible instead of hiding it in a null value or an undocumented special case. The argument must be information-free so that it cannot smuggle a third parameter into the derivation.

Construction Or Encoding

Python uses a frozen, slotted, zero-field dataclass `GenesisSeed` and the canonical instance `GENESIS_SEED`. Python can allocate distinct object identities, but every instance has the same empty dataclass value and therefore compares equal.

Lean uses the one-constructor inductive type

```
GenesisSeed.star
```

with `GenesisSeed.canonical := .star`. The theorem `eqCanonical` proves by constructor elimination that every value equals the canonical token. Python and Lean therefore implement the same singleton carrier while using language-appropriate object mechanics.

Boundary Case Or Countercheck

The node-local Python test checks two independent boundaries:

1. `dataclasses.fields(GenesisSeed) == ()`, so no hidden payload exists;
2. the canonical token lacks every downstream result-like attribute listed by the test.

A type with one constructor but a payload field would fail the A001 contract. Likewise, a numerical sentinel such as `0` would be semantically weaker because it belongs to a larger carrier and invites accidental arithmetic use. The singleton type prevents both failure modes.

Result

A001 closes the existence of exactly one explicit bootstrap token and proves at the type level that the token contains no choice. It does not prove that a downstream constructor ignores the token.

Downstream Handoff

- `E006: A001 -> C013` is `ACTIVE_VERIFIED` and supplies the information-free token.
- T001 later proves equality of the generated first-birth states for arbitrary explicit seed arguments.

Code Anchors

Python source

Path: `derivation/code/python/src/cnna/derivation/s01_primitive_response_coupled_finite_provenance/s02_bootstrap_of_the_first_relation/s02_a001__genesis_seed_star.py`
Source SHA-256: `3cdeb86555d9ed8468a0b1a0d27d0922fe4a6694458648312c0bd20934614f5f`

Symbol	Kind	Lines	Scientific role
`GenesisSeed`	`CLASS`	14-15	singleton carrier / zero-field runtime carrier

Python test

Path: `derivation/code/python/tests/derivation/s01_primitive_response_coupled_finite_provenance/s02_bootstrap_of_the_first_relation/test_s02_a001__genesis_seed_star.py`

Source SHA-256: 7f36399821677bd576ae63796e554cdfc987b846c49dd48a496feed940eca7e9

Symbol	Kind	Lines	Scientific role
TestGenesisSeed	CLASS	13-24	payload and singleton-value counterchecks

Lean core source

Path: derivation/code/lean/core/src/CNNA/Derivation/S01_PrimitiveResponseCoupledFiniteProvenance/S02_BootstrapOfTheFirstRelation/S02_A001_GenesisSeedStar.lean
Source SHA-256: 955dc9eafbdeea80ad1eabf998a0dcf576bfa632d2ae5e7a9e77ccb467cfb0d0

Symbol	Kind	Lines	Scientific role
GenesisSeed	INDUCTIVE	12-17	singleton carrier / zero-field runtime carrier
canonical	DEF	18-20	canonical singleton token
eqCanonical	THEOREM	21-27	universal singleton equality theorem

Registered anchors for A001: 5. Every path, line range and source hash is also present in derivation/registry/documentation/CODE_ANCHORS.tsv.

---

010 · N001 — Initial conductance normalization C⋆=1

Canonical node label: 010 · N001
Semantic ID: N001
Current section path: 1.2.3
Documentation tier: D0
Documentation state: COMPLETE_V2

Position In Derivation

N001 is a fixed-normalization graph root. It is introduced immediately before C013 because the first directed relation requires an initial conductance representative. It is independent of A001 and C004A. Its hard outgoing edges are E008: N001 -> C013 and E010: N001 -> M005.

Definition Or Statement

N001 fixes the dimensionless bootstrap representative

$$C_\star = 1$$

and the initial directed pair

$$(C_{r\to v_1}, C_{v_1\to r}) = (1, 1).$$

This is a normalization convention and not a free scalar parameter. N001 owns neither endpoint, the relation, the birth, nor the later proof that another positive unit is equivalent.

Introduction Reason

A weighted root-child relation must enter the recurrent development with a concrete representative. Choosing one removes an otherwise arbitrary common scale at the bootstrap boundary. The nontrivial justification that this choice does not add physical information must remain separate: N001 defines the representative; M005 proves the comparison theorem.

Construction Or Encoding

Python stores no payload in `InitialConductanceNormalization`. The properties `value` and `directed_values` return the module constant `C_STAR=1` and `(1,1)`.

Lean uses a one-constructor inductive type. `value` is definitionally `1 : Nat`; `directedValues` duplicates that value; `directedValues_eq_unit_pair` proves the pair is exactly `(1,1)` by reflexivity.

The local `Nat` carrier is part of the bootstrap representation only. M005 later embeds this numeral into `Rat`; later effective conductance carriers are not fixed by the present type choice.

Boundary Case Or Countercheck

Python rejects `InitialConductanceNormalization(2)`, demonstrating that 2 is not an alternative state of this node. The field audit excludes endpoint, relation, address, event and birth payloads. The Lean type has no alternative constructor or field.

This fixedness does not imply unit-independence. Without M005, the statement “one is only a representative” would be unsupported because no alternative positive unit appears in N001’s type.

Result

N001 closes the canonical bootstrap pair `(1,1)` and only that pair.

Downstream Handoff

- `E008: N001 -> C013` supplies the first-relation conductances.
- `E010: N001 -> M005` supplies the canonical representative whose scale class M005 analyzes.

Code Anchors

Python source

Path: `derivation/code/python/src/cnna/derivation/s01_primitive_response_coupled_finite_provenance/s02_bootstrap_of_the_first_relation/s03_n001__initial_conductance_normalization_c_star_1.py`
Source SHA-256: `d41e916f59a9b6c8b6fbec95f10f171a2f95714838f3948ac7f75112c7c8eab0`

Symbol	Kind	Lines	Scientific role
`InitialConductanceNormalization`	`CLASS`	23-34	fixed zero-payload normalization carrier

Python test

Path: derivation/code/python/tests/derivation/s01_primitive_response_coupled_finite_provenance/s02_bootstrap_of_the_first_relation/test_s03_n001__initial_conductance_normalization_c_star_1.py
Source SHA-256: fde5d880a23f305aed54815862fc8133bd7b209cc769a1d79048e9aa09a25327

Symbol	Kind	Lines	Scientific role
TestInitialConductanceNormalization	CLASS	14-29	unit-pair and no-free-value counterchecks

Lean core source

Path: derivation/code/lean/core/src/CNNA/Derivation/S01_PrimitiveResponseCoupledFiniteProvenance/S02_BootstrapOfTheFirstRelation/S03_N001_InitialConductanceNormalizationCStar1.lean
Source SHA-256: fec821addfc248edd0352e4977b46f92693b14bcac09de7080afe6a779c22c17

Symbol	Kind	Lines	Scientific role
InitialConductanceNormalization	INDUCTIVE	15-20	fixed zero-payload normalization carrier
canonical	DEF	21-23	canonical normalization token
value	DEF	24-26	fixed scalar representative one
directedValues	DEF	27-30	two-orientation initialization
directedValues_eq_unit_pair	THEOREM	31-37	exact unit-pair theorem

Registered anchors for N001: 7. Every path, line range and source hash is also present in derivation/registry/documentation/CODE_ANCHORS.tsv.

---

011 · C013 — First non-root provenance birth v1

Canonical node label: 011 · C013
Semantic ID: C013
Current section path: 1.2.4
Documentation tier: D1
Documentation state: COMPLETE_V2

Position In Derivation

C013 is the first node that joins all exceptional-bootstrap inputs:

- E007: C004A -> C013, the structural slot s_1;
- E006: A001 -> C013, the information-free seed token;
- E008: N001 -> C013, the unit conductance representative.

All three edges are ACTIVE_VERIFIED. The construction precedes T001 and C014 because those nodes certify and package its output.

Mathematical Contract

For a C004A slot

$$s_1 = (\varepsilon, 0, (0))$$

with a proof or executable check that $|(0)| \leq L$, and for the A001 token and N001 normalization, C013 constructs a first non-root birth B_1 satisfying

$$\text{root}(B_1) = \varepsilon, \qquad \text{newborn}(B_1) = (0),$$

$$\text{relations}(B_1) = ((\varepsilon, (0)), ((0), \varepsilon)), \qquad \text{conductances}(B_1) = (1, 1).$$

The seed is an input of the constructor but not a field of the output.

Introduction Reason

The recurrent rule cannot produce the first relation from a pre-existing response because no nontrivial weighted network exists before that relation. C013 isolates this unavoidable exceptional initialization instead of applying the later response-coupled law outside its domain.

Explicit Construction

Python defines the immutable record

```
FirstNonRootBirth(slot, normalization)
```

and derives the root, newborn, two directed orientations and conductance pair by projection. `build_first_non_root_birth` verifies the exact predecessor types and calls `slot.require_admitted_address()` before returning the record.

Lean defines

```
structure FirstNonRootBirth where
  slot          : FirstProvenanceSlot
  normalization : InitialConductanceNormalization
  withinCutoff  : FirstProvenanceSlot.WithinCutoff slot
```

and `FirstNonRootBirth.build` accepts the explicit seed but does not store it. The proof of finite admission is carried in the result type.

Invariants

The current Lean source proves:

1. `newborn_eq_first_slot`: the newborn address is exactly the C004A address;
2. `newborn_parent_root`: C003 parent reconstruction returns the root;
3. `directedConductances_eq_unit_pair`: the two orientations are `(1,1)`.

The directed relation is stored in both orientations, but this symmetry is only the bootstrap initialization. It is not a claim that all later live directed conductances remain symmetric.

Canonicity Or Uniqueness

Given the fixed C004A slot and N001 normalization, the endpoint, relation and conductance data are definitional projections. A001 carries no information, and the generated record contains no seed field. Nevertheless, C013 deliberately does not own the theorem comparing two explicit seed-indexed constructor calls; T001 owns that equality.

No separate theorem states uniqueness among all conceivable records satisfying the displayed equations. The closed claim is the canonicity of the provided constructor and its proved projections.

Boundary Cases

- For $L \geq 1$, C004A supplies a cutoff proof and C013 is constructible.
- For $L = 0$, the word (0) remains structurally defined, but the Python guard rejects it and no Lean value can be built without the false cutoff proposition.
- C013 computes no response, geometry, event number or time.

Python Lean Cross Layer

Aspect	Python	Lean	Semantic relation
predecessor typing	exact runtime `type` checks	static types	same admissible inputs
cutoff	rejecting method call	proposition argument stored in result	same depth-one gate
seed	explicit argument, not stored	explicit argument, not stored	same erasure boundary
endpoints and relations	properties	definitions	same C004A projections
conductances	N001 property	N001 definition and theorem	same `(1,1)` pair

The representation difference for the cutoff proof is intentional and does not create a semantic mismatch.

Countercheck

The focused Python tests construct the complete first relation at $L = 1$, verify both directed orientations and the unit pair, reject $L = 0$, and audit the dataclass fields as exactly `("slot", "normalization")`. These tests would fail if the seed were retained, if the child rank were shifted, if one orientation were omitted, or if a response-dependent value entered the bootstrap.

Result

C013 closes the exceptional first non-root birth and its initial weighted directed relation. Its registered obligation is `CLOSED_VERIFIED` and is supported by the listed source theorems.

Downstream Handoff

- `E009: C013 -> T001` requests the explicit seed-neutrality theorem.
- `E011: C013 -> C014` supplies the concrete birth record.

Both edges are `ACTIVE_VERIFIED`.

Code Anchors

Python source

Path: `derivation/code/python/src/cnna/derivation/s01_primitive_response_coupled_finite_provenance/s02_bootstrap_of_the_first_relation/s04_c013__first_non_root_provenance_birth_v1.py`
Source SHA-256: `02d01363b5e7746aff7358f21d65aa44c8128795f9f6e4be0ac0d4315249251f`

Symbol	Kind	Lines	Scientific role
`FirstNonRootBirth`	`CLASS`	23-43	first-birth output record
`build_first_non_root_birth`	`FUNCTION`	46-63	checked executable first-birth constructor

Python test

Path: `derivation/code/python/tests/derivation/s01_primitive_response_coupled_finite_provenance/s02_bootstrap_of_the_first_relation/test_s04_c013__first_non_root_provenance_birth_v1.py`
Source SHA-256: `1a22046c8bae87bc8424c4ccc0f5d2ac2cd975e658788a2e7330ec043ded4b84`

Symbol	Kind	Lines	Scientific role
`slot_at_depth`	`FUNCTION`	17-19	test fixture exposing cutoff dependence
`TestFirstNonRootBirth`	`CLASS`	22-33	endpoint, relation, conductance and L=0 tests

Lean core source

Path: `derivation/code/lean/core/src/CNNA/Derivation/S01_PrimitiveResponseCoupledFiniteProvenance/S02_BootstrapOfTheFirstRelation/S04_C013_FirstNonRootProvenanceBirthV1.lean`
Source SHA-256: `327bb483c673c175bba16bd30964f4b869a3a4d8f41d696d7f8a02fa06c9f78a`

Symbol	Kind	Lines	Scientific role
`FirstNonRootBirth`	`STRUCTURE`	22-29	first-birth output record
`build`	`DEF`	30-39	typed first-birth constructor
`rootAddress`	`DEF`	40-43	root endpoint projection
`newbornAddress`	`DEF`	44-47	C004A address projection
`directedRelations`	`DEF`	48-53	two stored orientations
`directedConductances`	`DEF`	54-57	N001 pair projection
`newborn_eq_first_slot`	`THEOREM`	58-62	newborn/C004A identity theorem
`newborn_parent_root`	`THEOREM`	63-69	parent reconstruction theorem
`directedConductances_eq_unit_pair`	`THEOREM`	70-77	unit-pair theorem

Registered anchors for C013: 13. Every path, line range and source hash is also present in `derivation/registry/documentation/CODE_ANCHORS.tsv`.

---

012 · T001 — Seed-neutrality theorem for first birth

Canonical node label: `012 · T001`
Semantic ID: `T001`
Current section path: `1.2.5`
Documentation tier: `D2`
Documentation state: `COMPLETE_V2`

Position In Derivation

T001 follows C013 through verified edge `E009`. A001 proves only that the seed carrier is singleton-valued; C013 shows that the seed is not stored. T001 is the theorem node that turns this design into an explicit equality statement and passes the certificate to C014 through `E012`.

Formal Statement

For any C004A slot `slot`, seeds `eta etaPrime : GenesisSeed`, N001 normalization `normalization`, and cutoff proof `h`, the current Lean theorem states

```
firstWeightedStateFromSeed slot eta normalization h =
  firstWeightedStateFromSeed slot etaPrime normalization h
∧ directedConductances (...) = (1, 1)
```

Equivalently,

$$B(slot, \eta, N, h) = B(slot, \eta', N, h) \quad \wedge \quad \mathcal{C}(B(slot, \eta, N, h)) = (1, 1).$$

Hypotheses

- one fixed structural C004A slot;
- two arbitrary explicit A001 seed values;
- one fixed N001 normalization;
- admission of the slot into the finite cutoff.

No response, geometry, time or recurrent state is assumed.

Introduction Reason

The seed is needed to expose the exceptional bootstrap boundary, but the model must not inherit a hidden state variable from it. Keeping T001 separate from A001 and C013 makes the no-information claim falsifiable: a later modification that stores or uses the seed would break the theorem rather than silently changing the meaning of the token.

Proof Strategy

`firstWeightedStateFromSeed` is a transparent wrapper around `FirstNonRootBirth.build`. The build function ignores its seed argument when creating the record. Therefore both generated states reduce to the same term and the equality branch is proved by `rfl`. The conductance branch invokes C013's `directedConductances_eq_unit_pair`.

Lemma Chain

```
C013.FirstNonRootBirth.build
  -> firstWeightedStateFromSeed
  -> definitional equality under eta / etaPrime

C013.directedConductances_eq_unit_pair
  -> second conjunct of seedNeutralityFirstBirth
```

A001's `eqCanonical` is available but is not used. This matters: the current theorem follows from structural seed erasure, not merely from rewriting all seeds to the unique constructor.

Formal Realization

Python's `first_weighted_state_from_seed` calls the same C013 builder. The test creates two distinct `GenesisSeed()` objects, verifies distinct identity, and then checks equality of the generated records and the pair `(1,1)`.

Lean quantifies over arbitrary seed values and proves the conjunction in the mathlib-free core. The theorem is kernel-checkable with the package's pinned Lean version and depends only on the imported C013 definitions and theorem.

Counterexamples Or Necessity Checks

- Adding a `seed` field to `FirstNonRootBirth` would destroy the direct `rfl` proof unless equality were separately quotiented or rewritten.
- Using the seed to choose a child address or conductance would invalidate the theorem even though A001 currently has one constructor; it would reveal a meaningless but structurally present dependency.
- Removing the common cutoff hypothesis would make the two terms ill-typed, because C013 is not defined outside the admitted first slot.

Axiom Profile

The theorem body uses `rfl` and an exact imported theorem. The source is in the dependency-free core package and contains no project `axiom`, `sorry`, `admit`, `opaque`, `unsafe` or `partial` declaration; its listed source hash is the verified current hash.

Result

T001 proves exact state equality, not merely equality of selected observables. The first birth leaves no seed state variable behind.

Remaining Limits

The theorem is restricted to the exceptional first birth. It says nothing about hypothetical seed arguments in later recurrent steps, and it does not prove M005's unit-independence statement.

Downstream Handoff

`E012: T001 -> C014` is `ACTIVE_VERIFIED` and certifies the seed-neutrality component of the packaged bootstrap state.

Code Line Register

Python source

Path: derivation/code/python/src/cnna/derivation/s01_primitive_response_coupled_finite_provenance/s02_bootstrap_of_the_first_relation/s05_t001__seed_neutrality_theorem_for_first_birth.py
Source SHA-256: 0fd2df73db6f59d70b83d22c58fa8089b418061e7ecfe4c8d4f1c167cd3beed4

Symbol	Kind	Lines	Scientific role
first_weighted_state_from_seed	FUNCTION	15-21	executable seed-indexed wrapper

Python test

Path: derivation/code/python/tests/derivation/s01_primitive_response_coupled_finite_provenance/s02_bootstrap_of_the_first_relation/test_s05_t001__seed_neutrality_theorem_for_first_birth.py
Source SHA-256: 24a74c3e5fc36b2d4cae7e8d1c517598e6d78556c9ed0e5645512254f82863da

Symbol	Kind	Lines	Scientific role
TestSeedNeutralityFirstBirth	CLASS	16-27	distinct-instance equality test

Lean core source

Path: derivation/code/lean/core/src/CNNA/Derivation/S01_PrimitiveResponseCoupledFiniteProvenance/S02_BootstrapOfTheFirstRelation/S05_T001_SeedNeutralityTheoremForFirstBirth.lean
Source SHA-256: e2da5d1e066d13c176bc9e05a0c3c892ee60c8169c7ddb3d050428f4e675b705

Symbol	Kind	Lines	Scientific role
firstWeightedStateFromSeed	DEF	15-26	formal seed-indexed wrapper
seedNeutralityFirstBirth	THEOREM	27-41	universal state equality and conductance theorem

Registered anchors for T001: 4. Every path, line range and source hash is also present in derivation/registry/documentation/CODE_ANCHORS.tsv.

---

013 · M005 — Conductance-unit normalization independence

Canonical node label: 013 · M005
Semantic ID: M005

Current section path: 1.2.6
Documentation tier: D2
Documentation state: COMPLETE_V2

Position In Derivation

M005 follows N001 through verified edge E010. N001 fixes one representative but cannot by itself establish representative independence. M005 introduces the comparison relation and proves the positive common-rescaling theorem before C014 uses it on the actual stored bootstrap conductances.

Formal Statement

Lean defines

$$\text{SameNormalizedResponse}(R, C, R', C') \quad :\Longleftrightarrow \quad RC' = R'C.$$

The source proves:

1. for scale > 0,
$$(R, C) \sim (scale\, R, scale\, C);$$
2. the rational lift of N001 is exactly one;
3. for every positive rational unit u and normalized value x,
$$(x, 1) \sim (ux, u).$$

Python realizes the quotient interpretation as exact Fraction(response) / Fraction(unit) with a positive-unit guard.

Hypotheses

- rational scalar responses and conductance units;
- a positive comparison scale for an admissible unit change;
- in the canonical-representative theorem, a positive alternative unit.

The algebraic cross-product relation itself is defined for all rationals, but its interpretation as equality of quotients requires nonzero units; the CNNA conductance convention strengthens this to positivity.

Introduction Reason

Without M005, fixing C⋆=1 would merely hide an unexamined choice inside a singleton type. The present theorem introduces an explicit family of alternative positive representatives and proves that the dimensionless normalized datum is unchanged. This makes the claim “no third physical input” falsifiable.

Proof Strategy

The common-rescaling theorem avoids division:

$$R(\lambda C) = (R\lambda)C = (\lambda R)C.$$

The canonical-representative theorem first rewrites the N001 rational lift to one and then uses commutativity and the right-unit law. Exact rational arithmetic is used throughout.

Lemma Chain

```
InitialConductanceNormalization.value
  -> n001ConductanceUnit
  -> n001ConductanceUnit_eq_one

SameNormalizedResponse
  + Rat.mul_assoc
  + Rat.mul_comm
  -> commonPositiveRescalingPreservesNormalizedResponse

n001ConductanceUnit_eq_one
  + Rat.mul_comm
  + Rat.mul_one
  -> n001CanonicalRepresentativeForPositiveUnit
```

Formal Realization

Python's `normalized_response` rejects a nonpositive unit and computes an exact `Fraction`. `common_positive_rescaling_preserves_normalized_response` rejects a nonpositive scale and compares the pre- and post-rescaling quotients exactly. The test includes positive integer and proper-fraction scales, positive and negative response values, and verifies that N001 normalization returns the exact response coordinate.

Lean imports only the core rational lemmas and N001. The theorem is division-free, so no inverse or regularization enters the proof. Positivity is retained as the semantic admissibility condition even though the polynomial identity does not consume it computationally.

Counterexamples Or Necessity Checks

- `(R,C) -> (lambda R,C)` changes the quotient unless $\lambda = 1$ or $R = 0$.
- `(R,C) -> (R,lambda C)` likewise changes it in general.
- `lambda=0` satisfies the bare cross-product equation but destroys the unit and is rejected by Python and by the positive-scale hypothesis.
- `lambda<0` preserves an algebraic ratio but leaves the positive-conductance domain.
- Edge-dependent or orientation-dependent rescalings are not covered by the theorem.

Axiom Profile

The Lean source is in the mathlib-free core package, uses transparent `Rat` definitions and elementary rational lemmas, and contains no project axiom or admitted proof.

Result

N001's value one is a canonical representative of a positive rational scaling class for the scalar normalized response. The comparison variable is not a CNNA input and is not stored in the state.

Remaining Limits

M005 does not prove homogeneity of the entire matrix-valued Schur/DtN construction and does not establish local gauge symmetry. Its exact downstream role is narrower: it supplies the fixed positive unit representative `C⋆=1` consumed by M003 after the C015 identity transform.

Downstream Handoff

- `E013: M005 -> C014` is `ACTIVE_VERIFIED` and certifies the actual bootstrap conductance pair.
- `E068: M005 -> M003` is `ACTIVE_VERIFIED` and supplies the fixed unit representative used by the canonical steering scalar.

Code Line Register

Python source

Path: `derivation/code/python/src/cnna/derivation/s01_primitive_response_coupled_finite_provenance/s02_bootstrap_of_the_first_relation/s06_m005__conductance_unit_normalization_independence.py`
Source SHA-256: `ba2e0d83204355d034d00d155442d024a8d700a91eebe92a6d55d8761bea73ba`

Symbol	Kind	Lines	Scientific role
`normalized_response`	`FUNCTION`	19-24	exact positive-unit quotient
`n001_normalized_response`	`FUNCTION`	27-32	N001 representative quotient
`common_positive_rescaling_preserves_normalized_response`	`FUNCTION`	35-46	executable exact rescaling check

Python test

Path: `derivation/code/python/tests/derivation/s01_primitive_response_coupled_finite_provenance/s02_bootstrap_of_the_first_relation/test_s06_m005__conductance_unit_normalization_independence.py`
Source SHA-256: `b0da8ba116abef2a971d43861c476f0fa31d7e84bf7eaac238adf6202cc8d0aa`

Symbol	Kind	Lines	Scientific role
`TestConductanceUnitNormalizationIndependence`	`CLASS`	14-25	positive rational scaling tests

Lean core source

Path: `derivation/code/lean/core/src/CNNA/Derivation/S01_PrimitiveResponseCoupledFiniteProvenance/S02_BootstrapOfTheFirstRelation/S06_M005_ConductanceUnitNormalizationIndependence.lean`
Source SHA-256: `fbb6b498f04c04acf21ab84c55f1ac32093bac845a08a6daae69d20f5820e221`

Symbol	Kind	Lines	Scientific role
`SameNormalizedResponse`	`DEF`	19-23	division-free cross-product relation

Symbol	Kind	Lines	Scientific role
n001ConductanceUnit	DEF	24-27	rational lift of N001
n001ConductanceUnit_eq_one	THEOREM	28-38	exact lift theorem
commonPositiveRescalingPreservesNormalizedResponse	THEOREM	39-55	common positive scaling theorem
n001CanonicalRepresentativeForPositiveUnit	THEOREM	56-73	canonical-representative theorem

Registered anchors for M005: 9. Every path, line range and source hash is also present in derivation/registry/documentation/CODE_ANCHORS.tsv.

---

014 · C014 — Bootstrap state X_1

Canonical node label: 014 · C014
Semantic ID: C014
Current section path: 1.2.7
Documentation tier: D1
Documentation state: COMPLETE_V2

Position In Derivation

C014 is the terminal construction of the exceptional-bootstrap group. It has three verified hard predecessors:

- E011: C013 -> C014, the concrete first birth;
- E012: T001 -> C014, seed-neutrality;
- E013: M005 -> C014, unit-representative independence.

Its verified outgoing edge E014: C014 -> C005 supplies the base case for recurrent states.

Mathematical Contract

The state X_1 contains

$$V_1 = \{\varepsilon, (0)\}, \quad \mathcal{R}_1 = ((\varepsilon, (0)), ((0), \varepsilon)), \quad \mathcal{C}_1 = (1, 1).$$

The implementation represents this information minimally by storing the C013 birth once and deriving the displayed components by projection. The set notation is a mathematical view, not a claim that the runtime object contains a redundant set field.

Introduction Reason

C013 creates the exceptional relation, but the recurrent layer needs a named state object that packages the relation and carries the two no-hidden-input certificates. C014 marks the exact transition from initialization to the domain on which later response constructions can act.

Explicit Construction

Python defines

```
BootstrapState(birth : FirstNonRootBirth)
```

with projections `root`, `newborn`, `directed_relations` and `directed_conductances`. `build_bootstrap_state` is the one-field constructor.

Lean defines the same one-field structure and constructor. It then provides:

- root and newborn projections;
- the inherited unit-pair theorem;
- `fromSeed_seedNeutral`, obtained by applying `congrArg build` to T001;
- `directedConductancesRat`, the rational lift of the actual stored pair;
- `directedConductancesRat_eq_n001_pair`;
- `conductanceUnit_isRepresentativeOnly`, which applies M005 to each orientation.

Invariants

1. the carrier view contains exactly root and first newborn;
2. the relation contains both provenance-parent orientations;
3. both stored conductances are one;
4. no seed field is present;
5. no variable unit field is present;
6. the comparison scale used by M005 produces no alternative `BootstrapState` value.

Canonicity Or Uniqueness

For each C013 birth there is exactly the transparent one-field wrapper produced by `build`. C014 does not prove uniqueness of C013 itself; it inherits C013's data. The T001 theorem proves independence from explicit seed arguments, and M005 proves representative invariance of the normalized conductance coordinate. Together these exclude the two intended hidden bootstrap choices.

Boundary Cases

- $L = 0$: C013 cannot be constructed, hence neither can C014.
- $L \geq 1$: C014 is available once the C013 birth is supplied.
- The phrase "response-capable" means a nontrivial weighted relation exists. No Schur complement, DtN response, steering scalar, geometry, event time or recurrent update is evaluated here.

Python Lean Cross Layer

Python stores only the birth and exposes executable projections. Lean stores the same birth and adds proof theorems around it. These theorems do not enlarge the runtime payload. The rational lift in Lean is a proof-side view of the natural-number pair used for M005; Python's C014 module does not duplicate that proof because the exact scalar theorem is tested in M005's module.

Countercheck

The Python test constructs the complete chain from grammar through C014, checks root, newborn, both relations and both conductances, and audits the dataclass fields as exactly `("birth",)`. The constructor's `__post_init__` rejects a value of any other exact type. These checks exclude duplicated state components and hidden certificate variables.

Result

C014 closes the first response-capable weighted provenance state and the exceptional initialization section. The D6 registry synchronization marks its obsolete blocked-obligation row as `CLOSED_VERIFIED`; no mathematical source is changed.

Downstream Handoff

- `E014: C014 -> C005` is `ACTIVE_VERIFIED` and supplies the recurrent base case.
- `E077: C014 -> C019` and `E153: C014 -> P005` remain `PLANNED_BLOCKED`; they require the later recurrent iteration and proof closure.

Code Anchors

Python source

Path: `derivation/code/python/src/cnna/derivation/s01_primitive_response_coupled_finite_provenance/s02_bootstrap_of_the_first_relation/s07_c014__bootstrap_state_x1_r_v1_c_r_v1_1.py`
Source SHA-256: `3b2115e677362fae2c1205e74b37f020a93f2a069e31a560484d899dd38d6f28`

Symbol	Kind	Lines	Scientific role
`BootstrapState`	`CLASS`	23-46	minimal one-field bootstrap-state carrier
`build_bootstrap_state`	`FUNCTION`	49-51	executable wrapper constructor

Python test

Path: `derivation/code/python/tests/derivation/s01_primitive_response_coupled_finite_provenance/s02_bootstrap_of_the_first_relation/test_s07_c014__bootstrap_state_x1_r_v1_c_r_v1_1.py`
Source SHA-256: `efa56d83c92ded843c844b52f9271816a58bf9f333a8942c109d1103334b8355`

Symbol	Kind	Lines	Scientific role
`TestBootstrapState`	`CLASS`	18-30	full state and payload-minimality test

Lean core source

Path: `derivation/code/lean/core/src/CNNA/Derivation/S01_PrimitiveResponseCoupledFiniteProvenance/S02_BootstrapOfTheFirstRelation/S07_C014_BootstrapStateX1RV1CRV11.lean`
Source SHA-256: `6894fdaf9854f4d7251d828bf756a54aa2730d94fe84f733acb9d829c57e65f5`

Symbol	Kind	Lines	Scientific role
BootstrapState	STRUCTURE	23-28	minimal one-field bootstrap-state carrier
build	DEF	29-32	formal wrapper constructor
rootAddress	DEF	33-36	root projection
newbornAddress	DEF	37-40	newborn projection
directedConductances_eq_unit_pair	THEOREM	41-45	inherited unit-pair theorem
fromSeed_seedNeutral	THEOREM	46-55	T001 certificate lifted to X1
directedConductancesRat	DEF	56-60	rational lift of actual pair
directedConductancesRat_eq_n001_pair	THEOREM	61-74	actual pair/N001 identity theorem
conductanceUnit_isRepresentativeOnly	THEOREM	75-99	M005 applied to both stored orientations

Registered anchors for C014: 12. Every path, line range and source hash is also present in derivation/registry/documentation/CODE_ANCHORS.tsv.

---

015 · C005 — Response-capable state schema X_n, n >= 1

Canonical node label: 015 · C005
Current section path: 1.3.1
Documentation tier: D1

Position In Derivation

C005 begins the recurrent layer after C014. It is the domain of every later pre-birth measurement and update step.

Mathematical Contract

A state X_n consists of a C003 grammar, its C018 schedule, a nonempty list bornNonRoot, and a finite list of positive directed conductances. The mathematical birth count is n = bornNonRoot.length, hence n >= 1. The born non-root list is exactly the first n schedule children. The carrier is the root together with this prefix.

Introduction Reason

C014 supplies only the exceptional base state. Recurrent growth needs a stable state type independent of the next-slot, cut, response, steering, and update constructions.

Explicit Construction

`DirectedConductance` stores a source address, a distinct target address, and a strictly positive exact rational value. `ResponseCapableState` stores the grammar, schedule, born prefix, and conductance list. `fromBootstrap` transports the C014 root/newborn pair and its two directed unit values into the recurrent representation.

Invariants

- grammar and schedule share the same branching parameter and cutoff;
- `bornNonRoot` is nonempty, duplicate-free, cutoff-admissible, and equal to the initial C018 prefix;
- every conductance has born endpoints, positive value, and distinct endpoints;
- ordered conductance pairs are unique;
- every born non-root address has both positive parent orientations;
- additional directed edges between already-born vertices are permitted.

Canonicity Or Uniqueness

C005 does not claim a unique state for fixed `n`; live conductance values may differ. It claims a unique schema and a canonical embedding of C014. The theorem `fromBootstrap_n` fixes the base image at `n=1`.

Boundary Cases

The schema excludes `n=0`; that regime is owned by C001/C002 before the first non-root birth. Self-loops, nonpositive values, unborn endpoints, duplicate ordered pairs, and non-prefix birth lists are rejected. Saturation is allowed as a state condition but yields no C004 successor.

Python Lean Cross Layer

Python enforces the invariants in immutable dataclasses. Lean stores the same obligations as fields of `ResponseCapableState` and proves the C014 transport facts. Lean does not need to mirror Python exception classes; the semantic agreement is the accepted-state predicate.

Countercheck

Removing the initial-prefix condition would make positional C004 selection unsound. Removing either parent orientation would destroy the guaranteed bidirectional provenance backbone. Forbidding additional born-born edges would impose an unsupported tree-only live network.

Result

C005 is a verified response-capable recurrent state domain with exact rational directed conductances and a canonical C014 base inhabitant.

Downstream Handoff

- `E015` to C004: exposes the next open slot;
- `E016` to M001: supplies the born carrier;
- `E022` to C007: supplies the current directed network;
- later codomain/update gates remain open.

Code Anchors

python

Path: derivation/code/python/src/cnna/derivation/s01_primitive_response_coupled_finite_provenance/s03_recurrent_pre_birth_measurement_and_steering/s01_c005__response_capable_state_schema_xn_n_ge_1.py Source SHA-256: 96b9867f44f4a5021cd017033e796f41e0fcc9195c09505796129bdbd469f240

- DirectedConductance - CLASS, lines 24-37; role SOURCE.
- ResponseCapableState - CLASS, lines 41-104; role SOURCE.
- response_capable_state_from_bootstrap - FUNCTION, lines 107-124; role SOURCE.

python_test

Path: derivation/code/python/tests/derivation/s01_primitive_response_coupled_finite_provenance/s03_recurrent_pre_birth_measurement_and_steering/test_s01_c005__response_capable_state_schema_xn_n_ge_1.py Source SHA-256: 76c931c265f48c2abaac94e149d867f8c7c6871798465855b52a3baf9e070579

- _x1 - FUNCTION, lines 19-25; role TEST.
- TestResponseCapableStateSchema - CLASS, lines 28-59; role TEST.

lean_core

Path: derivation/code/lean/core/src/CNNA/Derivation/S01_PrimitiveResponseCoupledFiniteProvenance/S03_RecurrentPreBirthMeasurementAndSteering/S01_C005_ResponseCapableStateSchemaXnNGe1.lean Source SHA-256: 732f5cf1227d57955c144b99043f0c58bb6535ae3a3d2bf3ac584385df0cb6cc

- DirectedConductance - STRUCTURE, lines 26-33; role SOURCE.
- NodeBorn - DEF, lines 34-39; role SOURCE.
- HasConductance - DEF, lines 40-45; role SOURCE.
- DistinctConductancePair - DEF, lines 46-50; role SOURCE.
- ResponseCapableState - STRUCTURE, lines 51-81; role SOURCE.
- n - DEF, lines 82-85; role SOURCE.
- one_le_n - THEOREM, lines 86-95; role SOURCE.
- rootAddress - DEF, lines 96-99; role SOURCE.
- rootBorn - THEOREM, lines 100-104; role SOURCE.
- bootstrap_root_ne_newborn - THEOREM, lines 105-116; role SOURCE.
- bootstrapForwardConductance - DEF, lines 117-127; role SOURCE.
- bootstrapBackwardConductance - DEF, lines 128-138; role SOURCE.
- base_case_transports_c014_forward_value - THEOREM, lines 139-144; role SOURCE.
- base_case_transports_c014_backward_value - THEOREM, lines 145-150; role SOURCE.
- bootstrap_bornOrdered - THEOREM, lines 151-158; role SOURCE.
- bootstrap_bornInitial - THEOREM, lines 159-178; role SOURCE.
- bootstrap_conductancePairsUnique - THEOREM, lines 179-194; role SOURCE.
- fromBootstrap - DEF, lines 195-245; role SOURCE.
- fromBootstrap_n - THEOREM, lines 246-251; role SOURCE.

Open-provenance role: Sufficient finite provenance state

C005 is the current finite candidate for a provenance-sufficient state: it carries the born prefix and response data needed by the next CNNA update. Whether an arbitrary empirical reduced state admits such a completion is outside the present theorem scope.

---

016 · C004 — Next open provenance slot s_{n+1}

Canonical node label: `016 · C004`
Current section path: `1.3.2`
Documentation tier: `D1`

Position In Derivation

C004 consumes a recurrent C005 state and the already fixed C018 order. It is the unique structural selector used by M001 and later birth nodes.

Mathematical Contract

For an unsaturated state `X`, an admissible open address is a non-root C003 address within cutoff that is not born. `IsNextOpenAddress X a` means that `a` is admissible and no admissible open address precedes it under C018 `BirthBefore`. `NextOpenSlot X` packages this child with its uniquely reconstructed parent and final rank.

Introduction Reason

The recurrent layer needs the next structural provenance location before it can define a local cut or measure a response. This selector must use no response value, geometry, or birth law.

Explicit Construction

Python returns `state.schedule.slots[state.birth_count]` after checking unsaturation and verifies it through `is_next_open_provenance_slot`. Lean finitely enumerates the C003 carrier, filters open candidates, and chooses the least candidate constructively under C018.

Invariants

The child is non-root, cutoff-admissible, not born, and least among all open addresses. Every admissible predecessor is born. The reconstructed parent is born and the child equals `parent.snoc(rank)`.

Canonicity Or Uniqueness

`exists_of_unsaturated` proves existence; `child_unique` and `unique` prove uniqueness. `parent_rank_unique` proves that the child determines exactly one parent/rank pair.

Boundary Cases

For a saturated finite approximant, `no_next_of_saturated` proves that no successor exists. No sentinel or out-of-cutoff child is introduced. At `L=0`, every recurrent state premise is already unavailable because C014 cannot be formed.

Python Lean Cross Layer

Python is positional because C005 proves the born list is the exact schedule prefix. Lean is extensional and proves least-open uniqueness. Their semantic lock is the independent Python predicate matching the Lean relation.

Countercheck

An arbitrary un-born child would not be canonical. A second schedule would duplicate C018. A sentinel would falsely add a provenance address beyond the finite carrier. Omitting parent-bornness would invalidate the M001 causal prefix cut.

Result

C004 provides exactly one next open provenance slot for every unsaturated C005 state and none for a saturated state.

Downstream Handoff

- E017 to M001 localizes the cut;
- E026 to M004 is ACTIVE_VERIFIED and supplies the unique next provenance slot used by the birth instruction;
- C004 also supplies the verified dynamic least-open content that P002 had attempted to own too early.

Code Anchors

python

Path: derivation/code/python/src/cnna/derivation/s01_primitive_response_coupled_finite_provenance/s03_recurrent_pre_birth_measurement_and_steering/s02_c004__next_open_provenance_slot_snplus1_n_ge_1.py Source SHA-256: d939caa5a08845e0ee8faa737e79efebdec29031de08d5d52b51f77c3aab0535

- is_next_open_provenance_slot - FUNCTION, lines 24-51; role SOURCE.
- next_open_provenance_slot - FUNCTION, lines 54-91; role SOURCE.

python_test

Path: derivation/code/python/tests/derivation/s01_primitive_response_coupled_finite_provenance/s03_recurrent_pre_birth_measurement_and_steering/test_s02_c004__next_open_provenance_slot_snplus1_n_ge_1.py Source SHA-256: e03128bfa022ebd4939c778fee4cc19eda4711544922d965ae227457392b725f

- _state - FUNCTION, lines 16-31; role TEST.
- TestNextOpenProvenanceSlot - CLASS, lines 34-107; role TEST.

lean_core

Path: derivation/code/lean/core/src/CNNA/Derivation/S01_PrimitiveResponseCoupledFiniteProvenance/S03_RecurrentPreBirthMeasurementAndSteering/S02_C004_NextOpenProvenanceSlotSnplus1NGe1.lean Source SHA-256: e729d8ac3ebcf263049f660b8ea33bd9087d60fd1796d076a7c55659e922281a

- addressesAtDepth - DEF, lines 39-47; role SOURCE.
- mem_addressesAtDepth_of_length_eq - THEOREM, lines 48-63; role SOURCE.
- addressesUpTo - DEF, lines 64-69; role SOURCE.
- mem_addressesUpTo_of_length_le - THEOREM, lines 70-88; role SOURCE.
- AdmissibleOpenAddress - DEF, lines 89-95; role SOURCE.
- Unsaturated - DEF, lines 96-99; role SOURCE.
- Saturated - DEF, lines 100-107; role SOURCE.
- IsNextOpenAddress - DEF, lines 108-116; role SOURCE.
- NextOpenSlot - ABBREV, lines 117-123; role SOURCE.
- admissibleOpenAddressDecidable - INSTANCE, lines 124-133; role SOURCE.
- birthBeforeDecidable - INSTANCE, lines 134-145; role SOURCE.

- preferEarlierOpen - DEF, lines 146-157; role SOURCE.
- MinimalAmong - DEF, lines 158-165; role SOURCE.
- minimalAmong_cons - THEOREM, lines 166-208; role SOURCE.
- leastOpenFrom - DEF, lines 209-218; role SOURCE.
- leastOpenFrom_minimal - THEOREM, lines 219-235; role SOURCE.
- exists_of_unsaturated - THEOREM, lines 236-251; role SOURCE.
- child_nonroot - THEOREM, lines 252-256; role SOURCE.
- child_withinCutoff - THEOREM, lines 257-261; role SOURCE.
- child_notBorn - THEOREM, lines 262-266; role SOURCE.
- no_open_before - THEOREM, lines 267-273; role SOURCE.
- child_unique - THEOREM, lines 274-285; role SOURCE.
- unique - THEOREM, lines 286-292; role SOURCE.
- born_before_next - THEOREM, lines 293-311; role SOURCE.
- earlier_admissible_is_born - THEOREM, lines 312-327; role SOURCE.
- admissible_born_iff_before_next - THEOREM, lines 328-340; role SOURCE.
- snoc_eq_append_singleton - THEOREM, lines 341-351; role SOURCE.
- child_ne_nil - THEOREM, lines 352-359; role SOURCE.
- parentAddress - DEF, lines 360-364; role SOURCE.
- rank - DEF, lines 365-369; role SOURCE.
- child_eq_snoc - THEOREM, lines 370-375; role SOURCE.
- child_parent - THEOREM, lines 376-381; role SOURCE.
- child_finalSlot - THEOREM, lines 382-387; role SOURCE.
- eq_root_of_depth_eq_zero - THEOREM, lines 388-400; role SOURCE.
- parent_born - THEOREM, lines 401-429; role SOURCE.
- parent_rank_unique - THEOREM, lines 430-442; role SOURCE.
- unsaturated_not_saturated - THEOREM, lines 443-450; role SOURCE.
- no_next_of_saturated - THEOREM, lines 451-459; role SOURCE.

Open-provenance role: Open slot as state-relative incompleteness

C004 makes openness relative to a current born prefix: the next slot is absent from the present state but admissible in the fixed provenance grammar. This is the finite CNNA specialization of openness relative to an incomplete state description.

---

017 · C006 — Birth-local Schur/DtN primitive

Canonical node label: `017 · C006`
Current section path: `1.3.3`
Documentation tier: `D1`

Position In Derivation

C006 is introduced at the first use of block elimination, before any state-dependent matrix assembly. It is a generic exact partial operator used by M002, C007, and P001.

Mathematical Contract

For ordered blocks `K_BB`, `K_BI`, `K_IB`, `K_II` with nonempty boundary, an interior solve `X` satisfies `K_II X = K_IB`. The exact domain is existence of exactly one encoded solve. On that domain the response value is `K_BB - K_BI X`.

Introduction Reason

The cut selector and the network realization must not be conflated with the algebraic elimination rule. C006 isolates the reusable mathematical primitive and makes partiality explicit.

Explicit Construction

Python uses exact `Fraction` matrices and Gauss-Jordan elimination without tolerance. Lean defines positive-denominator raw fractions, cross-multiplication value equality, matrix multiplication/subtraction, solve and response predicates, and the zero-interior witness.

Invariants

Boundary coordinates precede interior coordinates. Matrix dimensions are type- or constructor-checked. No transpose, symmetrization, pseudoinverse, regularization, threshold, or condition-number rule is admitted.

Canonicity Or Uniqueness

`response_exists_of_admissible` and `response_unique_of_admissible` prove existence and value-level uniqueness. `response_of_sameValue` proves independence of raw fraction representation. Structural numerator/denominator equality is intentionally not required.

Boundary Cases

Boundary size must be positive. Interior size may be zero; then the unique empty solve exists and the response is `K_BB`. A singular nonempty `K_II` is outside the domain rather than regularized numerically.

Python Lean Cross Layer

Python returns normalized fractions. Lean's core relation compares raw positive-denominator encodings by cross multiplication. P001 later proves the exact semantic bridge to ordinary rational matrix arithmetic; that later theorem is not silently imported into C006.

Countercheck

Making the operator total with a pseudoinverse would change the model. Comparing raw encodings structurally would make the response depend on normalization. Permuting coordinates inside C006 would violate ownership of the M001 order.

Result

C006 is a verified exact, partial, representative-independent Schur/DtN response primitive.

Downstream Handoff

- `E019` to C007 supplies elimination;
- `E135` to P001 supplies the native exact interface;
- later general-cut reuse remains open.

Code Anchors

python

Path: derivation/code/python/src/cnna/derivation/s01_primitive_response_coupled_finite_provenance/s03_recurrent_pre_birth_measurement_and_steering/s03_c006__birth_local_schur_dtn_primitive.py Source SHA-256: 7a16beb12e9cf5e892dbef27b783f4d0a13684aad72b95b6b5e8ef73c75eb64a

- InteriorNotAdmissibleError - CLASS, lines 32-33; role SOURCE.
- _validate_matrix - FUNCTION, lines 36-47; role SOURCE.
- matrix_subtract - FUNCTION, lines 50-61; role SOURCE.
- matrix_multiply - FUNCTION, lines 64-94; role SOURCE.
- OrderedSchurBlocks - CLASS, lines 98-122; role SOURCE.
- is_interior_solve - FUNCTION, lines 125-138; role SOURCE.
- _unique_interior_solve - FUNCTION, lines 141-179; role SOURCE.
- interior_is_admissible - FUNCTION, lines 182-190; role SOURCE.
- schur_dtn_response_from_solve - FUNCTION, lines 193-208; role SOURCE.
- schur_dtn_response - FUNCTION, lines 211-216; role SOURCE.

python_test

Path: derivation/code/python/tests/derivation/s01_primitive_response_coupled_finite_provenance/s03_recurrent_pre_birth_measurement_and_steering/test_s03_c006__birth_local_schur_dtn_primitive.py Source SHA-256: 7a8ff76ba2f23fc500aee3c73e81a1f2e8c3adeb701389493acd7b49b2468d94

- _raw_of_fraction - FUNCTION, lines 18-21; role TEST.
- _raw_value - FUNCTION, lines 24-26; role TEST.
- _raw_add - FUNCTION, lines 29-32; role TEST.
- _raw_mul - FUNCTION, lines 35-38; role TEST.
- _raw_sub - FUNCTION, lines 41-44; role TEST.
- TestBirthLocalSchurDtnPrimitive - CLASS, lines 47-204; role TEST.

lean_core

Path: derivation/code/lean/core/src/CNNA/Derivation/S01_PrimitiveResponseCoupledFiniteProvenance/S03_RecurrentPreBirthMeasurementAndSteering/S03_C006_BirthLocalSchurDtnPrimitive.lean Source SHA-256: da65a94576a7272e5bd0caae1a5f2b690fe8d57faac99283b68f0a9ae92b5fad

- RatMatrix - ABBREV, lines 41-45; role SOURCE.
- ExactFraction - STRUCTURE, lines 46-53; role SOURCE.
- ofRat - DEF, lines 54-59; role SOURCE.
- zero - DEF, lines 60-65; role SOURCE.
- add - DEF, lines 66-71; role SOURCE.
- mul - DEF, lines 72-77; role SOURCE.
- sub - DEF, lines 78-84; role SOURCE.
- SameValue - DEF, lines 85-89; role SOURCE.
- Represents - DEF, lines 90-93; role SOURCE.
- sameValue_refl - THEOREM, lines 94-97; role SOURCE.
- sameValue_symm - THEOREM, lines 98-102; role SOURCE.
- sameValue_trans - THEOREM, lines 103-123; role SOURCE.
- sameValue_equivalence - THEOREM, lines 124-133; role SOURCE.
- congrArgTwoInt - THEOREM, lines 134-144; role SOURCE.
- add_respects_sameValue - THEOREM, lines 145-196; role SOURCE.
- mul_respects_sameValue - THEOREM, lines 197-220; role SOURCE.
- sub_respects_sameValue - THEOREM, lines 221-272; role SOURCE.
- ofRat_represents - THEOREM, lines 273-278; role SOURCE.

- add_of_representatives - THEOREM, lines 279-284; role SOURCE.
- mul_of_representatives - THEOREM, lines 285-290; role SOURCE.
- sub_of_representatives - THEOREM, lines 291-296; role SOURCE.
- foldl_add_respects_sameValue - THEOREM, lines 297-318; role SOURCE.
- ExactFractionMatrix - ABBREV, lines 319-322; role SOURCE.
- MatrixSameValue - DEF, lines 323-328; role SOURCE.
- MatrixRepresents - DEF, lines 329-333; role SOURCE.
- matrixSameValue_refl - THEOREM, lines 334-339; role SOURCE.
- matrixSameValue_symm - THEOREM, lines 340-346; role SOURCE.
- matrixSameValue_trans - THEOREM, lines 347-354; role SOURCE.
- matrixSameValue_equivalence - THEOREM, lines 355-364; role SOURCE.
- rawMatrixMul - DEF, lines 365-376; role SOURCE.
- matrixMul - DEF, lines 377-384; role SOURCE.
- rawMatrixMul_respects_sameValue - THEOREM, lines 385-399; role SOURCE.
- rawMatrixMul_matches_canonicalEncoding - THEOREM, lines 400-413; role SOURCE.
- rawMatrixSub - DEF, lines 414-418; role SOURCE.
- matrixSub - DEF, lines 419-424; role SOURCE.
- rawMatrixSub_respects_sameValue - THEOREM, lines 425-434; role SOURCE.
- OrderedSchurBlocks - STRUCTURE, lines 435-444; role SOURCE.
- IsInteriorSolve - DEF, lines 445-451; role SOURCE.
- IsInteriorAdmissible - DEF, lines 452-459; role SOURCE.
- responseFromSolve - DEF, lines 460-467; role SOURCE.
- IsSchurDtnResponse - DEF, lines 468-475; role SOURCE.
- response_of_solve - THEOREM, lines 476-483; role SOURCE.
- response_exists_of_admissible - THEOREM, lines 484-493; role SOURCE.
- response_unique_of_admissible - THEOREM, lines 494-513; role SOURCE.
- response_of_sameValue - THEOREM, lines 514-522; role SOURCE.
- emptyInteriorSolve - DEF, lines 523-526; role SOURCE.
- zeroInterior_solve - THEOREM, lines 527-534; role SOURCE.
- zeroInterior_admissible - THEOREM, lines 535-545; role SOURCE.

Open-provenance role: Exact complement elimination

C006 realizes one specific open-provenance reduction: exact directed Schur/DtN elimination by a unique interior solve. It is not a partial trace, marginalization, or quantum instrument.

---

018 · M001 — Canonical birth-local measurement cut C_n(s_{n+1})

Canonical node label: 018 · M001
Current section path: 1.3.4
Documentation tier: D2

Position In Derivation

M001 follows C005 and C004. It owns the state- and slot-dependent ordered cut and owns proof gate P003 internally.

Formal Statement

For X_n and next slot (p,rho,c), the boundary is the canonical-carrier subsequence consisting of the root-to-parent prefix chain of p together with born same-parent siblings of ranks < rho. The interior is the complementary canonical-carrier subsequence. The unborn child c is in neither list.

Hypotheses

A valid C005 state, a valid C004 `NextOpenSlot`, the inherited C003 grammar, and the inherited C018 order. No response, conductance value, geometry, or downstream steering hypothesis is used.

Introduction Reason

The C006 operator requires explicit ordered boundary/interior coordinates. Birth locality must therefore be derived from provenance before numerical block entries are assembled.

Proof Strategy

Construct one canonical carrier list, define a decidable birth-local port predicate, and obtain boundary/interior by complementary filtering. Prove that every selected causal predecessor and older sibling is born, then derive partition properties from the common carrier filter.

Lemma Chain

`prefixChainAux_mem_prefix -> causalPredecessorPort_born`; `earlier_admissible_is_born` from C004 -> `olderSiblingPort_born`; these yield `birthLocalPort_born`. Filtering yields `boundary` and `interior`; `canonicalCut_isCanonical` and `unique` close canonicity. `boundary_node_born`, `interior_node_born`, `boundary_interior_disjoint`, `carrier_covered`, and the child-exclusion theorems close P003.

Formal Realization

Python constructs immutable filtered tuples and performs runtime partition assertions. Lean defines the same predicates over the canonical carrier and proves the complete owner certificate in `S04_M001_CanonicalBirthLocalMeasurementCutCnSnplus1.lean`.

Counterexamples Or Necessity Checks

Including the unborn child would turn a pre-birth measurement into a postulated-node measurement. Sorting the two blocks anew would add a second ordering convention. Using weights to select ports would make locality response-dependent. Omitting complementary coverage would leave the C007 state matrix under-specified.

Axiom Profile

The module is in the mathlib-free core and contains no project-local `axiom`, `sorry`, `admit`, `opaque`, `unsafe`, or `partial` declaration. The current 26-job core build evidence certifies the listed source.

Result

M001 and its P003 owner gate are closed: the cut is unique, born-only, disjoint, exhaustive, order-preserving by filtering, and strictly pre-birth.

Remaining Limits

M001 supplies no numerical block entries and proves no interior admissibility. P007 remains a later general-cut generalization and does not keep M001 yellow.

Downstream Handoff

- E018 to M002 supplies cut dimensions;
- E020 to C007 supplies coordinate order;
- E146 records P003 certification of this owner.

Code Line Register

python

Path: derivation/code/python/src/cnna/derivation/s01_primitive_response_coupled_finite_provenance/s03_recurrent_pre_birth_measurement_and_steering/s04_m001__canonical_birth_local_measurement_cut_cn_snplus1.py Source SHA-256: 5b840dcb3b47316e54241816804710479d193db236765717d10fb53a970b120a

- BirthLocalMeasurementCut - CLASS, lines 34-53; role SOURCE.
- causal_predecessor_ports - FUNCTION, lines 56-61; role SOURCE.
- older_sibling_ports - FUNCTION, lines 64-68; role SOURCE.
- is_birth_local_port - FUNCTION, lines 71-75; role SOURCE.
- is_canonical_birth_local_measurement_cut - FUNCTION, lines 78-95; role SOURCE.
- canonical_birth_local_measurement_cut - FUNCTION, lines 98-147; role SOURCE.

python_test

Path: derivation/code/python/tests/derivation/s01_primitive_response_coupled_finite_provenance/s03_recurrent_pre_birth_measurement_and_steering/test_s04_m001__canonical_birth_local_measurement_cut_cn_snplus1.py Source SHA-256: 537c5f7f7d66d77c5c906f7dd59e05d640dbe7a7e0b84ee7ce141d08536766b7

- _state - FUNCTION, lines 23-36; role TEST.
- TestCanonicalBirthLocalMeasurementCut - CLASS, lines 39-96; role TEST.

lean_core

Path: derivation/code/lean/core/src/CNNA/Derivation/S01_PrimitiveResponseCoupledFiniteProvenance/S03_RecurrentPreBirthMeasurementAndSteering/S04_M001_CanonicalBirthLocalMeasurementCutCnSnplus1.lean Source SHA-256: f8c7821fe45245ba62934d55daa5ab57323895df9ac11a55f14db3e491b5f3e4

- canonicalCarrier - DEF, lines 40-45; role SOURCE.
- prefixChainAux - DEF, lines 46-54; role SOURCE.
- prefixChainAux_mem_prefix - THEOREM, lines 55-80; role SOURCE.
- causalPredecessorPorts - DEF, lines 81-86; role SOURCE.
- olderSiblingPorts - DEF, lines 87-93; role SOURCE.
- BirthLocalPort - DEF, lines 94-100; role SOURCE.
- causalPredecessorPort_born - THEOREM, lines 101-141; role SOURCE.
- olderSiblingPort_born - THEOREM, lines 142-181; role SOURCE.
- birthLocalPort_born - THEOREM, lines 182-190; role SOURCE.
- birthLocalPortDecidable - INSTANCE, lines 191-197; role SOURCE.
- portFlag - DEF, lines 198-202; role SOURCE.
- boundary - DEF, lines 203-207; role SOURCE.
- interior - DEF, lines 208-212; role SOURCE.
- BirthLocalMeasurementCut - STRUCTURE, lines 213-217; role SOURCE.
- canonicalCut - DEF, lines 218-223; role SOURCE.
- IsCanonicalCut - DEF, lines 224-228; role SOURCE.
- canonicalCut_isCanonical - THEOREM, lines 229-233; role SOURCE.
- unique - THEOREM, lines 234-252; role SOURCE.
- carrier_mem_implies_born - THEOREM, lines 253-261; role SOURCE.
- born_implies_carrier_mem - THEOREM, lines 262-272; role SOURCE.

- `birthLocalPort_mem_boundary` - THEOREM, lines 273-285; role `SOURCE`.
- `boundary_mem_iff_birthLocalPort` - THEOREM, lines 286-299; role `SOURCE`.
- `boundary_node_born` - THEOREM, lines 300-306; role `SOURCE`.
- `interior_node_born` - THEOREM, lines 307-313; role `SOURCE`.
- `boundary_interior_disjoint` - THEOREM, lines 314-327; role `SOURCE`.
- `carrier_covered` - THEOREM, lines 328-349; role `SOURCE`.
- `child_not_in_carrier` - THEOREM, lines 350-363; role `SOURCE`.
- `child_not_in_boundary` - THEOREM, lines 364-369; role `SOURCE`.
- `child_not_in_interior` - THEOREM, lines 370-378; role `SOURCE`.

Open-provenance role: Cut-relative system and environment

M001 selects retained boundary ports and an eliminated interior. The interior is the environment relative to this cut, not a second primitive substance; changing the cut changes the system/environment roles.

---

019 · P003 — Birth-cut canonicity and partition well-formedness

Canonical node label: `019 · P003`
Current section path: `1.3.4.1`
Documentation tier: `D2`

Position In Derivation

P003 is the explicit proof-certification expansion of M001. It introduces no new cut and no parallel derivation branch.

Formal Statement

For every valid C005 state and C004 successor, the M001 boundary and interior are born-only, disjoint, exhaustive on the current born carrier, preserve inherited carrier order, exclude the unborn child, and are uniquely determined by the canonical selector.

Hypotheses

Exactly the hypotheses already carried by M001: C005 state invariants and the C004 next-slot certificate. C018 order is inherited transitively. No P002 dependency is required by the proof term.

Introduction Reason

M001's construction is scientifically usable by C007 only after its partition and pre-birth properties are independently visible as one falsifiable proof gate.

Proof Strategy

Reuse the owner definitions. Bornness is proved for the port predicate; complementary filters yield disjointness and coverage; filter order is inherited definitionally; canonical-cut equality yields uniqueness; C004 non-bornness yields child exclusion.

Lemma Chain

`canonicalCarrier`, `boundary`, `interior`, `IsCanonicalCut`, `canonicalCut_isCanonical`, `unique`, `boundary_node_born`, `interior_node_born`, `boundary_interior_disjoint`, `carrier_covered`, `child_not_in_boundary`, and `child_not_in_interior`.

Formal Realization

The complete certificate is intentionally internal to the M001 Lean module. P003's code register therefore contains supporting-evidence anchors into that owner module and its independent Python construction/test, rather than a duplicate P003 source file.

Counterexamples Or Necessity Checks

A boundary containing an un-born address violates the C005 domain. Non-disjoint blocks duplicate matrix coordinates. Non-exhaustive blocks omit live conductances. Reordering a filter result changes the matrix basis. Including the next child imports nonexistent state. Without `IsCanonicalCut`, multiple witnesses could satisfy a weaker partition predicate.

Axiom Profile

The supporting theorems are in the mathlib-free core and contain no project-local axiom or admitted proof. No new proof source is introduced in D7.

Result

P003 is closed by exact owner-internal theorems. Its certification edge to M001 is active and verified.

Remaining Limits

General-cut proof P007 remains future work.

Downstream Handoff

`E146` certifies M001. `E165` later allows P007 to reuse this birth-local special case but remains blocked.

Code Line Register

python

Path: `derivation/code/python/src/cnna/derivation/s01_primitive_response_coupled_finite_provenance/s03_recurrent_pre_birth_measurement_and_steering/s04_m001_canonical_birth_local_measurement_cut_cn_snplus1.py` Source SHA-256: `5b840dcb3b47316e54241816804710479d193db236765717d10fb53a970b120a`

- `is_canonical_birth_local_measurement_cut` - FUNCTION, lines 78-95; role `SUPPORTING_EVIDENCE`.
- `canonical_birth_local_measurement_cut` - FUNCTION, lines 98-147; role `SUPPORTING_EVIDENCE`.

python_test

Path: derivation/code/python/tests/derivation/s01_primitive_response_coupled_finite_provenance/s03_recurrent_pre_birth_measurement_and_steering/test_s04_m001__canonical_birth_local_measurement_cut_cn_snplus1.py Source SHA-256: 537c5f7f7d66d77c5c906f7dd59e05d640dbe7a7e0b84ee7ce141d08536766b7

- TestCanonicalBirthLocalMeasurementCut - CLASS, lines 39-96; role SUPPORTING_EVIDENCE.

lean_core

Path: derivation/code/lean/core/src/CNNA/Derivation/S01_PrimitiveResponseCoupledFiniteProvenance/S03_RecurrentPreBirthMeasurementAndSteering/S04_M001_CanonicalBirthLocalMeasurementCutCnSnplus1.lean Source SHA-256: f8c7821fe45245ba62934d55daa5ab57323895df9ac11a55f14db3e491b5f3e4

- canonicalCarrier - DEF, lines 40-45; role SUPPORTING_EVIDENCE.
- boundary - DEF, lines 203-207; role SUPPORTING_EVIDENCE.
- interior - DEF, lines 208-212; role SUPPORTING_EVIDENCE.
- IsCanonicalCut - DEF, lines 224-228; role SUPPORTING_EVIDENCE.
- canonicalCut_isCanonical - THEOREM, lines 229-233; role SUPPORTING_EVIDENCE.
- unique - THEOREM, lines 234-252; role SUPPORTING_EVIDENCE.
- carrier_mem_implies_born - THEOREM, lines 253-261; role SUPPORTING_EVIDENCE.
- born_implies_carrier_mem - THEOREM, lines 262-272; role SUPPORTING_EVIDENCE.
- boundary_node_born - THEOREM, lines 300-306; role SUPPORTING_EVIDENCE.
- interior_node_born - THEOREM, lines 307-313; role SUPPORTING_EVIDENCE.
- boundary_interior_disjoint - THEOREM, lines 314-327; role SUPPORTING_EVIDENCE.
- carrier_covered - THEOREM, lines 328-349; role SUPPORTING_EVIDENCE.
- child_not_in_boundary - THEOREM, lines 364-369; role SUPPORTING_EVIDENCE.
- child_not_in_interior - THEOREM, lines 370-378; role SUPPORTING_EVIDENCE.

---

020 · M002 — Birth-cut interior-domain theorem

Canonical node label: 020 · M002
Current section path: 1.3.5
Documentation tier: D2

Position In Derivation

M002 connects the M001 cut to the partial C006 primitive before C007 supplies state-dependent numerical entries.

Formal Statement

BirthCutBlocks next has exactly |B_n| boundary and |I_n| interior coordinates. InExactDomain next blocks is definitionally C006 IsInteriorAdmissible blocks. On this domain a C006 response exists and is unique in exact fraction value.

Hypotheses

A valid M001 next-slot cut and explicitly supplied C006 blocks of the cut-induced dimensions. Nonempty-interior admissibility is a hypothesis, not a consequence of dimensions alone.

Introduction Reason

The original obligation allowed either a universal invertibility theorem or an exact domain statement. M001 contains no numerical entries, so only the exact-domain branch is derivable at this point.

Proof Strategy

Derive nonempty boundary from the root causal port. Encode dimension agreement in the type of `BirthCutBlocks`. Reuse C006 admissibility for response existence/uniqueness. Prove zero-interior membership directly from C006.

Lemma Chain

`root_is_birthLocalPort -> canonicalBoundary_nonempty`; `mkBirthCutBlocks`; `InExactDomain`; `inExactDomain_iff_c006_admissible`; `exactDomain_response_exists`; `exactDomain_response_unique`; `zeroInterior_inExactDomain`.

Formal Realization

Python validates dimensions, calls the exact C006 domain predicate, and constructs an identity-versus-zero same-cut contrast witness. Lean encodes dimensions in types and proves the exact handoff without duplicating the executable contrast.

Counterexamples Or Necessity Checks

For any nonempty interior, identity `K_II` and zero `K_II` have the same dimensions but different admissibility. Therefore dimensions, provenance locality, boundary nonemptiness, positivity of stored conductances, or notation alone cannot justify a total Schur complement.

Axiom Profile

The module belongs to the mathlib-free core and introduces no project-local admitted theorem. Its source is unchanged by D7.

Result

M002 closes the domain obligation exactly: zero interior is unconditional; every other case requires exact unique solvability.

Remaining Limits

M002 does not prove that every reachable C007 realization lies in the domain. Generic first-hit reachability and linear well-posedness are treated later in P001; exact canonical-cut instantiation remains a separate proof target there.

Downstream Handoff

- `E021` certifies C007's domain;
- `E136` supplies the exact interior-domain interface to P001.

Code Line Register

python

Path: derivation/code/python/src/cnna/derivation/s01_primitive_response_coupled_finite_provenance/s03_recurrent_pre_birth_measurement_and_steering/s05_m002__birth_cut_interior_domain_theorem.py Source SHA-256: bf0b213349892ba660a958398cf6d808dca41f1f05bf0c7ac7a832c31bec041d

- _zero_matrix - FUNCTION, lines 37-38; role SOURCE.
- _identity_matrix - FUNCTION, lines 41-45; role SOURCE.
- validate_birth_cut_block_dimensions - FUNCTION, lines 48-60; role SOURCE.
- birth_cut_interior_is_admissible - FUNCTION, lines 63-77; role SOURCE.
- DomainContrastWitness - CLASS, lines 81-94; role SOURCE.
- domain_contrast_witness - FUNCTION, lines 97-131; role SOURCE.

python_test

Path: derivation/code/python/tests/derivation/s01_primitive_response_coupled_finite_provenance/s03_recurrent_pre_birth_measurement_and_steering/test_s05_m002__birth_cut_interior_domain_theorem.py Source SHA-256: 5c58c474588d868b35de0066273e40d0edf26c7aa3fdfdb5f5a0ecfddbaedc26

- _state - FUNCTION, lines 22-33; role TEST.
- _zero - FUNCTION, lines 36-37; role TEST.
- TestBirthCutInteriorDomainTheorem - CLASS, lines 40-91; role TEST.

lean_core

Path: derivation/code/lean/core/src/CNNA/Derivation/S01_PrimitiveResponseCoupledFiniteProvenance/S03_RecurrentPreBirthMeasurementAndSteering/S05_M002_BirthCutInteriorDomainTheorem.lean Source SHA-256: 171d52d05001dcffc60fd1a325e1b5ce4c0025bbce75a57cc172e10efdbae669

- root_is_birthLocalPort - THEOREM, lines 36-46; role SOURCE.
- canonicalBoundary_nonempty - THEOREM, lines 47-54; role SOURCE.
- BirthCutBlocks - ABBREV, lines 55-60; role SOURCE.
- mkBirthCutBlocks - DEF, lines 61-74; role SOURCE.
- InExactDomain - DEF, lines 75-80; role SOURCE.
- inExactDomain_iff_c006_admissible - THEOREM, lines 81-86; role SOURCE.
- exactDomain_response_exists - THEOREM, lines 87-95; role SOURCE.
- exactDomain_response_unique - THEOREM, lines 96-106; role SOURCE.
- admissible_of_interiorSize_eq_zero - THEOREM, lines 107-117; role SOURCE.
- zeroInterior_inExactDomain - THEOREM, lines 118-128; role SOURCE.

---

021 · C007 — Inter-birth directed response R_n(s_{n+1})

Canonical node label: 021 · C007
Current section path: 1.3.6
Documentation tier: D1

Position In Derivation

C007 is the first state-dependent response node. It combines C005, C004/M001, M002, and C006 after all structural and domain ownership has been fixed.

Mathematical Contract

Order the entire born carrier as M001 boundary followed by interior. From every stored directed conductance define the source/out-degree matrix `K[u,u]=sum_v c(u,v)` and `K[u,v]=-c(u,v)` for `u != v`. On M002 domain membership, `R_n(s_{n+1})` is the unique C006 Schur/DtN response.

Introduction Reason

The later steering law needs a measured property of the existing network before the next birth. This response must use the current directed weights without symmetrization or a response-independent bias.

Explicit Construction

Python assembles an exact `Fraction` matrix, slices the four blocks in M001 order, validates M002, and evaluates C006. Lean constructs exact-fraction outgoing/ordered-pair sums, defines raw blocks, and requires a `StateDirectedBlockRealization` proving that canonical rational inputs represent those exact values.

Invariants

Rows are sources and columns targets. Off-diagonal entries are negative ordered-pair conductances; the diagonal is total outgoing conductance. All current born vertices occur exactly once. The unborn child occurs nowhere. No external port, grounded load, geometry, averaging, transpose, or regularization is added.

Canonicity Or Uniqueness

M001 fixes coordinates, the directed conductance list fixes exact entries, M002 fixes the domain, and C006 proves response existence and value-level uniqueness. `response_of_sameValue` makes output-representative independence explicit.

Boundary Cases

Zero interior reduces to the boundary block. Singular nonempty interior is outside the domain. Saturated states have no C004 slot and therefore no C007 pre-birth response indexed by `s_{n+1}`.

Python Lean Cross Layer

Python normalizes fractions; Lean separates exact values from canonical `Rat` input representatives through `MatrixRepresents`. The P001 semantic bridge later identifies these with ordinary rational matrix arithmetic. Both layers preserve the same M001 coordinate order and source/out-degree sign convention.

Countercheck

Symmetrizing would erase directional data. Reversing row/column ownership would transpose the model. Adding the unborn child would make the measurement anticipatory. A tolerance-based inverse would alter M002. Omitting additional born-born edges would fail to represent the live C005 state.

Result

C007 is a verified exact directed pre-birth response on the stated M002 domain.

Downstream Handoff

- E024 to M003 is ACTIVE_VERIFIED and supplies the exact response consumed by the canonical steering functional;
- E035, E037, and E038 feed null/robustness controls;
- E137 supplies the directed block operator to P001.

Code Anchors

python

Path: derivation/code/python/src/cnna/derivation/s01_primitive_response_coupled_finite_provenance/s03_recurrent_pre_birth_measurement_and_steering/s06_c007__inter_birth_directed_response_rn_snplus1.py Source SHA-256: 9975a54c59b3385a416b6bb2f3df08135dd1f8caa0433ba809c65b1ba8462e0c

- _zero_matrix - FUNCTION, lines 52-53; role SOURCE.
- _freeze_block - FUNCTION, lines 56-66; role SOURCE.
- StateDirectedSchurRealization - CLASS, lines 70-96; role SOURCE.
- InterBirthDirectedResponse - CLASS, lines 100-122; role SOURCE.
- state_directed_schur_realization - FUNCTION, lines 125-166; role SOURCE.
- inter_birth_directed_response - FUNCTION, lines 169-190; role SOURCE.

python_test

Path: derivation/code/python/tests/derivation/s01_primitive_response_coupled_finite_provenance/s03_recurrent_pre_birth_measurement_and_steering/test_s06_c007__inter_birth_directed_response_rn_snplus1.py Source SHA-256: ddf93cee71f4da4c82ea4b17776e55ba63fb83b35c3ce9c8b3be68c5434f5426

- _bootstrap_state - FUNCTION, lines 31-37; role TEST.
- _state - FUNCTION, lines 40-55; role TEST.
- TestInterBirthDirectedResponse - CLASS, lines 58-134; role TEST.

lean_core

Path: derivation/code/lean/core/src/CNNA/Derivation/S01_PrimitiveResponseCoupledFiniteProvenance/S03_RecurrentPreBirthMeasurementAndSteering/S06_C007_InterBirthDirectedResponseRnSnplus1.lean Source SHA-256: 18ef5818a50460d6866847274cd196f581f7b7d15c8da940287e43b95a49eaaf

- outgoingSum - DEF, lines 40-52; role SOURCE.
- orderedPairSum - DEF, lines 53-65; role SOURCE.
- directedMatrixEntry - DEF, lines 66-74; role SOURCE.
- directedMatrixEntry_self - THEOREM, lines 75-82; role SOURCE.
- directedMatrixEntry_of_ne - THEOREM, lines 83-93; role SOURCE.
- boundaryAddress - DEF, lines 94-98; role SOURCE.
- interiorAddress - DEF, lines 99-103; role SOURCE.
- rawKBB - DEF, lines 104-110; role SOURCE.
- rawKBI - DEF, lines 111-117; role SOURCE.
- rawKIB - DEF, lines 118-124; role SOURCE.
- rawKII - DEF, lines 125-132; role SOURCE.
- RealizesStateDirectedBlocks - DEF, lines 133-142; role SOURCE.
- StateDirectedBlockRealization - STRUCTURE, lines 143-148; role SOURCE.
- IsInterBirthDirectedResponse - DEF, lines 149-155; role SOURCE.
- InResponseDomain - DEF, lines 156-162; role SOURCE.
- inResponseDomain_iff_m002 - THEOREM, lines 163-170; role SOURCE.
- response_exists - THEOREM, lines 171-182; role SOURCE.
- response_unique - THEOREM, lines 183-196; role SOURCE.

- response_of_sameValue - THEOREM, lines 197-208; role SOURCE.
- unborn_child_not_in_boundary - THEOREM, lines 209-213; role SOURCE.
- unborn_child_not_in_interior - THEOREM, lines 214-220; role SOURCE.

Open-provenance role: Effective response after complement elimination

C007 is the retained-port response of the pre-birth live state. It is the finite directed-linear effective dynamics associated with the M001 cut, before scalar steering or event creation.

---

022 · O001 — IST response-independent directed-bias obstruction

Canonical node label: 022 · O001
Semantic ID: O001
Current section path: 1.3.7
Documentation tier: D2

Position In Derivation

O001 is introduced after the exact pre-birth response C007 and before the active steering and birth-law handoff. Its role is to prevent the retained legacy implementation from reintroducing numerical channels that are independent of the response-derived scalar.

Formal Statement

Let

```
chi = (rank, forward, backward, node_load_scalar, nonlinear_mode,
       backreaction_scale, additive_baseline, geometric_attenuation)
```

be the explicit Boolean presence record. A candidate (state, slot, response, steering, chi) is admissible exactly when chi equals the all-false record. The accepted output preserves state, slot, response, and steering exactly and contains no bias record.

Hypotheses

The obstruction is source-bound to the retained legacy growth path and to the eight represented mechanism classes. It assumes neither that these names exhaust every imaginable response-independent mechanism nor that no alternative mathematical growth law could contain additional terms.

Introduction Reason

Earlier auditing covered only rank and forward/backward asymmetry. A later completeness review of the same executable path exposed five further classes: node-load scalars, nonlinear modes, fixed backreaction scales, additive baselines, and geometric attenuation. The admission gate must therefore cover all eight classes before M004 can be regarded as bias-free.

Proof Strategy

Python supplies an executable AST audit and a runtime admission guard. Lean supplies a generic eight-field record, exact equality with the all-false witness, field-preservation theorems, and one contradiction theorem per active field. The two layers agree on field names, the complete witness, and the four-field admitted output.

Lemma Chain

```
IndependentDirectedBiasPresence
  -> noIndependentDirectedBias
  -> IsRemoved / IsAdmissible
  -> acceptBiasFree
  -> accepted_preserves_state/slot/response/steering

one active field
  -> *_blocks_acceptance

legacyResponseIndependentChannels
  -> legacy_channels_not_removed
  -> legacy_candidate_not_admissible
```

Formal Realization

The Python AST traversal records executable names and assignments in six bound growth functions. Comments and docstrings are not AST symbol uses. Synthetic detection of additive baselines and geometric attenuation is restricted to executable arithmetic patterns. The Lean module contains no Mathlib import and was included in the user-reported prefix-free 26-job Core build.

Counterexamples Or Necessity Checks

- Activating any one of the eight fields must reject the candidate.
- The incomplete three-channel witness must still reject, but it is not treated as complete.
- The full eight-channel witness must reject.
- A source containing the forbidden words only in comments or docstrings must not trigger the AST audit.
- Acceptance must preserve object identity for all four admitted Python fields and must erase the bias record.

Axiom Profile

O001 belongs to the Mathlib-free Core. The current evidence is a successful Lean typecheck-/root build plus a static policy scan. No stronger claim about a separate `#print axioms` transcript for O001 is made.

Result

The represented legacy mechanisms are excluded from the active M004 dependency tuple. O001 is a closed and falsifiable implementation obstruction.

Remaining Limits

This is not a universal mathematical no-go theorem and does not prove M003 positivity. The AST audit is exact for the bound retained source and the declared mechanism classes only.

Downstream Handoff

O001 hands M004 exactly `(X_n, s_{n+1}, R_n, Sigma_b)` and certifies that the canonical candidate uses the all-false presence record.

Code Line Register

python / SOURCE: s07_o001__ist_response_independent_directed_bias_obstruction.py

Path: derivation/code/python/src/cnna/derivation/s01_primitive_response_coupled_finite_provenance/s03_recurrent_pre_birth_measurement_and_steering/s07_o001__ist_response_independent_directed_bias_obstruction.py
SHA-256: f7b9e6366d2f3b2bb9190d24625ca3afd7647e6f83b4f999905597486527e04b

Symbol	Kind	Lines
IndependentDirectedBiasPresence	CLASS	99-117
CandidateGrowthLawInputs	CLASS	157-164
AdmittedGrowthLawInputs	CLASS	168-174
ResponseIndependentBiasError	CLASS	177-178
LegacyBiasFinding	CLASS	182-186
LegacyBiasAudit	CLASS	190-207
admit_growth_law_inputs	FUNCTION	210-229
_function_symbol_references	FUNCTION	232-241
_target_names	FUNCTION	244-249
_contains_numeric_one	FUNCTION	252-258
_contains_addition	FUNCTION	261-262
_synthetic_findings	FUNCTION	265-300
audit_legacy_response_independent_bias	FUNCTION	303-348
audit_legacy_response_independent_bias_file	FUNCTION	351-353

python_test / TEST: test_s07_o001__ist_response_independent_directed_bias_obstruction.py

Path: derivation/code/python/tests/derivation/s01_primitive_response_coupled_finite_provenance/s03_recurrent_pre_birth_measurement_and_steering/test_s07_o001__ist_response_independent_directed_bias_obstruction.py
SHA-256: 838e65efd3035c9a02cb326a7e6734403873f3b5caf874b833a6cf7a9dba7d20

Symbol	Kind	Lines
_presence_only	FUNCTION	18-20
TestIstResponseIndependentDirectedBiasObstruction	CLASS	23-129

lean_core / SOURCE: S07_O001_IstResponseIndependentDirectedBiasObstruction.lean

Path: derivation/code/lean/core/src/CNNA/Derivation/S01_PrimitiveResponseCoupledFiniteProvenance/S03_RecurrentPreBirthMeasurementAndSteering/S07_O001_IstResponseIndependentDirectedBiasObstruction.lean
SHA-256: 94a45ef69de61a5339b6f288295122ff6e36a386b67ef225a64b5dc6ea2d66a8

Symbol	Kind	Lines
IndependentDirectedBiasPresence	STRUCTURE	30-40
noIndependentDirectedBias	DEF	41-51
legacyRankForwardBackwardBias	DEF	52-63
legacyResponseIndependentChannels	DEF	64-74

Symbol	Kind	Lines
IsRemoved	DEF	75-78
CandidateGrowthLawInputs	STRUCTURE	79-88
BiasFreeGrowthLawInputs	STRUCTURE	89-97
IsAdmissible	DEF	98-103
acceptBiasFree	DEF	104-114
accepted_preserves_state	THEOREM	115-121
accepted_preserves_slot	THEOREM	122-128
accepted_preserves_response	THEOREM	129-135
accepted_preserves_steering	THEOREM	136-141
true_ne_false	THEOREM	142-146
rank_bias_blocks_acceptance	THEOREM	147-156
forward_bias_blocks_acceptance	THEOREM	157-166
backward_bias_blocks_acceptance	THEOREM	167-176
node_load_scalar_blocks_acceptance	THEOREM	177-186
nonlinear_mode_blocks_acceptance	THEOREM	187-196
backreaction_scale_blocks_acceptance	THEOREM	197-206
additive_baseline_blocks_acceptance	THEOREM	207-216
geometric_attenuation_blocks_acceptance	THEOREM	217-226
legacy_channels_not_removed	THEOREM	227-233
legacy_candidate_not_admissible	THEOREM	234-248

---

023 · C015 — Active linear steering convention phi(x)=x

Canonical node label: `023 · C015`
Semantic ID: `C015`
Current section path: `1.3.8`
Documentation tier: `D0`

Position In Derivation

C015 is placed after the obstruction gate and before M003. It fixes how an already selected exact response scalar is transformed on the active path.

Definition Or Statement

For every scalar type S,

```
phi : S -> S
phi(x) = x
```

Introduction Reason

M003 needs one explicit active-path transform, while robustness controls must remain separate and falsifiable. The identity convention introduces no new model parameter.

Construction Or Encoding

Python returns its single positional-only argument unchanged. Lean defines `phi x := x` and proves `phi_eq_input` and `phi_eq_identity` by reflexivity.

Boundary Case Or Countercheck

The API must preserve the exact input object and expose no `mode`, `scale`, `slope`, or `coefficient`. Names associated with logarithmic, saturated, symmetric, or null controls must not appear in the public export set.

Result

C015 contributes no numerical operation beyond identity and no branch selection.

Downstream Handoff

The unchanged exact scalar is consumed by M003 after N001/M005 unit normalization.

Code Anchors

python / SOURCE: `s08_c015__active_linear_steering_mode_phi_x_x.py`

Path: `derivation/code/python/src/cnna/derivation/s01_primitive_response_coupled_finite_provenance/s03_recurrent_pre_birth_measurement_and_steering/s08_c015__active_linear_steering_mode_phi_x_x.py`
SHA-256: `92458bb05b2008b7458fa2a5d72fac8676796c696769993b989374a7b574c129`

Symbol	Kind	Lines
`active_linear_steering`	`FUNCTION`	20-22

python_test / TEST: `test_s08_c015__active_linear_steering_mode_phi_x_x.py`

Path: `derivation/code/python/tests/derivation/s01_primitive_response_coupled_finite_provenance/s03_recurrent_pre_birth_measurement_and_steering/test_s08_c015__active_linear_steering_mode_phi_x_x.py`
SHA-256: `97b94a48f62c0a1bd840bc13310a490f42ffded93f6bf433ff37f2edb8d9f394`

Symbol	Kind	Lines
`TestActivePathIdentityTransform`	`CLASS`	14-50

lean_core / SOURCE: `S08_C015_ActiveLinearSteeringModePhiXX.lean`

Path: `derivation/code/lean/core/src/CNNA/Derivation/S01_PrimitiveResponseCoupledFiniteProvenance/S03_RecurrentPreBirthMeasurementAndSteering/S08_C015_ActiveLinearSteeringModePhiXX.lean`
SHA-256: `34378ada95397d5c421585f94834e6fc5e019613bc87a873771e0b32a1630013`

Symbol	Kind	Lines
`phi`	`DEF`	23-25
`phi_eq_input`	`THEOREM`	26-28
`phi_eq_identity`	`THEOREM`	29-34

024 · M003 — Canonical response-steering functional Sigma_b[R_n,s]

Canonical node label: `024 · M003`
Semantic ID: `M003`
Current section path: `1.3.9`
Documentation tier: `D2`

Position In Derivation

M003 receives the exact C007 response in M001 boundary order and supplies the unique exact scalar and proved positive domain consumed by M004.

Formal Statement

For `p = parentAddress(next)`, `sigma(next,lambda)` is the address-filtered sum of the `lambda[i,i]` terms whose boundary address equals `p`. With `C_star = 1` and the C015 identity transform, no further normalization changes the value. `CanonicalM003Closure realization` states that the canonical realization lies in `InPositiveSteeringDomain`, has a response-steering pair, and every response-steering pair has `PositiveSteering`.

Hypotheses

- `next` is the canonical C004 slot of a C005 response-capable state.
- `realization` is the actual C005/M001/C006/C007 state-directed block realization.
- The P001 reusable directed-cut hypotheses are derived for that realization; no parent coordinate or positivity witness is a public input.

Introduction Reason

C007 returns a boundary response matrix while M004 requires one provenance-selected scalar. The closure must discharge the distinguished coordinate internally so downstream code cannot choose a different port.

Proof Strategy

Core establishes address membership, exact aggregation, uniqueness, and representative invariance. P001 establishes response well-posedness and strict distinguished-port positivity. `canonicalM003Closure` obtains the parent coordinate from `distinguishedParentIndex_exists`, constructs the internal witness, and packages the resulting domain, existence, and universal positivity statements.

Lemma Chain

```
parent_mem_boundary -> sigma -> responseSteeringPair_exists
P001 canonical cut closure -> canonicalInPositiveSteeringDomain
canonicalInPositiveSteeringDomain -> canonicalResponseSteeringPair_positive
internal distinguishedParentIndex_exists -> canonicalM003Closure
```

Formal Realization

The mathlib-free Core defines the scalar and predicates. The proof module `S01_CanonicalM003Closure.lean` imports only the verified P001 facade and exports a public theorem whose only explicit data argument is the canonical realization.

Counterexamples Or Necessity Checks

- A missing or duplicated parent address cannot be repaired by a selected coordinate.
- Zero is retained as an exact negative control but is not in the active positive domain.
- Equivalent fraction or matrix representatives must not alter the scalar.
- Rank, sibling number, depth, clipping, baseline, or hidden mode parameters are absent.

Axiom Profile

P001 remains bound to its verified 142-declaration profile. All four M003 closure declarations are kernel-compiled and axiom-audited: two use `propext` and `Quot.sound`, and two additionally use transitive `Classical.choice`. No project-local axiom or `sorry` is admitted.

Result

M003 has a closed end-to-end interface without an external parent index: canonical response-domain inhabitance, response-steering existence, and universal strict positivity are packaged in `CanonicalM003Closure`.

Remaining Limits

The result is finite and rational. Transitive `propext`, `Classical.choice`, and `Quot.sound` remain within the explicitly admitted Lean/mathlib trust boundary; their elimination is not claimed.

Downstream Handoff

M004 consumes `CanonicalM003Closure` directly. It does not reconstruct the Schur/DtN proof and does not receive positivity as a caller-supplied assumption.

Code Line Register

python / SOURCE: `s09_m003__canonical_response_steering_functional_sigma_b_rn_s.py`

Path: `derivation/code/python/src/cnna/derivation/s01_primitive_response_coupled_finite_provenance/s03_recurrent_pre_birth_measurement_and_steering/s09_m003__canonical_response_steering_functional_sigma_b_rn_s.py`
SHA-256: `bee99c2edb4d1de28d4366dea75f1d587eb0c25146f104f98862cc85120a1009`

Symbol	Kind	Lines
`CanonicalResponseSteering`	`CLASS`	47-72
`is_positive_response_steering`	`FUNCTION`	75-82
`parent_port_self_response`	`FUNCTION`	85-99
`canonical_response_steering_functional`	`FUNCTION`	102-120

python_test / TEST: `test_s09_m003__canonical_response_steering_functional_sigma_b_rn_s.py`

Path: `derivation/code/python/tests/derivation/s01_primitive_response_coupled_finite_provenance/s03_recurrent_pre_birth_measurement_and_steering/test_s09_m003__canonical_response_steering_functional_sigma_b_rn_s.py`
SHA-256: `911102ca3ae3fe8a4a822b1569f4d8b52e9cf040635971dc3812baa324a8a067`

Symbol	Kind	Lines
_bootstrap_state	FUNCTION	33-39
_state	FUNCTION	42-53
TestCanonicalResponseSteeringFunctional	CLASS	56-148

lean_core / SOURCE: S09_M003_CanonicalResponseSteeringFunctionalSigmaBRnS.lean

Path: derivation/code/lean/core/src/CNNA/Derivation/S01_PrimitiveResponseCoupledFiniteProvenance/S03_RecurrentPreBirthMeasurementAndSteering/S09_M003_CanonicalResponseSteeringFunctionalSigmaBRnS.lean
SHA-256: 1ecd0f0d2afe74458cae30490db430306c0ee9e79550eada2b2f80b66f398c8e

Symbol	Kind	Lines
terminal_mem_prefixChainAux	THEOREM	38-60
parent_mem_causalPredecessorPorts	THEOREM	61-71
parent_mem_boundary	THEOREM	72-79
parentDiagonalTerm	DEF	80-90
parentSelfResponse	DEF	91-101
unitNormalizedParentResponse	DEF	102-108
selected_conductance_unit_eq_one	THEOREM	109-114
sigma	DEF	115-121
IsCanonicalResponseSteering	DEF	122-128
sigma_eq_unitNormalizedParentResponse	THEOREM	129-136
sigma_eq_parentSelfResponse	THEOREM	137-145
PositiveSteering	DEF	146-150
parentDiagonalTerm_respects_matrixSameValue	THEOREM	151-168
parentSelfResponse_respects_matrixSameValue	THEOREM	169-184
sigma_respects_matrixSameValue	THEOREM	185-197
steering_exists	THEOREM	198-206
steering_unique	THEOREM	207-218
response_representatives_give_same_steering	THEOREM	219-231
IsResponseSteeringPair	DEF	232-241
IsPositiveResponseSteeringPair	DEF	242-252
DirectedKronParentPositivityAt	DEF	253-262
InPositiveSteeringDomain	DEF	263-268
inPositiveSteeringDomain_iff	THEOREM	269-276
responseSteeringPair_exists	THEOREM	277-289
responseSteeringPair_value_unique	THEOREM	290-311

lean_proof / PROOF: S01_CanonicalM003Closure.lean

Path: derivation/code/lean/proofs/src/CNNAProofs/M003M004/S01_CanonicalM003Closure.lean
SHA-256: d96d928f48f0780e3b728b3777d2b58e99349ef901743113f172b1a5c0e7ce9c

Symbol	Kind	Lines
CanonicalM003Closure	STRUCTURE	28-44
canonicalM003Closure	THEOREM	45-65
CanonicalM003ClosureContract	DEF	66-72
canonicalM003ClosureContract	THEOREM	73-76

Open-provenance role: Response-to-event specialization

M003 converts the effective boundary response into the unique positive scalar used by the next event. This is the response-coupling step of the current deterministic specialization, not a universal law for all open systems.

---

025 · P001 — Reusable directed Schur/DtN/Kron channel closure

Canonical node label: `025 · P001`
Semantic ID: `P001`
Current section path: `1.3.9.1`
Documentation tier: `D2`

Position In Derivation

P001 is the sole proof owner for the reusable directed Schur/DtN/Kron closure used by C006, C007, M003, and M004. The Core package remains mathlib-free. The proof package may import pinned mathlib and may depend on Core; the reverse dependency is forbidden.

Formal Statement

For finite boundary and interior index types, ordered rational blocks `kBB`, `kBI`, `kIB`, and `kII`, and a distinguished boundary coordinate, P001 proves the unregularized closure from four explicit hypotheses:

- all off-diagonal entries of the full block operator are nonpositive;
- every full row sum is exactly zero;
- every interior coordinate reaches the boundary along positive arcs;
- the distinguished boundary coordinate reaches a different boundary coordinate along a positive path.

The resulting public contract includes exact semantic agreement, interior-solve existence and uniqueness, C006 admissibility, response existence, response-witness independence, directed-Laplacian response structure, strict distinguished-port positivity, the canonical M003 scalar handoff, M003/M004 supporting theorems, and independent reuse on a second cut family.

Hypotheses

The theorem uses the original ordered blocks. It assumes no symmetry, reversibility, inverse, pseudoinverse, regularization, additional grounding vertex, or separately postulated strong connectivity. For the canonical birth-local cut, the four generic hypotheses are derived from C005, M001, M002, and C007. For the independent cut, they are proved directly from two strictly positive rational edge weights.

Introduction Reason

C006 defines a directed Schur/DtN response at the Core level, while M003 requires a strict positive parent-port response. Without one explicit proof owner, analytic invertibility, sign conventions, representative independence, and the canonical connectivity argument could be duplicated or silently strengthened. P001 centralizes those obligations and makes reuse falsifiable.

Proof Strategy

Exact semantic bridge

Core `ExactFraction` values are mapped to rational values, and Core matrix addition, multiplication, subtraction, interior solve, harmonic sign, and response construction are proved entrywise equivalent to transparent rational operations. Rectangular multiplication is stated as the explicit row-by-column finite sum.

Directed maximum principle and finite solve

A zero-boundary harmonic function satisfies a maximum-defect sum identity. Every defect is nonnegative; zero total defect forces equality across each positive arc. Positive reachability propagates an interior maximum to the boundary, forcing the zero-boundary solution to vanish. The resulting interior linear map is injective; finite equal dimension gives surjectivity and hence existence and uniqueness of the solve.

Response structure and strict positivity

Harmonic boundary basis functions lie between zero and one. This yields nonpositive off-diagonal response entries and exact row conservation. The distinguished diagonal is nonnegative. If it were zero, the value one would propagate along the distinguished positive path to a different boundary port whose Dirichlet value is zero, a contradiction.

Canonical cut derivation

M001 coordinates are proved duplicate-free and complete. C007 entries are rewritten as outgoing-degree indicators minus ordered-pair conductance sums. Stored positive conductances induce positive arcs. Interior paths follow the provenance parent relation and decrease depth strictly until the boundary is reached. The distinguished parent reaches another boundary port through the stored bidirectional backbone.

M003/M004 supporting interface

The unique canonical parent coordinate reduces M003's address-filtered parent aggregate to the distinguished response diagonal. The P001 M004 predicate hides only proposition-valued positivity evidence; `Subsingleton.elim` aligns such proofs before invoking the existing Core uniqueness theorem. No proof witness is retained as physical model data.

Independent cut-family reuse

For positive `leftWeight` and `rightWeight`, S11 defines two boundary coordinates and one interior coordinate with blocks

```
KBB = diag(leftWeight, rightWeight)
KBI = (-leftWeight, -rightWeight)^T
KIB = (-leftWeight, -rightWeight)
KII = (leftWeight + rightWeight)
```

All four generic hypotheses are proved directly, and `independentBidirectedChainClosure` calls only `directedSchurDtnClosure`. The module has no state, next-slot, provenance-address, M001, or C007 parameter.

Lemma Chain

1. exact fraction and matrix semantics;
2. zero-boundary extension and Laplacian action;
3. maximum-defect nonnegativity and positive-arc propagation;
4. interior-kernel triviality;
5. finite linear existence and uniqueness;
6. response existence and witness independence;
7. harmonic boundary basis bounds;
8. response off-diagonal sign, row conservation, and diagonal nonnegativity;
9. strict distinguished-port positivity;
10. canonical coordinate, matrix, and reachability derivation;
11. M003/M004 supporting theorems;
12. independent bidirected-chain hypotheses and closure.

Formal Realization

Lean source is split into the aggregate contract module plus S01–S11. The aggregate import is `CNNAProofs.P001.S11_IndependentBidirectedChainCutReuse`, so the public package includes the independent reuse theorem. The proof build is pinned to Lean 4.31.0 and mathlib v4.31.0.

Counterexamples Or Necessity Checks

- Dropping interior-to-boundary reachability permits a nontrivial zero-boundary interior kernel.
- Dropping the distinguished path to another boundary port removes the strict-positivity contradiction.
- Replacing row conservation by an approximate identity does not prove the exact directed-Laplacian contract.
- Assuming symmetry would exclude the intended directed setting rather than prove it.
- Using an inverse or regularizer would change the C006 model and is therefore forbidden.
- Verifying only the canonical birth cut would not establish generic reuse; the independent bidirected-chain family closes this countercheck.

Axiom Profile

The exact user-local build completed 26 Core jobs and 8595 proof jobs. All 142 registered declarations passed `P001_CURRENT_PROOF_AXIOM_AUDIT`; `FULL_PACKAGE_BOUNDARY_AUDIT` passed; and the P001 source emitted no warning. The exact transcript is:

```
derivation/code/lean/audit/evidence/USER_LOCAL_P001_FULL_BUILD_20260806.txt
SHA-256 3329291658b2d7a5f46acc6c1bf48b8a60f6bade5d010aa96d7978be3943170a
```

The registered profile partition is:

```
117 declarations: propext, Classical.choice, Quot.sound
23 declarations:  propext, Quot.sound
2 declarations:   no axioms
```

No project-local axiom or `sorryAx` occurs. The three transitive Lean/mathlib axioms remain part of the declared trust boundary; their constructive elimination is not claimed.

Result

P001 is kernel-verified for all 142 registered declarations. The generic directed closure is instantiated on the canonical birth-local cut and independently on a state-free bidirected-chain cut. M003 strict positive steering and the unique M004 birth-law relation are derived without changing their Core definitions.

Remaining Limits

The result is finite and rational. It does not by itself establish continuum limits, spectral asymptotics, infinite-volume operator algebras, or elimination of the transitive Lean/mathlib axioms. Those claims require separate proof nodes.

Downstream Handoff

M003 receives a verified positive-domain witness and strict response-steering theorem. M004 receives existence, uniqueness, and representative independence for the derived canonical birth law. Later cuts may reuse `directedSchurDtnClosure` after proving their own four explicit hypotheses.

Code Line Register

lean_proof: `S04_ResponseWellDefinedness.lean`

Path: `derivation/code/lean/proofs/src/CNNAProofs/P001/S04_ResponseWellDefinedness.lean`
SHA-256: `c80d1dc00f55cd84420beb3d747d8b3ec33c0137a41a2a97627f23c94f7ffa07`

Symbol	Kind	Lines
`exactMatrixValue_eq_of_matrixSameValue`	THEOREM	32-41
`c006InteriorAdmissible`	THEOREM	42-56
`responseExists`	THEOREM	57-66
`responseRepresentativeAgreement`	THEOREM	67-84
`responseWitnessIndependent`	THEOREM	85-95
`responseWellDefined`	THEOREM	96-107

lean_proof: `S02_DirectedMaximumPrinciple.lean`

Path: `derivation/code/lean/proofs/src/CNNAProofs/P001/S02_DirectedMaximumPrinciple.lean`
SHA-256: `9cc036efcd855b49df4a8c0abe5d58a0ef793b0bc2c302e8f312a654ab561aa4`

Symbol	Kind	Lines
`maximumDefectTerm`	DEF	30-37
`zeroBoundaryExtension_vanishesOnBoundary`	THEOREM	38-44
`laplacianAction_zeroBoundaryExtension`	THEOREM	45-66
`zeroBoundaryExtension_isInteriorHarmonic`	THEOREM	67-77
`laplacianAction_neg`	THEOREM	78-95
`maximumDefectSum_eq_zero`	THEOREM	96-126

Symbol	Kind	Lines
maximumDefectTerm_nonnegative	THEOREM	127-145
maximum_propagates_across_positive_arc	THEOREM	146-186
maximum_propagates_to_boundary	THEOREM	187-225
interior_le_zero_of_harmonic_zero_boundary	THEOREM	226-263
interior_nonnegative_of_harmonic_zero_boundary	THEOREM	264-289
interior_eq_zero_of_harmonic_zero_boundary	THEOREM	290-305
interiorKernelTrivial	THEOREM	306-322

lean_proof: S05_ResponseDirectedLaplacian.lean

Path: derivation/code/lean/proofs/src/CNNAProofs/P001/S05_ResponseDirectedLaplacian.lean
SHA-256: 7cfb905f5c409e8daa737037527f364ccc6fb6a23a49b1d851a0c6cf941270dc

Symbol	Kind	Lines
boundaryBasis	DEF	35-39
boundaryBasis_nonnegative	THEOREM	40-48
boundaryBasis_le_one	THEOREM	49-58
harmonicBasisPotential	DEF	59-67
interiorSolve_columnEquation	THEOREM	68-82
harmonicBasisPotential_isInteriorHarmonic	THEOREM	83-127
interior_le_of_harmonic_boundary_le	THEOREM	128-165
interior_ge_of_harmonic_boundary_ge	THEOREM	166-193
harmonicBasisPotential_nonnegative	THEOREM	194-213
harmonicBasisPotential_le_one	THEOREM	214-234
mathlibResponse_entry_eq_laplacianAction_harmonicBasis	THEOREM	235-281
responseOffDiagonalNonpositive	THEOREM	282-314
interiorSolve_rowSum_eq_neg_one	THEOREM	315-387
mathlibResponse_rowConservative	THEOREM	388-444
responseRowConservative	THEOREM	445-465

Symbol	Kind	Lines
responseDiagonalNonnegative_of_offDiagonal_rowConservative	THEOREM	466-508
responseDiagonalNonnegative	THEOREM	509-523
directedLaplacianClosure	THEOREM	524-540

lean_proof: S03_FiniteLinearWellPosedness.lean

Path: derivation/code/lean/proofs/src/CNNAProofs/P001/S03_FiniteLinearWellPosedness.lean
SHA-256: b0ef0574fd41c4f11e6dd43b81e73634030776a8d9265021d07e8ea4f302f317

Symbol	Kind	Lines
interiorLinearMap	DEF	31-37
interiorLinearMap_apply	THEOREM	38-51
interiorLinearMap_injective	THEOREM	52-75
interiorLinearMap_surjective	THEOREM	76-83
interiorRightHandSideSolveExists	THEOREM	84-93
interiorSolveExists	THEOREM	94-120
interiorSolveUnique	THEOREM	121-163
interiorWellPosed	THEOREM	164-171

lean_proof: DirectedSchurDtnKronChannelClosure.lean

Path: derivation/code/lean/proofs/src/CNNAProofs/P001/DirectedSchurDtnKronChannelClosure.lean
SHA-256: 19fce24b2000c8ff64690c0b78e4dacc9115bdf34b713bf6054f95f37886db43

Symbol	Kind	Lines
RationalMatrix	ABBREV	46-50
coreRatMatrixValue	DEF	51-57
rationalMatrixMul	DEF	58-63
CutVertex	ABBREV	64-67
exactFractionValue	DEF	68-71
exactMatrixValue	DEF	72-76
blockEntry	DEF	77-87
PositiveArc	DEF	88-93
PositivePath	INDUCTIVE	94-107
InteriorPathToBoundary	INDUCTIVE	108-120
DirectedCutHypotheses	STRUCTURE	121-137
CutPotential	ABBREV	138-141
laplacianAction	DEF	142-148
VanishesOnBoundary	DEF	149-153
IsInteriorHarmonic	DEF	154-160
IsInteriorKernelVector	DEF	161-166
zeroBoundaryExtension	DEF	167-175
InteriorKernelTrivial	DEF	176-182
IsMathlibInteriorSolve	DEF	183-189
IsHarmonicExtension	DEF	190-196

Symbol	Kind	Lines
mathlibResponseFromSolve	DEF	197-204
ExactSemanticBridge	STRUCTURE	205-219
InteriorSolveExists	DEF	220-225
InteriorSolveUnique	DEF	226-234
ResponseWitnessIndependent	DEF	235-242
ResponseOffDiagonalNonpositive	DEF	243-247
ResponseRowConservative	DEF	248-252
ResponseDiagonalNonnegative	DEF	253-258
IsDirectedLaplacianResponse	DEF	259-265
DistinguishedPortStrictlyPositive	DEF	266-271
DirectedSchurDtnClosure	STRUCTURE	272-293
ReusableDirectedClosureContract	DEF	294-302
DistinguishedParentIndex	STRUCTURE	303-310
CanonicalBirthCutClosure	STRUCTURE	311-321
CanonicalBirthCutClosureContract	DEF	322-332
PublicContract	DEF	333-336

lean_proof: S01_ExactSemanticBridge.lean

Path: derivation/code/lean/proofs/src/CNNAProofs/P001/S01_ExactSemanticBridge.lean
SHA-256: 7139f45922bc14d7f8de4180075f7fe4ac687bd9d54b893dcb0e21a092b723d1

Symbol	Kind	Lines
exactFractionValue_ofRat	THEOREM	29-34
sameValue_iff_exactFractionValue_eq	THEOREM	35-43
represents_iff_exactFractionValue_eq	THEOREM	44-50
exactFractionValue_zero	THEOREM	51-56
exactFractionValue_add	THEOREM	57-65
exactFractionValue_mul	THEOREM	66-72
exactFractionValue_sub	THEOREM	73-92
exactFractionValue_finFoldl_add	THEOREM	93-111
finFoldl_add_eq_initial_add_sum	THEOREM	112-123
finFoldl_add_eq_sum	THEOREM	124-129
exactFractionValue_matrixMul_entry	THEOREM	130-145
exactMatrixValue_matrixMul	THEOREM	146-156
exactMatrixValue_matrixSub	THEOREM	157-181
matrixRepresents_iff_exactMatrixValue_eq	THEOREM	182-197
rationalMatrixMul_neg_right	THEOREM	198-213
interiorSolveAgreement	THEOREM	214-223
harmonicSignAgreement	THEOREM	224-239
responseValueAgreement	THEOREM	240-250
exactSemanticBridge	THEOREM	251-258

lean_proof: S06_DistinguishedPortStrictPositivity.lean

Path: derivation/code/lean/proofs/src/CNNAProofs/P001/S06_DistinguishedPortStrictPositivity.lean
SHA-256: 4a4c4a94cd888638f483bdefa345798337acd39bbe0d5734524599ad1aa761c5

Symbol	Kind	Lines
maximum_propagates_from_distinguished_boundary_across_positive_arc	THEOREM	35-74
harmonicBasis_one_propagates_across_positive_arc	THEOREM	80-146
harmonicBasis_one_propagates_along_positive_path	THEOREM	150-174
harmonicBasis_distinguished_action_ne_zero	THEOREM	179-208
distinguishedResponseDiagonal_ne_zero	THEOREM	212-246
distinguishedPortStrictlyPositive	THEOREM	250-267
directedSchurDtnClosure	THEOREM	270-288
reusableDirectedClosureContract	THEOREM	292-294

lean_proof: S07_CanonicalBirthCutInstantiation.lean

Path: derivation/code/lean/proofs/src/CNNAProofs/P001/S07_CanonicalBirthCutInstantiation.lean
SHA-256: b2ebfafbb319bc64049eacec675f34fb4b7473280b73e03ba6d5d31a31217146

Symbol	Kind	Lines
bornNonRoot_nodup	THEOREM	33-39
root_not_mem_bornNonRoot	THEOREM	42-48
canonicalCarrier_nodup	THEOREM	51-57
boundary_nodup	THEOREM	58-61
interior_nodup	THEOREM	64-70
distinguishedParentIndex_exists	THEOREM	71-80
positiveSteering_of_exactFractionValue_pos	THEOREM	81-98
parentSelfResponse_value_eq_parentDiagonal	THEOREM	99-135
m003ParentPositivity_of_genericClosure	THEOREM	136-164
canonicalBirthCutClosure_of_hypotheses	THEOREM	165-181
canonicalBirthCutClosureContract	THEOREM	182-184

lean_proof: S08_CanonicalDirectedMatrixStructure.lean

Path: derivation/code/lean/proofs/src/CNNAProofs/P001/S08_CanonicalDirectedMatrixStructure.lean
SHA-256: 0975bb3958c81032d07f27d1d0c58c5057942d70b884d4ad5bc69aeea99475f5

Symbol	Kind	Lines
canonicalCutAddress	DEF	30-38
canonicalCutAddress_injective	THEOREM	39-89
canonicalCutCoordinate_exists	THEOREM	90-103
conductanceSourceCoordinate_exists	THEOREM	104-111
conductanceTargetCoordinate_exists	THEOREM	112-120

Symbol	Kind	Lines
ratOutgoingSum	DEF	121-128
ratOrderedPairSum	DEF	129-138
exactFractionValue_outgoingFold	THEOREM	139-175
exactFractionValue_orderedPairFold	THEOREM	176-223
exactFractionValue_outgoingSum	THEOREM	224-230
exactFractionValue_orderedPairSum	THEOREM	231-239
ratDirectedMatrixEntry	DEF	240-246
exactFractionValue_directedMatrixEntry	THEOREM	247-259
ratOrderedPairSum_nonnegative	THEOREM	260-284
ratOrderedPairSum_pos_of_hasConductance	THEOREM	285-328
ratOrderedPairSum_self_zero	THEOREM	329-349
sum_single_edge_target_indicator	THEOREM	350-388
sum_ratOrderedPairSum_eq_ratOutgoingSum	THEOREM	389-456
ratDirectedMatrixEntry_eq_indicator_sub_pair	THEOREM	457-481
ratDirectedMatrixEntry_row_sum_zero	THEOREM	482-544
blockEntry_eq_ratDirectedMatrixEntry	THEOREM	545-607
canonicalBlocks_offDiagonalNonpositive	THEOREM	608-624
canonicalBlocks_rowConservative	THEOREM	625-641
canonicalPositiveArc_of_hasConductance	THEOREM	642-670

lean_proof: S09_CanonicalBackboneReachability.lean

Path: derivation/code/lean/proofs/src/CNNAProofs/P001/S09_CanonicalBackboneReachability.lean
SHA-256: 49110c72b52c07384f0f67a37945f53b88565940464a5c5e62c9dcbaa60074a1

Symbol	Kind	Lines
eq_snoc_of_parent?_eq_some	THEOREM	101-123
depth_parent_lt_of_parent?_eq_some	THEOREM	124-155
immediateParent_mem_causalPredecessorPorts	THEOREM	156-166
hasConductance_endpoints_distinct	THEOREM	167-176
firstProvenanceSlotOfState	DEF	177-181
firstProvenanceAddress_born	THEOREM	182-222
firstProvenanceAddress_mem_olderSiblingPorts_of_parent_root	THEOREM	223-296
canonicalInteriorPathToBoundary_aux	THEOREM	297-371
canonicalEveryInteriorReachesBoundary	THEOREM	372-400
canonicalDistinguishedReachesOtherBoundary	THEOREM	401-489
canonicalDirectedCutHypotheses	THEOREM	490-502
canonicalBirthCutClosure_derived	THEOREM	503-511

Symbol	Kind	Lines
DerivedCanonicalBirthCutClosureContract	DEF	512-519
derivedCanonicalBirthCutClosureContract	THEOREM	520-526
DerivedPublicContract	DEF	527-530
derivedPublicContract	THEOREM	531-534

lean_proof: S10_M003M004ProofFacades.lean

Path: derivation/code/lean/proofs/src/CNNAProofs/P001/S10_M003M004ProofFacades.lean
SHA-256: 1b0432d2df791fb24dc4d70937b078bea8f89fe42824b9aeb41113977088c53e

Symbol	Kind	Lines
canonicalInPositiveSteeringDomain	THEOREM	32-43
canonicalResponseSteeringPair_positive	THEOREM	44-59
IsDerivedCanonicalBirthLaw	DEF	60-71
derivedCanonicalBirthLaw_exists	THEOREM	72-90
derivedCanonicalBirthLaw_unique	THEOREM	91-111
derivedCanonicalBirthLaw_existsUnique	THEOREM	112-131
canonicalActiveBirthInstruction_exists	THEOREM	132-152
derivedCanonicalBirthLaws_sameValue	THEOREM	153-182
M003M004ProofFacadeContract	DEF	183-214
m003M004ProofFacadeContract	THEOREM	215-229

lean_proof: S11_IndependentBidirectedChainCutReuse.lean

Path: derivation/code/lean/proofs/src/CNNAProofs/P001/S11_IndependentBidirectedChainCutReuse.lean
SHA-256: 1bf9b8cb1a7793c1a2c15f07b1d09fe3f959588f8f7eabcd4591bf1c9f85f470

Symbol	Kind	Lines
independentChainBoundaryWeight	DEF	28-30
independentBidirectedChainBlocks	DEF	34-45
independentBidirectedChainOffDiagonalNonpositive	THEOREM	48-81
independentBidirectedChainRowConservative	THEOREM	84-156
independentBidirectedChainInteriorReachesBoundary	THEOREM	159-175
independentBidirectedChainDistinguishedReachesOtherBoundary	THEOREM	179-202
independentBidirectedChainHypotheses	THEOREM	206-223
independentBidirectedChainClosure	THEOREM	227-236

Symbol	Kind	Lines
SecondCutReuseContract	DEF	240-245
secondCutReuseContract	THEOREM	249-252

External References

EXT-REF-MATHLIB-009 — api reuse. Johannes Hölzl, Patrick Massot, Casper Putz, and Anne Baanen, *Mathlib module: LinearAlgebra.Matrix.ToLin*, Leanprover Community (2026). DOI status: NOT_ASSIGNED_SOFTWARE_SOURCE. Stable source: https://leanprover-community.github.io/mathlib4_docs/Mathlib/LinearAlgebra/Matrix/ToLin.html; accessed 2026-08-03. Exact location: Matrix.mulVecLin; Matrix.mulVecLin_apply; pinned source commit fabf563a7c95a166b8d7b6efca11c8b4dc9d911f. Context: Documents the exact bundled matrix action used by the kernel-verified finite-linear proof. Formal status: PROOF_API_USED_FINITE_LINEAR_WELL_POSEDNESS

EXT-REF-LEAN-004 — api reuse. The Lean 4 Development Team, *Lean core module: Init.Data.Rat.Lemmas*, Lean FRO, LLC (2026). DOI status: NOT_ASSIGNED_SOFTWARE_SOURCE. Stable source: https://leanprover-community.github.io/mathlib4_docs/Init/Data/Rat/Lemmas.html; accessed 2026-08-03. Exact location: Rat.mkRat_self; Rat.mkRat_eq_iff; Rat.mkRat_add_mkRat; Rat.mkRat_mul_mkRat; Rat.neg_mkRat; Lean toolchain v4.31.0. Context: Supplies the exact constructor lemmas used to identify C006 fraction arithmetic with ℚ. Formal status: PROOF_API_USED_EXACT_SEMANTIC_BRIDGE

EXT-REF-LEAN-005 — api reuse. The Lean 4 Development Team, *Lean core module: Init.Data.Fin.Fold*, Lean FRO, LLC (2026). DOI status: NOT_ASSIGNED_SOFTWARE_SOURCE. Stable source: https://leanprover-community.github.io/mathlib4_docs/Init/Data/Fin/Fold.html; accessed 2026-08-03. Exact location: Fin.foldl_zero; Fin.foldl_succ; Lean toolchain v4.31.0. Context: Supplies the recursion equations used by the fold-to-sum induction. Formal status: PROOF_API_USED_EXACT_SEMANTIC_BRIDGE

EXT-REF-MATHLIB-005 — api reuse. Leanprover Community, *Mathlib module: Algebra.BigOperators.Fin*, Leanprover Community (2026). DOI status: NOT_ASSIGNED_SOFTWARE_SOURCE. Stable source: https://leanprover-community.github.io/mathlib4_docs/Mathlib/Algebra/BigOperators/Fin.html; accessed 2026-08-03. Exact location: Fin.sum_univ_zero; Fin.sum_univ_succ; pinned source commit fabf563a7c95a166b8d7b6efca11c8b4dc9d911f. Context: Supplies the finite-sum recursion paired with Fin.foldl. Formal status: PROOF_API_USED_EXACT_SEMANTIC_BRIDGE

EXT-REF-MATHLIB-004 — api reuse. Ellen Arlt, Blair Shi, Sean Leather, Mario Carneiro, Johan Commelin, and Lu-Ming Zhang, *Mathlib module: Data.Matrix.Mul*, Leanprover Community (2026). DOI status: NOT_ASSIGNED_SOFTWARE_SOURCE. Stable source: https://leanprover-community.github.io/mathlib4_docs/Mathlib/Data/Matrix/Mul.html; accessed 2026-08-03. Exact location: implementation notes and rectangular multiplication definition; pinned source commit fabf563a7c95a166b8d7b6efca11c8b4dc9d911f. Context: Documents the transparent rectangular product at the Core-to-proof boundary. Formal status: PROOF_API_USED_EXACT_SEMANTIC_BRIDGE

EXT-REF-MATHLIB-007 — api reuse. Leanprover Community, *Mathlib module: Algebra.Order.BigOperators.Group.Finset*, Leanprover Community (2026). DOI status: NOT_ASSIGNED_SOFTWARE_SOURCE. Stable source: https://leanprover-community.github.io/mathlib4_docs/Mathlib/Algebra/Order/BigOperators/Group/Finset.html; accessed 2026-08-03. Exact location: Finset.sum_eq_zero_iff_of_nonneg; pinned source commit fabf563a7c95a166b8d7b6efca11c8b4dc9d911f. Context: Documents the exact finite ordered-sum implication used by the directed maximum-principle proof. Formal status: PROOF_API_USED_DIRECTED_MAXIMUM_PRINCIPLE_AND_STRICT_POSITIVITY

EXT-REF-MATHLIB-006 — api reuse. Leanprover Community, *Mathlib module: Data.Finset.Max*, Leanprover Community (2026). DOI status: NOT_ASSIGNED_SOFTWARE_SOURCE. Stable source: https://leanprover-community.github.io/mathlib4_docs/Mathlib/Data/Finset/Max.html; accessed 2026-08-03. Exact location: Finset.exists_max_image;

pinned source commit fabf563a7c95a166b8d7b6efca11c8b4dc9d911f. Context: Documents the finite maximum-selection theorem used in the directed maximum-principle proof. Formal status: PROOF_API_USED_DIRECTED_MAXIMUM_PRINCIPLE

EXT-REF-MATHLIB-008 — api reuse. Leanprover Community, *Mathlib module: Data.Fintype.BigOperators*, Leanprover Community (2026). DOI status: NOT_ASSIGNED_SOFTWARE_SOURCE. Stable source: https://leanprover-community.github.io/mathlib4_docs/Mathlib/Data/Fintype/BigOperators.html; accessed 2026-08-03. Exact location: Fintype.sum_sum_type; Fintype.sum_eq_single; Fintype.sum_eq_zero; pinned source commit fabf563a7c95a166b8d7b6efca11c8b4dc9d911f. Context: Documents the sum-type decomposition and whole-Fintype selected/zero sum APIs used by P001. Formal status: PROOF_API_USED_DIRECTED_MAXIMUM_PRINCIPLE

EXT-REF-MATHLIB-002 — api reuse. Chris Hughes, *Mathlib module: FiniteDimensional.Basic*, Leanprover Community (2026). DOI status: NOT_ASSIGNED_SOFTWARE_SOURCE. Stable source: https://leanprover-community.github.io/mathlib4_docs/Mathlib/LinearAlgebra/FiniteDimensional/Basic.html; accessed 2026-08-03. Exact location: LinearMap.surjective_of_injective; LinearMap.injective_iff_surjective; pinned source commit fabf563a7c95a166b8d7b6efca11c8b4dc9d911f. Context: Documents the finite-dimensional injectivity-to-surjectivity bridge used by the kernel-verified P001 finite-linear proof. Formal status: PROOF_API_USED_FINITE_LINEAR_WELL_POSEDNESS

EXT-REF-MATHLIB-001 — api coverage audit. Alexander Bentkamp, Eric Wieser, Jeremy Avigad, and Johan Commelin, *Mathlib module: Matrix.SchurComplement*, Leanprover Community (2026). DOI status: NOT_ASSIGNED_SOFTWARE_SOURCE. Stable source: https://leanprover-community.github.io/mathlib4_docs/Mathlib/LinearAlgebra/Matrix/SchurComplement.html; accessed 2026-08-03. Exact location: module header and main results; pinned source commit fabf563a7c95a166b8d7b6efca11c8b4dc9d911f. Context: Records the exact mathlib Schur-complement API boundary used by the declared contract. Formal status: API_CONTEXT_ONLY_NO_IMPORTED_HYPOTHESIS

EXT-REF-MATHLIB-003 — api scope. Alexander Bentkamp and Mohanad Ahmed, *Mathlib module: Matrix.PosDef*, Leanprover Community (2026). DOI status: NOT_ASSIGNED_SOFTWARE_SOURCE. Stable source: https://leanprover-community.github.io/mathlib4_docs/Mathlib/LinearAlgebra/Matrix/PosDef.html; accessed 2026-08-03. Exact location: module header; Matrix.PosSemidef and Matrix.PosDef definitions; pinned source commit fabf563a7c95a166b8d7b6efca11c8b4dc9d911f. Context: Documents the deliberate exclusion of Hermitian positive definiteness from the directed P001 contract. Formal status: API_CONTEXT_ONLY_NO_IMPORTED_HYPOTHESIS

EXT-REF-DKP-001 — load-bearing theorem source. Tomohiro Sugiyama and Kazuhiro Sato, *Kron Reduction and Effective Resistance of Directed Graphs*, SIAM Journal on Matrix Analysis and Applications 44(1) (2023), 270–292. DOI: 10.1137/22M1480823. arXiv: 2202.12560v2; arXiv DOI: 10.48550/arXiv.2202.12560. Exact location: Definition 3.2; Lemmas 3.3–3.4; Theorem 3.9; arXiv v2 PDF pp. 4, 5, 7. Context: Exact source provenance and theorem-to-CNNA assumption map for the analytical positivity closure. Formal status: REFERENCE_CONTEXT_INTERNAL_KERNEL_VERIFIED

EXT-REF-LEAN-006 — formalization guidance. The Lean 4 Development Team, *Lean 4 source module: Init.Core*, Lean FRO, LLC (2026). DOI status: NOT_ASSIGNED_SOFTWARE_SOURCE. Stable source: https://leanprover-community.github.io/mathlib4_docs/Init/Core.html; accessed 2026-08-05. Exact location: Subsingleton.elim declaration; Lean 4.31.0 Init.Core. Context: Official Core API used to align proposition-valued positivity witnesses. Formal status: GUIDANCE_ONLY_INTERNAL_PROOF_TERM

Open-provenance role: Reusable elimination theorem

P001 certifies the directed Schur/DtN/Kron specialization independently of the canonical birth cut. Its generality is mathematical reuse across admissible finite cuts; it does not identify Schur reduction with other open-system reductions.

026 · M004 — Response-coupled birth law B_b

Canonical node label: `026 · M004`
Semantic ID: `M004`
Current section path: `1.3.10`
Documentation tier: `D1`

Position In Derivation

M004 follows C004, O001, M003, and P001. It constructs one immutable birth instruction and exposes the exact output boundary consumed by C008.

Mathematical Contract

The zero-inclusive pure lift transports one exact nonnegative scalar over the provenance-determined support. The active law requires strict positivity. `CanonicalM004Closure realization` consumes `CanonicalM003Closure realization` and proves active instruction existence, uniqueness for each exact pair, and representative independence.

Introduction Reason

The response must determine the birth data through an explicit, bias-free map while state mutation remains a separate C008 responsibility.

Explicit Construction

```
parent -> child          sigma
child -> parent          sigma
child -> strict ancestor sigma
older sibling -> child   sigma
child -> older sibling   sigma
birth lapse              sigma
```

`canonicalM004Closure` obtains a response-steering pair from M003, derives its positivity from M003, invokes the Core `birthLaw`, and packages the result. `IsCanonicalBirthInstructionHandoff` hides response representatives and proof witnesses while retaining the Core instruction.

Invariants

- All endpoints are provenance addresses; no self-loop is emitted.
- The direct parent is excluded from the strict-ancestor updates.
- Sibling updates occur in paired orientations and canonical order.
- Every relation value and the lapse use the same exact steering value.
- No state mutation occurs in M004.

Canonicity Or Uniqueness

For each exact response-steering pair, the active instruction exists uniquely. Any two canonical handoffs have `BirthInstructionSameValue`, so representative changes cannot alter provenance support or exact scalar values.

Boundary Cases

The zero lift is exact and annihilates all values but is not an active C005 birth. Negative values are outside both domains. A root-parent birth has no strict-ancestor update.

Python Lean Cross Layer

Python and Core Lean implement the same support and exact scalar transport. The proof module `S02_CanonicalM004ClosureAndHandoff.lean` imports only the closed M003 interface and uses it directly; it does not select a parent coordinate again.

Countercheck

- Zero remains boundary data and fails active admission.
- Negative or noncanonical inputs are rejected.
- No rank, depth, load, mode, scale, baseline, clipping, fallback, or second newborn scalar occurs.
- C008 is not implemented here; the handoff proves M004 output closure without claiming state mutation.

Axiom Profile

All seven M004 closure and handoff declarations are kernel-compiled and axiom-audited. Three declaration-level interfaces use `propext` and `Quot.sound`; four constructive theorems additionally use transitive `Classical.choice`. No project-local axiom or `sorry` is admitted.

Result

M004 has a kernel-verified active-law interface and an immutable C008 handoff without external positivity or parent-index arguments.

Downstream Handoff

`canonicalBirthInstructionHandoff_exists` supplies an instruction; `canonicalBirthInstructionHandoff_sameValue` proves representative-independent output equivalence. C008 alone applies it to record/live state.

Code Anchors

python / SOURCE: `s10_m004__response_coupled_birth_law_birthlaw_b.py`

Path: `derivation/code/python/src/cnna/derivation/s01_primitive_response_coupled_finite_provenance/s03_recurrent_pre_birth_measurement_and_steering/s10_m004__response_coupled_birth_law_birthlaw_b.py`
SHA-256: `fb5ae77e13b3596f806136edb4e4854631e3ee3911e7a6baf788409d9b67691c`

Symbol	Kind	Lines
`BirthLawDomainError`	`CLASS`	57-58
`DirectedRelationUpdate`	`CLASS`	62-77
`ResponseCoupledBirthInstruction`	`CLASS`	81-136
`direct_response_lift`	`FUNCTION`	139-180
`canonical_bias_free_birth_law_inputs`	`FUNCTION`	183-221
`response_coupled_birth_law`	`FUNCTION`	224-243

python_test / TEST: test_s10_m004__response_coupled_birth_law_birthlaw_b.py

Path: derivation/code/python/tests/derivation/s01_primitive_response_coupled_finite_provenance/s03_recurrent_pre_birth_measurement_and_steering/test_s10_m004__response_coupled_birth_law_birthlaw_b.py
SHA-256: 364ae68fbcc74863b4756b271d38bf3c1444a02f2aa23db3eec4de54557f3dcf

Symbol	Kind	Lines
_bootstrap_state	FUNCTION	35-41
_state	FUNCTION	44-55
_instruction	FUNCTION	58-62
TestResponseCoupledBirthLaw	CLASS	65-201

lean_core / SOURCE: S10_M004_ResponseCoupledBirthLawBirthlawB.lean

Path: derivation/code/lean/core/src/CNNA/Derivation/S01_PrimitiveResponseCoupledFiniteProvenance/S03_RecurrentPreBirthMeasurementAndSteering/S10_M004_ResponseCoupledBirthLawBirthlawB.lean
SHA-256: 03d417a92f6591d4325ee62bda1d3d1f227456b58bfa91478b206251b721c382

Symbol	Kind	Lines
NonnegativeLiftValue	DEF	48-51
DirectedRelationUpdate	STRUCTURE	52-57
ResponseCoupledBirthInstruction	STRUCTURE	58-67
strictAncestorPorts	DEF	68-72
directRelationUpdate	DEF	73-80
parentChildUpdates	DEF	81-87
ancestorBackreactionUpdates	DEF	88-94
siblingBackreactionAux	DEF	95-105
siblingBackreactionUpdates	DEF	106-111
directResponseLift	DEF	112-122
candidateInputs	DEF	123-135
candidateInputs_admissible	THEOREM	136-143
biasFreeInputs	DEF	144-156
birthLaw	DEF	157-166
IsCanonicalBirthLaw	DEF	167-176
DirectedRelationUpdateSameValue	STRUCTURE	177-185
DirectedRelationUpdatesSameValue	INDUCTIVE	186-197
BirthInstructionSameValue	STRUCTURE	198-210
directRelationUpdate_respects_sameValue	THEOREM	211-221
ancestorAux_respects_sameValue	THEOREM	222-236
siblingAux_respects_sameValue	THEOREM	237-253
directResponseLift_respects_sameValue	THEOREM	254-277
birthLaw_respects_sameValue	THEOREM	278-295
birthLaw_exists	THEOREM	296-307
birthLaw_unique	THEOREM	308-321
responseSteeringPairs_give_same_birthLaw	THEOREM	322-342
birthLaw_parentChild_eq_directLift	THEOREM	343-354
birthLaw_lapse_eq_steering	THEOREM	355-366
directResponseLift_zero_lapse	THEOREM	367-371

lean_proof / PROOF: S02_CanonicalM004ClosureAndHandoff.lean

Path: derivation/code/lean/proofs/src/CNNAProofs/M003M004/S02_CanonicalM004ClosureAndHandoff.lean
SHA-256: 2882f029f40a3b5742e987e3b201602ba5a5dd532121818418837747e825e93e

Symbol	Kind	Lines
CanonicalM004Closure	STRUCTURE	31-67
canonicalM004Closure	THEOREM	68-108
IsCanonicalBirthInstructionHandoff	DEF	109-119
canonicalBirthInstructionHandoff_exists	THEOREM	120-131
canonicalBirthInstructionHandoff_sameValue	THEOREM	132-146
CanonicalM004ClosureContract	DEF	147-159
canonicalM004ClosureContract	THEOREM	160-167

Reference Context Retained

EXT-REF-LEAN-001 — formalization guidance. The Lean 4 Development Team, *The Lean Language Reference: Inductive Types*, Lean FRO, LLC (2026). DOI status: NOT_ASSIGNED_OFFICIAL_DOCUMENTATION. Stable source: https://lean-lang.org/doc/reference/latest/The-Type-System/Inductive-Types/; accessed 2026-07-31. Exact location: Section 4.4, constructors and generated recursors. Context: Documents the official source consulted when replacing unavailable List.Forall₂ with a module-local inductive relation. Formal status: GUIDANCE_ONLY_NO_MATHLIB_DEPENDENCY

Open-provenance role: Birth instruction before record/live mutation

M004 turns the positive response scalar into a representative-independent provenance instruction. C008 remains responsible for applying that instruction to the immutable record and mutable live channels.